



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

TRABAJO TERMINAL

**“Modelo para representar comportamientos
gravitacionales con dos cuerpos”**

PRESENTAN:

**José Emiliano Carrillo Barreiro
José Ángel Robles Otero**

DIRECTORES:

**Dr. Cesar Hernández Vasquez
Dr. Mauricio Olguín Carbajal**



Ciudad de México
16 de octubre de 2025



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA



No. de TT: 2025-B065

16 de octubre de 2025

Documento técnico

“Modelo para representar comportamientos gravitacionales con dos cuerpos”

Presentan:

José Emiliano Carrillo Barreiro.¹
José Ángel Robles Otero.²

DIRECTORES:

Dr. Cesar Hernández Vasquez
Dr. Mauricio Olguín Carbajal

Resumen:

En los últimos años, la elaboración de entornos virtuales complejos, como *REBOUND*, *Universe sandbox*, *Stellarium* y *Celestia*, han experimentado un crecimiento exponencial en sus capacidades. Más, cuando se habla de implementar mejoras en la precisión y escalabilidad dentro de sistemas de n -cuerpos celestes, por ejemplo, la simulación de nacimientos de galaxias, todavía existen dificultades, ya que, actualmente, los modelos que los simulan no permiten introducir cambios en los parámetros de los cuerpos durante su ejecución, parámetros como la velocidad, posición y masa del cuerpo, lo que impide contar con escenarios más fidedignos a los comportamientos físicos dentro de sistemas de n -cuerpos atípicos. En este proyecto, se propone solucionar este problema mediante la construcción de un modelo que permita realizar cambios al parámetro más significativo del sistema, compuesto por dos cuerpos, en tiempo de ejecución, implementando técnicas como el método multipolar rápido (FMM, por sus siglas en inglés, *Fast Multipole Method*), el *algoritmo de Barnes-Hut* y algoritmos bioinspirados. La solución que se desarrolle podría usarse para simular comportamientos físicos dentro de entornos virtuales.

Palabras clave: Modelado y Simulación de sistemas, problema de los n -cuerpos, Mecánica Celeste, y Algoritmos Bioinspirados.

Fecha: 16 de octubre de 2025

¹carrillobarjosee@gmail.com

²robles.otero.jose.angel@gmail.com

Advertencia

“Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan.”

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n Teléfono: 57296000, extensión 52000.

Resumen.

En los últimos años, el desarrollo de entornos virtuales complejos—como *REBOUND*, *Universe sandbox*, *Stellarium* y *Celestia*—ha experimentado un crecimiento exponencial en sus capacidades. Sin embargo, al abordar la estabilidad y escalabilidad en sistemas celestes de n cuerpos, por ejemplo, la simulación de nuevas formas de formación de galaxias, persisten importantes desafíos. Los modelos actuales que simulan estos fenómenos normalmente no permiten modificar parámetros clave de los cuerpos celestes—como velocidad, posición y masa—durante la ejecución. Esta limitación restringe la capacidad de generar escenarios físicamente más precisos, especialmente en sistemas atípicos de n cuerpos.

En este proyecto terminal, proponemos una solución a esta restricción mediante la construcción de un modelo que posibilita la modificación en tiempo real de la masa en un sistema de dos cuerpos. El modelo incorpora diversas técnicas de cálculo gravitatorio, incluyendo el cálculo directo y el algoritmo de Barnes-Hut, así como algoritmos bioinspirados. La solución propuesta podría emplearse para simular interacciones físicas dinámicas en entornos virtuales.

Abstract.

In recent years, the development of complex virtual environments, such as *REBOUND*, *Universe sandbox*, *Stellarium* and *Celestia*, has experienced exponential growth in capabilities. However, when addressing stability and scalability in n -body celestial systems, such as the simulation of novel forms of galaxy formation, significant challenges remain. Current models that simulate these phenomena typically do not allow changes to key parameters of celestial bodies, such as velocity, position, and mass, during runtime. This limitation restricts the ability to generate more physically accurate scenarios, especially in atypical n -body systems.

In this terminal project, we propose a solution to this limitation by constructing a model that enables modification of mass in a two-body system during execution time. The model incorporates various gravitational computation techniques, including direct calculation and the Barnes-Hut algorithm, as well as bio-inspired algorithms. The proposed solution could be used to simulate dynamic physical interactions in virtual environments.

Agradecimientos.

El presente trabajo terminal, documentado en este informe técnico, no habría sido posible sin el invaluable apoyo, la guía y la inspiración de numerosas personas a lo largo de nuestra trayectoria académica. A todos ellos, expresamos nuestro más profundo y sincero agradecimiento.

En primer lugar, deseamos reconocer a nuestros directores, el Dr. Cesar Hernández Vásquez y el Dr. Mauricio Olguín Carbajal, por su dedicación, paciencia y valioso apoyo intelectual durante este proceso. Su orientación especializada, sus críticas constructivas y su constante motivación han sido pilares fundamentales en el desarrollo de la investigación y la propuesta de solución planteada en este proyecto. Sin su confianza y disposición para compartir su conocimiento, esta primera presentación de resultados no habría alcanzado su forma actual.

Asimismo, extendemos nuestra gratitud a los miembros del comité sinodal, la Dra. Yesica Sonia Flores Meraz, el M. en C. Jesús Alfredo Martínez Nuño y el Dr. Genaro Juárez Martínez, por su tiempo, esfuerzo y compromiso al evaluar este trabajo terminal. Sus comentarios y sugerencias han enriquecido significativamente la calidad de esta investigación y seguirán siendo esenciales para su mejora.

De manera especial, agradecemos a la Dra. Lorena Chavarría Báez por su apoyo incondicional en el planteamiento del protocolo y la delimitación del proyecto. Sus aportaciones y sugerencias fueron cruciales para afinar el enfoque de esta investigación, contribuyendo a la profundidad y claridad que caracterizan el trabajo aquí presentado.

Igualmente, queremos destacar la colaboración de los Dr. Daniel Molina Pérez y M. en C. José Alberto Torres León, quienes, en su rol de consultores expertos, revisaron el propósito, los objetivos y el desarrollo de este proyecto. Su retroalimentación, validada mediante el “juicio de experto”, junto con sus valiosos comentarios, nos permitió perfeccionar las técnicas y la calidad de este informe.

Por último, pero no menos importante, rendimos homenaje a nuestras familias por su amor incondicional, paciencia y sacrificios a lo largo de este arduo camino. Su respaldo constante ha dado un significado especial a esta presentación. A través de esta primera parte del trabajo terminal, hemos enfrentado desafíos que nos han permitido crecer tanto personal como académicamente. Este proceso nos ha revelado nuevas dimensiones del conocimiento, así como la resiliencia y determinación que habitan en nosotros. A todos quienes nos han acompañado con su apoyo, guía e inspiración, les debemos una parte esencial de este logro inicial. Gracias por todo.

Índice general

Resumen.	iii
Abstract.	iv
Agradecimientos.	v
1. Introducción	1
1.1. Antecedentes	1
1.2. Planteamiento del problema	2
1.3. Propuesta de solución	2
1.4. Objetivo general	3
1.4.1. Objetivos específicos	3
1.5. Justificación	4
2. Estado del Arte	6
2.1. <i>Framework</i> <code>ode_num_int</code>	6
2.2. Representación de Objetos	7
2.3. Método n-NNN	7
2.4. Métodos Hidrodinámicos Sin Malla	8
2.5. Método híbrido <i>SPH</i> / <i>N</i> -cuerpos	9
2.6. Integrador simpléctico para problemas gravitacionales	9
2.7. Solver gravitacional híbrido TPM	10
2.8. Algoritmo TPM para simulaciones <i>N</i> -cuerpos	11
2.9. REBOUND para dinámica <i>N</i> -cuerpos	11
2.10. Modelo de Estabilidad Planetaria	12
2.11. Tabla Comparativa del Estado del Arte	12
3. Marco Teórico	14
3.1. Problema de los n cuerpos	14
3.1.1. Fundamentos Matemáticos	14
3.1.2. Contexto Histórico	15
3.1.3. Enfoques de Solución	15
3.1.4. Aplicaciones Típicas y Extendidas	15
3.2. Problema de dos cuerpos	16
3.2.1. Definición y Principio de Reducción	16
3.2.2. Fundamentos Matemáticos y Leyes de Conservación	16
3.2.3. Contexto Histórico y Las Leyes de Kepler	17

3.2.4.	Enfoques Metodológicos y Parametrización Orbital	17
3.2.5.	Aplicaciones Típicas y Conclusión	18
3.3.	Exponente de Lyapunov	18
3.3.1.	Definición y Cuantificación del Caos	18
3.3.2.	Criterio de Caos (Significado de λ_1)	18
3.3.3.	El Espectro de Lyapunov	19
3.3.4.	Cálculo y Limitaciones	19
3.4.	Integradores Simpléticos	19
3.4.1.	Definición y principios	19
3.4.2.	Contexto histórico	20
3.4.3.	Variantes representativas	20
3.4.4.	Propiedades clave	20
3.4.5.	Aplicaciones	21
3.4.6.	Ventajas	21
3.4.7.	Limitaciones	21
3.4.8.	Direcciones actuales	21
3.5.	Integrador Simplético WHFast	21
3.5.1.	Idea clave (para ingeniería)	21
3.5.2.	Qué hace rápido a WHFast	22
3.5.3.	Opciones prácticas (como hiperparámetros)	22
3.5.4.	Cuándo usarlo / cuándo evitarlo	22
3.6.	REBOUND	22
3.6.1.	Arquitectura (resumen)	23
3.6.2.	Módulos clave	23
3.6.3.	Notas	24
3.6.4.	Cuándo usarlo / evitarlo	24
3.6.5.	Aplicaciones típicas	24
3.7.	Problemas de Factibilidad y Optimización	24
3.7.1.	Formulación estándar de optimización	24
3.7.2.	Clasificación y optimización convexa	25
3.7.3.	Problemas de factibilidad	25
3.7.4.	Enfoque algorítmico: oráculos y planos cortantes	25
3.7.5.	Relación factibilidad–optimización y métodos de dos fases	25
3.8.	Algoritmos Genéticos (AGs) Simples Mono-objetivo mediante <code>pymoo</code>	26
3.8.1.	Fundamentos de los AGs canónicos	26
3.8.2.	Arquitectura modular de <code>pymoo</code>	26
3.8.3.	AG mono-objetivo en <code>pymoo</code>	26
3.8.4.	Operadores y terminación	27
3.8.5.	Ejemplo mínimo con <code>GA</code>	27
3.9.	FrameworkQt & PyQt	27
3.9.1.	Introducción a Qt	27
3.9.2.	Principios fundamentales de Qt	27
3.9.3.	Widgets, layouts y Qt Quick	29
3.9.4.	Licenciamiento y versiones	29
3.9.5.	Aplicaciones típicas	29
3.9.6.	Ventajas y desventajas de Qt	29
3.9.7.	Introducción a PyQt	29
3.9.8.	Principios fundamentales de PyQt	29

3.9.9. Aplicaciones típicas de PyQt (perfil IA)	29
3.9.10. Ventajas y desventajas de PyQt	30
3.10. Matplotlib	30
3.10.1. Introducción	30
3.10.2. Arquitectura y conceptos clave	30
3.10.3. Estilos de API: pyplot vs. OO	30
3.10.4. Capacidades de trazado y personalización	31
3.10.5. Aplicaciones típicas	31
3.10.6. Ventajas	31
3.10.7. Limitaciones	31
4. Planeación	32
4.1. Validez por Juicio de Expertos	32
4.1.1. Consulta con experto núm. 01	32
4.1.2. Consulta con experto núm. 02	35
4.2. Análisis de Requerimientos	38
4.2.1. Requerimientos Funcionales (RF)	39
4.2.2. Requerimientos No Funcionales (RNF)	40
4.3. Planteamiento del Problema de Factibilidad	41
4.3.1. Definiciones Preliminares: Variables, Parámetros y Función de Evaluación	41
4.3.2. Definición Formal Matemática del Problema de Factibilidad	43
4.3.3. Abordaje del Problema de Factibilidad mediante Formula- ción de Optimización	43
4.3.4. Consideraciones Adicionales	44
5. Análisis	46
5.1. Análisis de Procesos Internos del Programa	46
5.1.1. Proceso Interno 01: Captura Parámetros	46
5.1.2. Proceso Interno 02: Inicializar Población	50
5.1.3. Proceso Interno 03: Generar Nueva Generación	53
5.1.4. Proceso Interno 04: Crear Nueva Simulación	57
5.1.5. Proceso Interno 05: Agregar Cuerpos	60
5.1.6. Proceso Interno 06: Definir Condiciones	63
5.1.7. Proceso Interno 07: Iniciar Simulación	66
5.1.8. Proceso Interno 08: Resolver Ecuaciones	69
5.1.9. Proceso Interno 09: Actualizar Estado	73
5.1.10. Proceso Interno 10: Recolectar Datos	78
5.1.11. Proceso Interno 11: Procesar Datos	83
5.1.12. Proceso Interno 12: Generar Gráficos	90
5.1.13. Proceso Interno 13: Evaluar Fitness	95
5.1.14. Proceso Interno 14: Formar Siguiete Población	98
5.1.15. Proceso Interno 15: Mostrar Resultados	101
5.1.16. Matriz de Procesos	105
5.2. Diagrama de Actividades	111
5.2.1. Diagrama de Actividades	112
5.2.2. Descripción del Diagrama de Actividades	113
5.2.3. Aspectos Clave del Diseño Reflejados en el Diagrama . . .	115

5.3.	diccionario de datos	115
5.3.1.	Relacion de datos	117
6.	Diseño	119
6.1.	Diseño Arquitectónico	119
6.1.1.	Introducción y Objetivos de Diseño	119
6.1.2.	Arquitectura en Capas y Componentes Principales	120
6.1.3.	Interacciones Principales y Flujos de Datos y Control	124
6.1.4.	Estilo y Paradigmas Arquitectónicos	128
6.1.5.	Consideraciones Adicionales de Diseño	129
7.	Bitácora Implementación	131
7.1.	Módulo de Simulación	131
7.1.1.	Introducción	131
7.1.2.	Arquitectura de Alto Nivel	131
7.1.3.	Stack tecnológico	131
7.1.4.	<code>rebound_adapter.py</code> — Adaptador REBOUND	132
7.1.5.	<code>dynamics.py</code> — Dinámica (CPU/GPU)	135
7.1.6.	<code>lyapunov.py</code> — Estimación de Exponente de Lyapunov	143
7.1.7.	<code>__init__.py</code> — API del Paquete	150
.1.	Pruebas realizadas	151
.1.1.	Primeros pasos con REBOUND	151
.1.2.	Calculo de métricas	153
.1.3.	Primeras pruebas con algoritmos bioinspirados	153
	Bibliografía	155

Índice de figuras

5.1. Diagrama de Proceso Interno 01: Captura de Parámetros	49
5.2. Diagrama de Proceso Interno 02: Inicializar Población	52
5.3. Diagrama de Proceso Interno 03: Generar Nueva Generación	56
5.4. Diagrama de Proceso Interno 04: Crear Simulación	59
5.5. Diagrama de Proceso Interno 05: Agregar Cuerpos	62
5.6. Diagrama de Proceso Interno 06: Definir Condiciones	65
5.7. Diagrama de Proceso Interno 07: Iniciar Simulación	68
5.8. Diagrama de Proceso Interno 08: Resolver Ecuaciones	72
5.9. Diagrama de Proceso Interno 09: Actualizar Estado	77
5.10. Diagrama de Proceso Interno 10: Recolectar Datos	82
5.11. Diagrama de Proceso Interno 12: Generar Gráfico	89
5.12. Diagrama de Proceso Interno 05: Agregar Cuerpos	94
5.13. Diagrama de Proceso Interno 13: Calcular Fitness	97
5.14. Diagrama de Proceso Interno 14: Selección de Individuos	100
5.15. Diagrama de Proceso Interno 15: Mostrar Resultados	104
5.16. Diagrama de Actividades UML del proceso de simulación y optimi- zación.	112
6.1. Diagrama de Arquitectura.	119

Índice de cuadros

2.1. Resumen comparativo de proyectos y artículos relacionados con simulaciones N-cuerpos analizados en el Estado del Arte.	13
3.1. Enfoques en n cuerpos	15
3.2. Familias de integradores simplécticos para N -cuerpos (resumen). .	20
3.3. Comparativa de APIs en Matplotlib.	30
4.1. Requerimientos Funcionales	39
4.2. Requerimientos No Funcionales	40
5.1. Matriz de Procesos	106
5.2. Cuerpo celeste.	116
5.3. Simulación.	117
5.4. Métricas.	117
7.1. Parámetros de <code>ReboundSim.__init__</code>	132
7.2. Valor de retorno de <code>ReboundSim.__init__</code>	132
7.3. Parámetros de <code>setup_simulation</code>	133
7.4. Valor de retorno de <code>setup_simulation</code>	133
7.5. Parámetros de <code>integrate</code>	134
7.6. Valor de retorno de <code>integrate</code>	134
7.7. Parámetros de <code>DynamicsCPU.f</code>	136
7.8. Valor de retorno de <code>DynamicsCPU.f</code>	136
7.9. Parámetros de <code>DynamicsCPU.jac</code>	137
7.10. Valor de retorno de <code>DynamicsCPU.jac</code>	137
7.11. <code>DynamicsGPU</code> : interfaz	139
7.12. Parámetros de <code>DynamicsGPU.f</code>	139
7.13. Valor de retorno de <code>DynamicsGPU.f</code>	139
7.14. Parámetros de <code>DynamicsGPU.jac</code>	141
7.15. Valor de retorno de <code>DynamicsGPU.jac</code>	141
7.16. Parámetros de <code>mLCE_short</code>	143
7.17. Valor de retorno de <code>mLCE_short</code>	143
7.18. Parámetros y retorno de <code>mLCE_long</code>	145

List of Algorithms

CAPÍTULO 1

Introducción

1.1. Antecedentes

El desarrollo de simulaciones de n -cuerpos ha sido ampliamente investigado con el objetivo de mejorar su capacidad para modelar fenómenos complejos en entornos como la colisión de galaxias y la evolución de sistemas planetarios. Sin embargo, una limitación crítica es la incapacidad de ajustar los parámetros de los cuerpos durante la ejecución de la simulación, lo que afecta la representación precisa de eventos masivos como colisiones en las simulaciones disponibles.

El problema de los n -cuerpos¹ ha sido uno de los desafíos más complejos en la mecánica celeste, desde los trabajos pioneros de Newton en el siglo XVII. Si bien el problema de los dos cuerpos tiene una solución analítica exacta bajo ciertas condiciones, cuando se introducen factores adicionales, se requieren métodos numéricos avanzados para mejorar la precisión de las simulaciones.

Avances como el método multipolar rápido (FMM, por sus siglas en inglés, *Fast Multipole Method*)² y el algoritmo de Barnes-Hut³ han optimizado las simulaciones de n -cuerpos al reducir la complejidad computacional en el cálculo de las interacciones gravitacionales. Sin embargo, la falta de capacidad para ajustar dinámicamente las propiedades de los cuerpos durante la ejecución sigue siendo una limitación en la representación realista de eventos astronómicos.

Estudios recientes han introducido herramientas y enfoques que podrían mejorar las simulaciones de n -cuerpos, como el uso de árboles cuádruples y octales para la representación geométrica y la aplicación de inteligencia artificial en las simulaciones moleculares que podrían inspirar nuevas técnicas en simulaciones celestes. Además, se ha demostrado la eficacia de integradores simplécticos⁴ para

¹El problema de n -cuerpos se refiere al estudio del movimiento e interacción gravitacional de varios cuerpos bajo sus influencias mutuas. En la sección 3.1 se profundiza a fondo el termino.

²El FMM es un algoritmo que optimiza el cálculo de interacciones entre partículas distantes en sistemas físicos, como los gravitacionales, agrupando partículas y aproximando su influencia colectiva, lo que reduce la complejidad computacional de $O(n^2)$ a $O(n)$ o $O(n \log n)$.

³El algoritmo de Barnes-Hut es una técnica de simulación de n -cuerpos que aproxima las fuerzas gravitacionales al agrupar cuerpos distantes en una misma región del espacio, representándolos como una única masa, lo que reduce la complejidad computacional de $O(n^2)$ a $O(n \log n)$.

⁴Los integradores simplécticos son métodos numéricos utilizados para resolver ecuaciones

garantizar la estabilidad de las simulaciones durante colisiones y la paralelización para mejorar la eficiencia en simulaciones masivas.

1.2. Planteamiento del problema

Las simulaciones de sistemas de n -cuerpos celestes han sido una herramienta fundamental en la astronomía y la mecánica celeste para estudiar la evolución de sistemas planetarios, la dinámica estelar y la interacción gravitacional a gran escala. Sin embargo, una limitación crítica en los modelos actuales es la imposibilidad de modificar dinámicamente los parámetros de los cuerpos durante la ejecución de la simulación. En la mayoría de los simuladores, estos parámetros, como la masa, la energía del sistema, la posición y el tiempo de existencia de los cuerpos se definen antes de la ejecución, impidiendo la representación precisa de eventos atípicos o transitorios, como colisiones de galaxias, acreción de materia en discos protoplanetarios o cambios de masa en estrellas variables.

Uno de los problemas más importantes derivados de esta limitación es la incapacidad de ajustar la masa de los cuerpos celestes durante el lapso de duración de la simulación. En eventos como fusiones de agujeros negros o estrellas en procesos de acreción, la masa de los cuerpos cambia significativamente a lo largo del tiempo, alterando la evolución del sistema. Sin la posibilidad de modificar este parámetro de manera dinámica, los modelos actuales ofrecen solo aproximaciones estáticas que no reflejan con precisión la naturaleza de estos fenómenos.

Además, el rendimiento computacional también representa un desafío. La simulación de n -cuerpos es un problema computacionalmente costoso, ya que la cantidad de interacciones a calcular crece cuadráticamente con el número de cuerpos ($O(n^2)$), lo que hace que las simulaciones a gran escala sean prohibitivas en términos de tiempo de ejecución y recursos computacionales. Métodos como el algoritmo de Barnes-Hut y el método multipolar rápido han sido desarrollados para mitigar este problema reduciendo la cantidad de cálculos directos, pero no abordan la falta de flexibilidad en la modificación de parámetros en tiempo real.

En este contexto, la incapacidad de modificar parámetros dinámicamente en simulaciones de n -cuerpos afecta no solo a la estabilidad del sistema representado durante la simulación, respecto a los fenómenos astronómicos internos del sistema; sino también la capacidad de realizar simulaciones interactivas, algo que podría tener aplicaciones en la enseñanza, la exploración espacial y la industria del entretenimiento. La ausencia de modelos que permitan esta flexibilidad limita el alcance de las simulaciones actuales y plantea la necesidad de desarrollar enfoques más adaptativos y eficientes.

1.3. Propuesta de solución

La solución propuesta se basa en el desarrollo de un modelo que permite la modificación dinámica de parámetros durante la simulación, lo que representa una

diferenciales en sistemas dinámicos conservando las propiedades geométricas del sistema, como la conservación de la energía a largo plazo, lo que los hace especialmente adecuados para simular interacciones físicas, como las gravitacionales, en sistemas de n -cuerpos.

mejora significativa con respecto a los simuladores tradicionales, que requieren reconfiguraciones antes de cada ejecución. Este modelo integrará técnicas avanzadas para optimizar el cálculo de interacciones gravitacionales y garantizar la estabilidad de la simulación.

Para lograrlo, se emplearán distintos métodos de cálculo sobre fuerzas gravitacionales y cálculos de colisión. Respecto a los cálculos de fuerzas gravitacionales, uno de los elementos plausibles a integrar es el método multipolar rápido (FMM) y el algoritmo de Barnes-Hut, ambos ampliamente utilizados para reducir la complejidad computacional en simulaciones de n -cuerpos sin comprometer la precisión. Estas técnicas permiten agrupar cuerpos en estructuras jerárquicas, disminuyendo el número de cálculos directos y mejorando la eficiencia del sistema.

Además, la implementación de algoritmos bioinspirados, como la optimización por enjambre de partículas (PSO) y algoritmos genéticos (AGs), permitirá el ajuste dinámico de los parámetros de los cuerpos celestes, garantizando un comportamiento estable incluso en escenarios de colisión o interacciones complejas. Estas estrategias facilitarán la identificación y ajuste de valores óptimos sin necesidad de intervención manual, lo que hará que la simulación sea más adaptable a eventos inesperados.

El modelo se diseñará para ser escalable, permitiendo la modificación de los parámetros, conjunto de parámetros que solo involucra a la masa de los cuerpos dada las limitaciones de tiempo para el trabajo terminal, de hasta dos cuerpos celestes, con la posibilidad de extenderse a más cuerpos en implementaciones futuras. Su integración con entornos virtuales como Unreal Engine o motores de simulación especializados permitirá su aplicación en campos como la educación, los videojuegos y la industria aeroespacial; dicha implementación se deja para mejoras futuras al proyecto.

Para garantizar un rendimiento eficiente, el modelo se desarrollará optimizado para ejecutarse en hardware de gama media, como procesadores multinúcleo con al menos 16 GB de RAM. Esto asegurará que la solución sea accesible sin requerir infraestructuras de alto rendimiento, ampliando su potencial uso en distintos ámbitos.

Esta combinación de técnicas avanzadas y adaptabilidad en tiempo real establece una base innovadora para el desarrollo de simulaciones gravitacionales más dinámicas, eficientes y versátiles, superando las limitaciones de los modelos pre-configurados actuales.

1.4. Objetivo general

Desarrollar un modelo teórico para la simulación del problema de dos cuerpos que permita la modificación dinámica de la masa, mejorando la estabilidad local del sistema visible en la representación de sus interacciones gravitacionales y eventos asociados.

1.4.1. Objetivos específicos

- Implementar el módulo de simulación encargado de integrar los distintos procedimientos algorítmicos requeridos para obtener la descripción numérica del sistema de interacción de n cuerpos, incluyendo la aplicación de métodos de integración numérica, el cálculo de fuerzas gravitatorias y la detección de colisiones.
- Diseñar e implementar el módulo de optimización orientado al ajuste dinámico de las masas de los cuerpos que conforman el sistema de n -cuerpos, mediante el uso de algoritmos bioinspirados, con el fin de identificar el primer conjunto de valores que satisfaga las restricciones impuestas en cuanto a estabilidad y viabilidad del sistema.
- Desarrollar e implementar el modelo computacional de para la simulación dinámica de un sistema de dos cuerpos bajo interacción gravitatoria.
- Implementar el módulo de visualización gráfica para la representación dinámica del sistema simulado, mostrando su evolución temporal a lo largo de un conjunto limitado de iteraciones, a fin de apoyar la interpretación de los resultados del modelo
- Diseñar e implementar una interfaz básica que permita el ingreso estructurado de parámetros asociados a los cuerpos del sistema, diferenciando entre atributos fijos (e.g., radio) y variables susceptibles de optimización (e.g., masa), así como la definición de restricciones obligatorias y opcionales que condicionan el espacio de soluciones. Además de permitir visualizar los resultados generados por los módulos de simulación y optimización

1.5. Justificación

Este proyecto es relevante porque introduce un enfoque innovador en la simulación de sistemas gravitacionales al permitir la modificación dinámica de parámetros, superando las limitaciones de los modelos tradicionales. La elección de algoritmos avanzados, como el método multipolar rápido (FMM) y el algoritmo de Barnes-Hut, garantiza un equilibrio entre precisión y eficiencia computacional, lo que permite su aplicación en escenarios más complejos sin un costo computacional excesivo. Además, la implementación de técnicas de optimización bioinspiradas ofrece un método adaptable para ajustar el sistema en tiempo real, mejorando la fidelidad de las simulaciones.

El uso de estas tecnologías no solo optimiza el rendimiento de la simulación, sino que también sienta las bases para su posible escalabilidad a problemas de mayor complejidad, como la simulación de múltiples cuerpos. Aunque el presente trabajo se centra en el problema de dos cuerpos, su enfoque y metodología podrían aplicarse en otros ámbitos, como el modelado de interacciones gravitacionales en videojuegos y simulaciones interactivas.

Los principales beneficiarios de este proyecto serán investigadores y académicos en los campos de la astronomía, astrofísica y mecánica celeste, quienes podrán utilizar el modelo para mejorar el análisis y la comprensión de interacciones gravitacionales. Además, su posible aplicación en videojuegos y simulaciones interactivas podría impactar significativamente la industria del entretenimiento

y la educación, proporcionando herramientas más flexibles y precisas para la enseñanza y la creación de simulaciones realistas.

Asimismo, la industria aeroespacial y los organismos encargados de planificar maniobras espaciales podrían beneficiarse del modelo, ya que permitiría simular eventos dinámicos, como colisiones orbitales o ajustes en la trayectoria de satélites. Al ofrecer la posibilidad de modificar dinámicamente los parámetros de los cuerpos en simulación, esta herramienta facilitaría la planificación y optimización de maniobras espaciales, mejorando la toma de decisiones en distintos tipos de misiones.

CAPÍTULO 2

Estado del Arte

2.1. Producto 01: *Framework* `ode_num_int` para Integración Numérica

En la simulación de sistemas gravitacionales, la integración numérica de ecuaciones diferenciales ordinarias es clave para describir con precisión el movimiento bajo fuerzas mutuas. El *framework* `ode_num_int` [1] ofrece un entorno en C++11 pensado para el desarrollo y evaluación de métodos de integración personalizados, destacando por su arquitectura modular y su orientación investigativa.

Su diseño se apoya en tres componentes principales:

1. Una infraestructura común con patrón observador, factorías, gestión de callbacks y utilidades de temporización.
2. Una capa de álgebra lineal basada en *templates* para vectores y matrices dispersas con soporte LU.
3. *Solvers* explícitos e implícitos para EDOs, iteradores tipo Newton y controladores de tamaño de paso. Esta modularidad facilita la experimentación con distintos métodos y la combinación de piezas a medida.

Entre sus particularidades, `ode_num_int` permite monitoreo de rendimiento integrado y soporte para métodos explícitos, implícitos y de extrapolación, resultando útil en simulaciones de gran complejidad física. Por ejemplo, en la dinámica de transmisiones continuamente variables (3 600 variables de estado), se comprobó que el método del trapecio con paso 10^{-5} alcanzaba similar precisión al RK4 con paso 10^{-8} , sugiriendo mejoras en eficiencia computacional.

Las ventajas de este *framework* incluyen su alta extensibilidad, enfoque en rendimiento y licencia GNU GPL, que favorecen el desarrollo colaborativo; sus limitaciones residen en la escala media para la que está diseñado, lo que puede no ser óptimo para problemas masivos de n -cuerpos, y en la necesidad de un conocimiento profundo de C++. Actualmente, se centra en métodos de un solo paso, aunque se prevé ampliar esta capacidad.

Para el trabajo terminal que nos concierne, el modelo de simulación gravitacional de dos cuerpos con masas dinámicos, `ode_num_int` se perfila como complemento,

pues combina integradores personalizables con herramientas de monitoreo, y puede integrarse con técnicas como FMM y Barnes–Hut para calcular fuerzas eficientemente, permitiendo modificar parámetros en tiempo de ejecución según los objetivos del modelo.

2.2. Producto 02: Representación de Objetos mediante *Quadrees* y *Octrees* de División No Minimal

La representación eficiente de estructuras geométricas es necesaria en la simulación de sistemas gravitacionales, ya que impacta directamente la precisión en el modelado de trayectorias y colisiones. El método de *quadrees* y *octrees* de división no minimal, descrito en “Object Representation by Means of Nonminimal Division *Quadrees* and *Octrees*” [2], propone una estructura jerárquica que subdivide el espacio recursivamente, optimizando tanto la memoria como la fidelidad geométrica.

En lugar de las etiquetas tradicionales WHITE, BLACK y GRAY, este enfoque añade nodos EDGE y VERTEX para capturar segmentos de borde y vértices, lo que mejora la representación de contornos complejos y facilita operaciones booleanas y conversiones entre formatos arbóreos y de borde. La subdivisión no minimal evita particiones innecesarias en regiones homogéneas, reduciendo el uso de memoria y manteniendo una complejidad algorítmica lineal, adecuada para aplicaciones en CAD, modelado por computadora y simulaciones gráficas.

Este método ofrece ventajas como la eficiencia en operaciones de intersección, unión y diferencia, y una notable precisión en detalles de borde, pero conlleva una implementación más compleja y una posible pérdida de resolución en detalles de escala muy pequeña debido a la naturaleza discreta de la subdivisión. Para el proyecto de simulación gravitacional de dos cuerpos, la integración de estas estructuras permitiría optimizar la geometría de los cuerpos celestes, complementando los cálculos de fuerza con FMM y Barnes–Hut y mejorando el rendimiento al manejar dinámicamente parámetros como masa, posición y velocidad.

A pesar de no haber implementado directamente la técnica, su lógica sugiere una alta viabilidad para simulaciones que exijan precisión geométrica y eficiencia de memoria, representando un componente valioso para el modelado avanzado de interacciones cercanas o colisiones en sistemas de dos cuerpos.

2.3. Método de Red de n-Vecinos Más Cercanos (n-NNN) para Simulaciones Moleculares

En las simulaciones de sistemas dinámicos, la eficiencia computacional y la precisión física son esenciales. El método de Red de n-Vecinos Más Cercanos (n-NNN), presentado en “Artificial Intelligent Molecular Dynamics and Hamiltonian Surgery” [3], ofrece una alternativa a los enfoques tradicionales basados en Hamiltonianos aditivos por pares. En lugar de recomputar exhaustivamente todas las interacciones en cada paso, n-NNN organiza la información de fuerzas en

matrices multidimensionales que representan el vecindario local de cada partícula, emulando una red neuronal. La técnica de “cirugía Hamiltoniana” permite incorporar selectivamente términos de orden superior necesarios para mantener la precisión estructural, reduciendo significativamente la carga computacional.

Este método ha demostrado ser eficaz en sistemas complejos como líquidos de Lennard-Jones [4] y cristales iónicos [5], preservando la fidelidad física al considerar un número limitado de vecinos. Su independencia respecto a la no aditividad facilita la inclusión de interacciones de múltiples cuerpos sin un incremento exponencial del tiempo de cálculo. No obstante, su validación se limita a entornos relativamente sencillos, y la precisión depende críticamente de la elección del número de vecinos y de la función generadora, lo que puede complicar su traslado a sistemas altamente variables.

Para el presente trabajo terminal de simulación gravitacional de dos cuerpos con parámetros dinámicos, el enfoque n -NNN, nos ayuda a visualizar estrategias de optimización análogas, combinándose con técnicas como el Método Multipolar Rápido (FMM) y el algoritmo de Barnes–Hut para el cálculo de fuerzas. La capacidad de ajustar dinámicamente los términos relevantes mediante cirugía Hamiltoniana se alinea con el objetivo de gestionar cambios en masa, posición y velocidad durante la ejecución, prometiendo simulaciones más rápidas y adaptativas sin sacrificar la precisión.

2.4. Implementación de Métodos Hidrodinámicos Sin Malla en PKDGRAV3 para Simulaciones Cosmológicas

El artículo de Alonso Asensio [6] presenta la integración de métodos hidrodinámicos sin malla en el código PKDGRAV3, originalmente orientado a simulaciones cosmológicas. En lugar de depender de una malla estructurada para resolver las ecuaciones hidrodinámicas, se emplean partículas con esquemas *Meshless Finite Mass* (MFM) y *Meshless Finite Volume* (MFV) que utilizan algoritmos de búsqueda de vecinos y paralelización avanzada. Esta aproximación adaptativa permite capturar choques y discontinuidades sin una rejilla fija, reduciendo la complejidad computacional y manteniendo la precisión física.

Los métodos sin malla en PKDGRAV3 destacan por su capacidad para manejar grandes contrastes de densidad y fenómenos dinámicos con coste de cálculo sublineal respecto al número total de elementos. La discretización basada en vecinos más cercanos facilita la escalabilidad en simulaciones de formación de estructuras a gran escala, mejorando el rendimiento por medio de optimizaciones específicas de paralelismo.

Entre las principales ventajas se encuentran la resolución adaptativa sin necesidad de remallado, la escalabilidad para simulaciones a gran volumen y la fidelidad en la conservación de cantidades físicas. Como desventajas, la validación de estos métodos se ha centrado en contextos cosmológicos específicos, y su eficacia depende de la elección óptima de parámetros de búsqueda de vecinos y suavizado, lo que podría requerir calibración adicional en entornos no estudiados.

Para el proyecto de simulación gravitacional de dos cuerpos con masas dinámicas, la filosofía de discretización flexible de MFM/MFV puede inspirar nuevas estrategias de cálculo de fuerzas. Combinado con técnicas como el Método Multipolar Rápido y Barnes–Hut, este enfoque sin malla ofrece una vía para optimizar simulaciones en tiempo real, adaptando la resolución según cambios en masa, posición y velocidad de los cuerpos durante la ejecución.

2.5. Método híbrido *SPH*/*N*-cuerpos para simulaciones de cúmulos estelares

El método híbrido presentado en el artículo combina *Smoothed Particle Hydrodynamics* (SPH) con simulaciones *n*-cuerpos dentro del código *SEREN*, integrando la evolución del gas y la dinámica estelar en un único esquema Lagrangiano que conserva energía y momento. Para calcular la gravedad de partículas gaseosas autogravitantes se utiliza el árbol Barnes–Hut junto con el criterio de apertura multipolar (MAC), mejorando la precisión de la fuerza y reduciendo los errores energéticos acumulativos. Además, el cálculo del jerk gravitacional en las estrellas y el block timestepping adaptativo permiten ajustar dinámicamente los pasos de integración según la aceleración de cada partícula, optimizando el rendimiento computacional.

Entre las ventajas destacan la interacción realista entre gas y estrellas sin separar componentes, la eficiencia del árbol Barnes–Hut y del MAC para reducir la complejidad computacional, y la adaptabilidad temporal que equilibra precisión y coste. No obstante, sigue siendo más costoso que una simulación *N*-body pura, carece de tratamiento avanzado para sistemas binarios y su fidelidad depende críticamente de la calibración de parámetros numéricos.

Para el trabajo terminal que engloba este reporte, donde se trata la simulación gravitacional de dos cuerpos con parámetros dinámicos, el enfoque que ofrece este artículo nos da un marco sólido sobre: su uso combinado de Barnes–Hut, MAC y block timestepping puede integrarse con el Método Multipolar Rápido (FMM) para acelerar cálculos de fuerza ajustables en tiempo real, y la inclusión de algoritmos bioinspirados podría automatizar la optimización de parámetros como distribución de masa y criterios de resolución, proporcionando simulaciones adaptativas y de alta precisión.

2.6. Integrador simpléctico para problemas gravitacionales *N*-cuerpos colisionables

Se presenta un nuevo integrador simpléctico, reversible en el tiempo y de segundo orden, diseñado para sistemas gravitacionales *N*-cuerpos con colisiones. Inspirado en el método no simpléctico SAKURA de Gonçalves Ferrari, el esquema descompone el problema general en parejas de cuerpos resolviendo cada par con solucionadores de Kepler, preservando con precisión de máquina nueve integrales del movimiento y la estructura de fase a largo plazo. Mediante composición de Yoshida es posible elevar el orden sin sacrificar la simplécticidad, y al emplear un paso de tiempo fijo se evita la pérdida de reversibilidad y la degradación en la

conservación de cantidades físicas.

En pruebas comparativas con el método Hermite de cuarto orden y otros integradores simplécticos de segundo orden, el nuevo integrador mostró igual o mejor desempeño, destacando su estabilidad y precisión en escenarios con colisiones frecuentes. Esto lo hace adecuado para estudios de cúmulos estelares densos, núcleos galácticos activos y discos protoplanetarios, donde las colisiones son relevantes.

Las principales ventajas son la conservación exacta de propiedades simplécticas y de los integrales de movimiento, así como su extensión a órdenes superiores. Su limitación radica en la necesidad de un paso de tiempo fijo, lo que puede restar flexibilidad frente a métodos con paso adaptativo, y en un mayor coste computacional por su complejidad estructural. Como alternativa robusta para simulaciones a largo plazo, este integrador ofrece un equilibrio excepcional entre estabilidad numérica y fidelidad física.

2.7. Solver gravitacional híbrido TPM para simulaciones de N-cuerpos en arquitecturas paralelas

El método TPM (*Tree-Particle-Mesh*) combina partículas en malla (*PM*) y técnicas jerárquicas de árbol (*TREE*) para simulaciones gravitacionales en arquitecturas paralelas, adaptando el tratamiento de cada partícula a su densidad local. En regiones de baja densidad se emplea el enfoque PM para el cálculo de fuerzas de largo alcance, mientras que en zonas sobredensas se utiliza un árbol que aplica PM para contribuciones externas y TREE para las internas, logrando alta resolución espacial sin sacrificar eficiencia global.

La implementación logra escalabilidad casi lineal en sistemas como IBM SP2 con 32 procesadores (80 % de eficiencia y 95 % del tiempo de CPU en cálculo) mediante la distribución equilibrada de carga y memoria en plataformas SP1, SGI Challenge y clusters de estaciones de trabajo. Este diseño paralelo optimiza la relación cómputo/comunicación, permitiendo simular decenas de millones de partículas con balance óptimo entre precisión y rendimiento.

TPM es idóneo para estudios de formación de estructuras cósmicas, evolución de cúmulos y formación de galaxias, así como para la dinámica de materia oscura y subestructuras en volúmenes cosmológicos, al ofrecer alta resolución local donde se requiera y bajo costo computacional en el resto del dominio. Su principal ventaja es la combinación de la eficiencia PM con la resolución TREE y la integración multiescala en el tiempo; entre sus limitaciones figura la complejidad de sincronización y gestión de fronteras, además de la necesidad de afinar parámetros en escenarios muy dinámicos.

Para el trabajo termianal que aborla la temática de simulación gravitacional de dos cuerpos con parámetros dinámicos, el solver TPM propone un marco multiescala y modular que puede integrarse con métodos como FMM o Barnes–Hut y algoritmos bioinspirados, optimizando la asignación de recursos según densidad local y permitiendo simulaciones adaptativas de alta fidelidad en tiempo real.

2.8. Algoritmo Tree Particle-Mesh (TPM) para simulaciones N-cuerpos cosmológicas

El algoritmo Tree Particle-Mesh (TPM) integra el método jerárquico de árbol para el cálculo de fuerzas en regiones de alta densidad con el esquema Particle-Mesh (PM) en zonas de baja densidad, logrando un equilibrio entre precisión y eficiencia. Mediante una descomposición del dominio basada en la densidad, las regiones densas se resuelven con árbol—incluyendo fuerzas de marea externas—mientras que el resto emplea PM con pasos de tiempo más amplios.

La adopción de escalas temporales diferenciadas—grandes para PM y adaptativas para las regiones de árbol—facilita la integración eficiente de la dinámica. La independencia de los subdominios de árbol permite una paralelización natural mediante paso de mensajes, lo que lo hace ideal para entornos de cómputo distribuido y arquitecturas de alto rendimiento.

TPM ha sido usado con éxito en formaciones de estructuras cósmicas—cúmulos de galaxias, halos de materia oscura y la red de filamentos—y puede extenderse a dinámicas granulares o de plasma cuando la distribución de densidad presente contrastes bien definidos. Su principal ventaja es la asignación de resolución donde más se necesita sin sacrificar eficiencia global, aunque su implementación con múltiples escalas temporales y dominios segmentados suele ser más compleja y puede perder beneficios en entornos sin claros contrastes de densidad.

Para el proyecto de simulación gravitacional de dos cuerpos con parámetros dinámicos, TPM ofrece un marco multiescala y modular que puede combinarse con Barnes-Hut, FMM o algoritmos adaptativos, permitiendo simulaciones en tiempo real que ajusten la resolución y asignación de recursos de acuerdo con la densidad local de los cuerpos.

2.9. Código REBOUND para dinámica N-cuerpos colisionante y no colisionante

REBOUND [7] es un código abierto y modular para dinámica N-cuerpos, especializado en colisiones (anillos planetarios, planetesimales) y en dinámica clásica sin colisiones. Su arquitectura permite escoger entre tres integradores simplécticos—*leap-frog*, SEI y Wisdom-Holman—, y gestionar condiciones de frontera abiertas, periódicas y de *shearing-sheet*, esenciales para anillos planetarios y discos protoplanetarios.

Para el cálculo gravitacional y de colisiones, REBOUND emplea Barnes-Hut con paralelización MPI y OpenMP mediante descomposición estática de dominio y árboles distribuidos. Su diseño facilita combinar integradores, detectores de colisiones tipo *plane-sweep* y distintos esquemas de frontera, ofreciendo alta escalabilidad desde PCs hasta supercomputadoras.

REBOUND ha sido aplicado con éxito en estudios de anillos de Saturno, formación de planetesimales y discos de escombros, así como en simulaciones puramente gravitacionales de cúmulos estelares. Su principal fortaleza radica en la flexibilidad

y el rendimiento en entornos bidimensionales y tridimensionales, aunque la curva de aprendizaje y la comunicación en grandes desplegados pueden ser un reto.

Para este trabajo terminal atañado a la simulación gravitacional de dos cuerpos con parámetros dinámicos, REBOUND ofrece un marco robusto: su modularidad permite integrar FMM, Barnes–Hut o algoritmos adaptativos, y sus esquemas de frontera facilitan ensayar configuraciones rotacionales o periódicas, todo ello escalando eficientemente en arquitecturas paralelas.

2.10. Modelo de Simulación de Estabilidad Planetaria Basado en Uniformidad de Masas

El trabajo de Wu, Jin y Steffen en “Enhanced Stability in Planetary Systems with Similar Masses” [8] investiga cómo la uniformidad de las masas en sistemas multiplanetarios influye en su estabilidad a largo plazo. Usando el código *REBOUND* con el integrador híbrido *Mercurius* [7][9], simulan ocho planetas en órbitas circulares y coplanares alrededor de una estrella solar, manteniendo constante la masa total ($30 M_{\oplus}$, con exploraciones también a 10 y $100 M_{\oplus}$). La separación entre planetas, K , se mide en radios de Hill mutuo y varía entre 3 y 10. La uniformidad de masas se cuantifica mediante el índice de Gini ajustado $\mathcal{G}_m$ ($0 =$ masas idénticas, $1 =$ una masa dominante). Cada sistema se integra hasta un encuentro cercano (separación menor al radio de Hill) o 10^8 años, registrando el tiempo de inestabilidad normalizado t/t_0 .

Los resultados muestran una fuerte anticorrelación entre $\log(t/t_0)$ y $\mathcal{G}_m$ para sistemas no resonantes con $K > 4$ (por ejemplo, Spearman $r = -0.69$, $p < 10^{-4}$ para $K = 6.5$), indicando que mayor uniformidad promueve estabilidad. Cerca de resonancias de movimiento medio (e.g., 4:3, 5:4) esta relación se atenúa o se vuelve más compleja. En sistemas “bien ordenados” alrededor de la MMR 5:4 ($K \approx 7.5$), la estabilidad máxima ocurre para $0.2 < \mathcal{G}_m < 0.4$, lo cual coincide mejor con observaciones de pares resonantes. El estudio utiliza la equipartición de energía aleatoria como mecanismo físico para explicar la mayor excitación de excentricidades en planetas de masa inferior.

Sin embargo, el modelo detiene la simulación en el primer encuentro cercano, impidiendo analizar la evolución posterior (colisiones, eyecciones, reconfiguraciones). Más importante para nuestro proyecto, el sistema no permite modificar parámetros (masa) durante la ejecución, operando únicamente con condiciones iniciales fijas. Esto lo hace inadecuado para escenarios que requieran cambios paramétricos en tiempo de ejecución, como acreción de masa o introducción/eliminación de cuerpos. A pesar de su valor como referencia para estabilidad pasiva, nuestro enfoque exige una arquitectura que soporte modificaciones dinámicas de parámetros, más allá de la evolución puramente gravitacional pasiva descrita en [8].

2.11. Tabla Comparativa del Estado del Arte

Tabla 2.1: Resumen comparativo de proyectos y artículos relacionados con simulaciones N-cuerpos analizados en el Estado del Arte.

Producto/Método	Enfoque Principal y Características Distintivas	Escalabilidad	Usa IA	Cambios Dinámicos*
Framework ode_num_int ^a	Framework C++11 modular (template-based) para <i>desarrollo y prueba</i> de integradores numéricos (EDOs); monitoreo de rendimiento. Orientado a sistemas de escala media.	Media/Limitada	NO	NO
Representación Geométrica ^b	Representación eficiente de geometría 2D/3D usando Quadrees/Octrees <i>no minimales</i> (nodos EDGE/VERTEX); optimiza memoria y operaciones booleanas. No es un simulador dinámico.	Alta (Geom.)**	NO	NO
Método n-NNN ^c	Simulación molecular usando Red de n-Vecinos Cercanos (matrices multidimensionales) y <i>cirugía Hamiltoniana</i> ; inspirado en IA para evitar cálculo completo de interacciones.	Alta	SÍ	NO
PKDGRAV3 (Hidrodinámica) ^d	Métodos hidrodinámicos <i>sin malla</i> (MFM/MFV) en PKDGRAV3; enfoque adaptativo basado en vecinos, bueno para altos contrastes de densidad (cosmología).	Alta	NO	NO
Método Híbrido SPH/N-body ^e	Combina SPH (gas) y N-body (estrellas) usando árbol Barnes-Hut con <i>criterio MAC</i> y <i>block timestepping</i> para interacciones gas-estrella.	Alta	NO	NO
Integrador Simpléctico ^f	Integrador simpléctico (orden 2+, reversible) basado en descomposición Kepler para problemas N-cuerpos <i>colisionables</i> ; preserva estructura de fase. Requiere paso fijo.	Media	NO	NO
Solver Híbrido TPM ^g	Combina Particle-Mesh (largo alcance) y Tree (corto alcance), asignando tratamiento <i>dinámicamente según densidad local</i> ; optimizado para arquitecturas paralelas.	Alta	NO	NO
Algoritmo TPM ^h	Tree-Particle-Mesh basado en <i>descomposición de dominio por densidad</i> ; usa árbol para regiones densas y PM para el resto; integración multi-escala temporal.	Alta	NO	NO
REBOUND ⁱ	Código N-cuerpos <i>modular</i> y abierto; incluye integradores simplécticos (WHFast, SEI), Barnes-Hut, manejo de colisiones y condiciones de frontera diversas. Paralelizable (MPI/OpenMP).	Alta	NO	NO
Modelo Estabilidad Planetaria ^j	<i>Estudio teórico</i> de estabilidad N-cuerpos (usando REBOUND) enfocado en correlación entre <i>uniformidad de masa</i> (índice Gini) y tiempo de inestabilidad. No simula eventos dinámicos internos.	Alta (Estudio)	NO	NO
Solución Propuesta	Combina FMM/Barnes-Hut para cálculo gravitacional eficiente con Algoritmos Bioinspirados para la optimización y ajuste dinámico de parámetros (masa) durante la simulación.	Alta	SÍ	SÍ

* **Cambios Dinámicos:** Capacidad de modificar parámetros clave durante la ejecución

^a Sección 2.1: Framework ode_num_int para Integración Numérica

^b Sección 2.2: Representación de Objetos mediante Quadrees y Octrees de División No Minimal

^c Sección 2.3: Método de Red de n-Vecinos Más Cercanos (n-NNN) para Simulaciones Moleculares

^d Sección 2.4: Implementación de Métodos Hidrodinámicos Sin Malla en PKDGRAV3 para Simulaciones Cosmológicas

^e Sección 2.5: Método híbrido SPH/N-cuerpos para simulaciones de cúmulos estelares

^f Sección 2.6: Integrador simpléctico para problemas gravitacionales N-cuerpos colisionables

^j Sección 2.7: Solver gravitacional híbrido TPM para simulaciones de N-cuerpos en arquitecturas paralelas

^h Sección 2.8: Algoritmo Tree Particle-Mesh (TPM) para simulaciones N-cuerpos cosmológicas

ⁱ Sección 2.9: Código REBOUND para dinámica N-cuerpos colisionante y no colisionante

^j Sección 2.10: Algoritmo TPM (Bode & Ostriker 2000)

** Escalabilidad: "Alta (Geom.)" = específica para operaciones geométricas

CAPÍTULO 3

Marco Teórico

3.1. Problema de los n cuerpos

El **problema de n cuerpos** es un desafío fundamental en la **mecánica celeste** y la **astrofísica** que estudia el movimiento de n objetos (como planetas o estrellas) que interactúan exclusivamente bajo su **atracción gravitacional mutua**, de acuerdo con las leyes de Newton. La distinción clave reside en el número de cuerpos: para $n = 2$ (el problema de Kepler), existe una solución analítica exacta, generalmente órbitas elípticas. Sin embargo, para $n \geq 3$ (e.g., el sistema Tierra-Luna-Sol), el problema carece de una solución general analítica, presentando a menudo un comportamiento **caótico** y requiriendo predominantemente **métodos de simulación numérica**.

El análisis presentado a continuación se enfoca en los principios fundamentales, los enfoques de solución (analíticos y numéricos) y las amplias aplicaciones de este sistema dinámico.

3.1.1. Fundamentos Matemáticos

El problema de n cuerpos se rige por la **Ley de Gravitación Universal** de Newton, que define la magnitud de la fuerza entre dos cuerpos (m_i y m_j) a una distancia r :

$$F = \frac{Gm_i m_j}{r^2}$$

donde G es la constante gravitacional. Al aplicar la segunda ley de Newton ($\vec{F} = m\vec{a}$), la **aceleración** $\vec{a}_i$ de un cuerpo i es el resultado vectorial de la suma de las fuerzas gravitacionales ejercidas por los demás j cuerpos. Este principio se expresa como el siguiente sistema de $6n$ ecuaciones diferenciales ordinarias (tres coordenadas de posición y tres de velocidad por cuerpo):

$$\vec{a}_i = \frac{d^2 \vec{r}_i}{dt^2} = \sum_{j \neq i} \frac{Gm_j (\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \quad (3.1)$$

La dificultad de este sistema radica en su naturaleza **no lineal**, que impide, salvo en el caso $n = 2$, una solución general cerrada.

3.1.2. Contexto Histórico

El origen del problema de n cuerpos se vincula directamente a la formulación de la Ley de Gravitación Universal por **Isaac Newton** (1687). Los hitos clave en su estudio incluyen:

- **Problema de tres cuerpos:** A finales del siglo XIX, **Henri Poincaré** (1889) demostró la no integrabilidad analítica del problema de $n = 3$, lo que llevó al descubrimiento del **caos determinista** en sistemas dinámicos.
- **Solución convergente:** A principios del siglo XX, **Karl Fritiof Sundman** proporcionó una solución teórica para $n = 3$ utilizando series infinitas convergentes, aunque su convergencia es demasiado lenta para ser práctica en cálculos reales.

3.1.3. Enfoques de Solución

Ante la ausencia de una solución analítica general, la resolución del problema se aborda mediante distintos enfoques, que varían en precisión, coste computacional (complejidad) y el número de cuerpos (n).

Tabla 3.1: Tabla de enfoques principales a la hora de resolver problemas de n cuerpos

Enfoque	Descripción	Complejidad	Referencia
Solución Analítica ($n=2$)	Método que resuelve el problema de Kepler para dos cuerpos. Proporciona soluciones exactas para sistemas de dos objetos.	$O(1)$	[1]
Integración Numérica	Utiliza métodos como Runge-Kutta o el Leapfrog para simular movimientos. Permite aproximaciones numéricas para sistemas complejos con coste cuadrático.	$O(n^2)$	[5]
Algoritmo de Barnes-Hut	Método de agrupamiento jerárquico que modela regiones distantes como un centro de masa. Reduce la complejidad para simulaciones de n muy grande.	$O(n \log n)$	[6]

3.1.4. Aplicaciones Típicas y Extendidas

El problema de n cuerpos es central en la **simulación de sistemas dinámicos** a diversas escalas:

Aplicaciones Astronómicas (Típicas)

- **Dinámica Planetaria:** Predicción de trayectorias en sistemas solares y estabilidad orbital.
- **Evolución de Cúmulos Estelares:** Simulación de la dinámica interna de cúmulos globulares (donde $n \sim 10^6$) y abiertos.
- **Formación de Galaxias:** Modelado de la interacción de miles de millones de partículas para entender la estructura a gran escala del universo ($n \sim 10^9$).

Aplicaciones Fuera de la Astronomía (Extendidas)

- **Dinámica Molecular (MD):** Aunque la fuerza de interacción no es estrictamente newtoniana, la estructura computacional (el cálculo de interacciones entre n partículas) es análoga al problema de n cuerpos, utilizándose para simular el movimiento de átomos y moléculas (e.g., proteínas).
- **Computación Gráfica:** Para simular efectos de partículas y fluidos que requieren interacción masiva.

3.2. Problema de dos cuerpos

El **problema de dos cuerpos** es el caso fundamental y soluble del problema de n cuerpos. Constituye un pilar de la **mecánica clásica** y la **mecánica celeste**, al estudiar el movimiento de dos masas puntuales que interactúan exclusivamente mediante una fuerza central, típicamente la **gravitación newtoniana**. A diferencia de sistemas más complejos ($n \geq 3$), este problema posee una **solución analítica exacta** que describe las trayectorias orbitales como **secciones cónicas**.

Su trascendencia radica en que permite una predicción precisa de las órbitas y sirve como base para los **métodos de perturbación**, esenciales para analizar sistemas reales como los planetarios y satelitales.

3.2.1. Definición y Principio de Reducción

El problema se define como la dinámica de dos cuerpos (m_1 y m_2) cuya interacción está gobernada por la Ley de Gravitación Universal:

$$F = \frac{Gm_1m_2}{r^2} \quad (3.2)$$

donde G es la constante gravitacional y r la distancia entre las masas.

El principal enfoque metodológico para su solución es la **reducción a un problema de un solo cuerpo** bajo una fuerza central. Esto se logra al cambiar el sistema de referencia al centro de masa y definir el concepto de **masa reducida** (μ):

$$\mu = \frac{m_1m_2}{m_1 + m_2} \quad (3.3)$$

Esta transformación permite modelar el movimiento relativo de m_1 y m_2 como el movimiento de una única partícula de masa μ orbitando un centro fijo de fuerza.

3.2.2. Fundamentos Matemáticos y Leyes de Conservación

La naturaleza soluble del problema de dos cuerpos se debe a la existencia de **integrales de movimiento** (cantidades conservadas). Específicamente, se conservan la **energía total** (E) y el **momento angular** (L).

La dinámica relativa está regida por la siguiente ecuación diferencial, que es una simplificación del sistema general de n cuerpos:

$$\mu \frac{d^2 \mathbf{r}}{dt^2} = -\frac{Gm_1m_2}{r^3} \mathbf{r} \quad (3.4)$$

donde $\mathbf{r} = \mathbf{r}_1 - \mathbf{r}_2$ es el vector de posición relativa.

La integración de esta ecuación, junto con la conservación del momento angular, resulta en la ecuación de la órbita, cuya solución general en coordenadas polares $r(\theta)$ es una **sección cónica**, dada por:

$$r(\theta) = \frac{p}{1 + e \cos \theta} \quad (3.5)$$

donde p es el **semilatus rectum** y e es la **excentricidad** (determinando el tipo de sección cónica: $e < 1$ para elipses, $e = 1$ para parábolas y $e > 1$ para hipérbolas).

3.2.3. Contexto Histórico y Las Leyes de Kepler

El fundamento teórico fue establecido por **Isaac Newton** (1687) en *Philosophiæ Naturalis Principia Mathematica*, quien demostró que la ley de la fuerza inversa del cuadrado implica trayectorias cónicas.

El estudio se correlaciona directamente con las tres **Leyes del Movimiento Planetario** de Johannes Kepler (principios del siglo XVII), las cuales son una consecuencia directa de la solución analítica del problema de dos cuerpos:

1. **Primera Ley (Órbitas):** La trayectoria es una elipse con el cuerpo central en uno de sus focos.
2. **Segunda Ley (Áreas):** El radio vector barre áreas iguales en tiempos iguales (conservación del momento angular).
3. **Tercera Ley (Periodo):** El cuadrado del periodo orbital (T^2) es proporcional al cubo del semieje mayor (a^3).

3.2.4. Enfoques Metodológicos y Parametrización Orbital

Si bien la solución analítica existe, distintos enfoques son usados para su representación y aplicación:

- **Elementos Orbitales Clásicos:** Es el método estándar en astrodinámica. Utiliza seis parámetros (semieje mayor, excentricidad, inclinación, longitud del nodo ascendente, argumento del periastro y anomalía verdadera/media) para definir la órbita de manera geométrica en el espacio. Son ideales para el análisis geométrico y para la propagación de órbitas.
- **Formulación Vectorial (Posición y Velocidad):** Emplea los vectores de posición ($\mathbf{r}$) y velocidad ($\mathbf{v}$) en un instante dado. Es útil para la integración numérica inicial y para el análisis dinámico directo, especialmente cuando se requiere un sistema de coordenadas cartesiano.
- **Formulación Hamiltoniana:** Utiliza variables canónicas (coordenadas y momentos generalizados), lo que es particularmente adecuado para el estudio de **perturbaciones** y para la transición a la teoría de n cuerpos mediante métodos analíticos avanzados.

3.2.5. Aplicaciones Típicas y Conclusión

El problema de dos cuerpos es indispensable en:

- **Astrodinámica:** Predicción de la órbita de satélites artificiales (tierra-satélite) y el diseño de trayectorias interplanetarias.
- **Astrofísica:** Análisis de la dinámica y la determinación de masas en **sistemas estelares binarios**.
- **Mecánica Celeste:** Cálculo de órbitas de cometas, planetas y asteroides bajo la simplificación del cuerpo central dominante (e.g., Sol-Planeta).

La principal **ventaja** es su solución analítica exacta, que lo convierte en el modelo base para cualquier sistema gravitacional. Su principal **limitación** es la restricción estricta a dos masas, lo que obliga a recurrir a la **teoría de perturbaciones** para abordar la complejidad de los sistemas reales (Problema de n cuerpos).

3.3. Exponente de Lyapunov

El **Exponente de Lyapunov** (λ) es una medida fundamental en la teoría de sistemas dinámicos para cuantificar la **sensibilidad a las condiciones iniciales** y el grado de **caos** en un sistema. Mide la tasa promedio de **separación o convergencia exponencial** de trayectorias infinitesimalmente cercanas en el espacio de fases. Si el sistema es caótico, esta tasa es positiva.

3.3.1. Definición y Cuantificación del Caos

El concepto se basa en la comparación de la evolución temporal de dos trayectorias $\mathbf{z}(t)$ y $\mathbf{z}'(t)$ que inician con una pequeña perturbación $\delta_0 = \|\mathbf{z}(0) - \mathbf{z}'(0)\|$. La perturbación se propaga en el tiempo como $\delta(t) \approx \delta_0 e^{\lambda t}$.

El **Exponente de Lyapunov Máximo** (λ_1) es la métrica principal. Formalmente, se define como:

$$\lambda_1 = \lim_{t \rightarrow \infty} \lim_{\|\delta_0\| \rightarrow 0} \frac{1}{t} \ln \left(\frac{\|\delta(t)\|}{\|\delta_0\|} \right) \quad (3.6)$$

donde el límite $\lim_{\|\delta_0\| \rightarrow 0}$ se realiza mediante la **linealización** del sistema (ecuación variacional) alrededor de la trayectoria $\mathbf{z}(t)$.

3.3.2. Criterio de Caos (Significado de λ_1)

El signo del Exponente de Lyapunov Máximo establece un criterio claro para la predictibilidad del sistema:

- **Caos** ($\lambda_1 > 0$): Las trayectorias divergen exponencialmente. El sistema es extremadamente sensible a cualquier error en la condición inicial, volviéndose impredecible a largo plazo.
- **Movimiento Regular** ($\lambda_1 = 0$): La divergencia es lineal o polinómica. Característico de sistemas conservativos o movimientos periódicos/cuasiperiódicos.
- **Estabilidad** ($\lambda_1 < 0$): Las trayectorias convergen exponencialmente hacia un atractor (punto fijo o ciclo límite). El sistema es estable y predecible.

3.3.3. El Espectro de Lyapunov

Para un sistema dinámico n -dimensional, existe un **Espectro de Lyapunov** compuesto por n exponentes $(\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n)$.

- **Interpretación Geométrica:** Cada exponente λ_i mide la tasa de expansión o contracción a lo largo de un eje principal en el espacio de fases.
- **Volumen del Espacio de Fases:** La suma de todos los exponentes relaciona la dinámica con la conservación de volumen:

$$\sum_{i=1}^n \lambda_i = \text{Tasa de cambio de volumen}$$

Si la suma es **cero** (como en el problema de n cuerpos), el sistema es **conservativo** (preserva el volumen). Si es negativa, es disipativo.

- **Dimensión de Atractores:** El espectro completo es necesario para calcular la **Dimensión de Lyapunov** (D_{KY}), que estima la dimensión fractal del atractor en el que se desarrolla el movimiento caótico.

3.3.4. Cálculo y Limitaciones

El cálculo numérico de los exponentes utiliza el **Teorema Ergódico Multiplicativo de Oseledec** y se implementa generalmente a través del **método de Benettin**, que emplea la descomposición QR o SVD de la matriz fundamental, incluyendo la **reortogonalización periódica** de los vectores de perturbación.

- **Necesidad de Reortogonalización:** Es crucial para evitar que todos los vectores colapsen en la dirección de λ_1 , permitiendo estimar los exponentes menores.
- **Limitación de Convergencia:** La principal limitación práctica es la lentitud de la convergencia. El exponente es una propiedad asintótica ($t \rightarrow \infty$), por lo que el cálculo requiere tiempos de integración muy largos, especialmente en sistemas conservativos.

3.4. Integradores Simplécticos Gravitacionales

Los integradores simplécticos para sistemas gravitacionales son métodos geométricos para problemas hamiltonianos, fundamentales en simulaciones de largo plazo del problema de N -cuerpos (planetas, satélites, asteroides, cúmulos, galaxias) [10-13]. Su rasgo distintivo es preservar la forma simpléctica (estructura de fase), lo que confiere estabilidad numérica y errores de energía acotados a lo largo del tiempo [11, 14, 15].

3.4.1. Definición y principios

Sea $H(\mathbf{q}, \mathbf{p}, t)$ un Hamiltoniano. Un paso numérico define un mapa

$$\Phi_h : (\mathbf{q}_n, \mathbf{p}_n) \mapsto (\mathbf{q}_{n+1}, \mathbf{p}_{n+1}),$$

con Jacobiano $J = \partial(\mathbf{q}_{n+1}, \mathbf{p}_{n+1})/\partial(\mathbf{q}_n, \mathbf{p}_n)$. El método es *simpléctico* si $J^T \Omega J = \Omega$, con Ω la matriz simpléctica estándar, de donde se sigue la preservación de

volumen (Liouville) [14, 15]. En gravedad N -cuerpos, $H = T(\mathbf{p}) + V(\mathbf{q})$ es separable, habilitando *splitting* de flujos¹: el esquema básico Leapfrog/Verlet (orden 2) compone *drift-kick-drift* y es simpléctico [11, 14, 16].

3.4.2. Contexto histórico

Desde los 80–90 se construyen integradores simplécticos de orden superior por composición simétrica (*Forest–Ruth*, *Yoshida*) [14, 15, 17]. En dinámica planetaria, el mapeo de *Wisdom–Holman* [10] explota la dominancia Kepleriana. En paralelo, los integradores variacionales discretizan el principio de acción y preservan exactamente mapas de momento asociados a simetrías [14, 16].

3.4.3. Variantes representativas

Tabla 3.2: Familias de integradores simplécticos para N -cuerpos (resumen).

Variante	Idea y uso típico	Ref.
Leapfrog/Verlet	Orden 2, base para Hamiltonianos separables $H = T + V$; robusto y barato por paso.	[11, 14, 16]
Wisdom–Holman (WH)	$H = H_{\text{Kepler}} + H_{\text{Interaction}}$; ideal para órbitas casi Keplerianas (sistemas planetarios).	[10, 17, 18]
Composición de alto orden	Esquemas simétricos (Yoshida) elevan orden a costa de más evaluaciones (a veces con subpasos negativos).	[14, 15]
Integradores variacionales (GGL)	Discretización lagrangiana; preservan exactamente momentos por simetría y son simplécticos.	[14, 16]
Simplécticos híbridos	Conmutan a métodos adaptativos (IAS15/BS) en encuentros cercanos; vuelven a simplécticos fuera de eventos.	[9, 13, 18]
Forward simplécticos (FSI)	Esquemas solo con pasos positivos ² ; eficientes en ciertos regímenes.	[12]
Basados en Encke	Integran perturbaciones respecto a órbita Kepleriana de referencia; muy precisos si la perturbación es pequeña.	[13]
Gauge-compatibles	Preservan leyes/discretizaciones con simetrías de <i>gauge</i> ³ .	[19]
Marcos corrotantes	Extensiones K-simplécticas ⁴ para sistemas en rotación.	[20]

3.4.4. Propiedades clave

1. **Simplecticidad**: preserva la 2-forma simpléctica y el volumen de fase [11, 14].
2. **Energía acotada**: no conserva H exactamente, pero el error oscila en torno a un *Hamiltoniano sombra*⁵ [15, 21].
3. **Conservación de momento**: exacta en integradores variacionales bajo simetrías; muy buena en otros esquemas [14, 16].
4. **Estabilidad a largo plazo**: evita deriva secular en elementos orbitales [10, 15].

¹*Splitting*: descomponer H en sumandos con flujos integrables (o casi exactos) y componer sus exponenciales para aproximar $\exp(h\mathcal{L}_H)$.

⁵Hamiltoniano sombra: $\tilde{H}$ próximo a H tal que Φ_h es exactamente el flujo de $\tilde{H}$ por tiempos largos.

3.4.5. Aplicaciones

Dinámica del Sistema Solar y exoplanetas, cúmulos estelares (con cuidado en encuentros cercanos), materia oscura en cosmología (Leapfrog), y física de plasmas con métodos geométricos (PIC) [10, 11, 13, 18, 19].

3.4.6. Ventajas

Estabilidad y precisión a largo plazo; error energético sin deriva secular; buena conservación de integrales; alta eficiencia para $H = T + V$ [10, 11, 15, 16].

3.4.7. Limitaciones

- **Paso fijo:** la simplecticidad exige, en general, *timestep* fijo; ineficiente en sistemas multiescala o con encuentros [11, 13].
- **Adaptatividad delicada:** variar el paso rompe simplecticidad; enfoques *block power-of-two*⁶ mitigan el problema [16, 21].
- **Encuentros cercanos y $1/r$:** pueden requerir regularización o integradores no simplécticos de alto orden (híbridos) [13, 18].
- **Fuerzas no conservativas:** disipación/arrastre comprometen propiedades geométricas; requieren extensiones específicas [21].

3.4.8. Direcciones actuales

Híbridos (simpléctico + adaptativo), adaptatividad compatible con estructura (p. ej., potencias de dos, correctores), integradores geométricos más allá de lo simpléctico (gauge, marcos corrotantes) y control fino de error de redondeo en doble precisión [9, 16-20].

3.5. Integrador Simpléctico WHFast

WHFast (Wisdom–Holman Fast) es un integrador simpléctico optimizado para dinámica planetaria a largo plazo [22]. Preserva la estructura de Hamilton (simplecticidad), por lo que la energía no deriva secularmente y las oscilaciones de error quedan acotadas [23]. En la práctica: pasos grandes y simulaciones muy largas con estabilidad numérica superior a métodos no simplécticos.

3.5.1. Idea clave (para ingeniería)

Muchos sistemas tienen un cuerpo central dominante. Se separa el Hamiltoniano:

$$H = H_{\text{Kepler}} + H_{\text{Interaction}}$$

y se integra por *splitting* tipo Leapfrog (DKD): *drift* con H_{Kepler} , *kick* con $H_{\text{Interaction}}$, *drift* con H_{Kepler} [10]. Así, la parte Kepleriana se resuelve casi-exacta y las perturbaciones se aplican como impulsos.

⁶Esquemas con pasos cuantizados en potencias de dos por partícula/intervalo; preservan propiedades geométricas “casi siempre”.

3.5.2. Qué hace rápido a WHFast

- **Solucionador de Kepler robusto:** Newton/Laguerre–Conway con manejo cuidadoso de funciones de Stumpff y criterios de convergencia en punto flotante [7, 22].
- **Coordenadas de Jacobi estables:** transformaciones sin sesgo entre inerciales y Jacobi para minimizar error de redondeo en el *drift* Kepleriano [7, 22].
- **Ley de Brouwer:** con Δt suficientemente pequeño el error energético sigue un paseo aleatorio $\propto \sqrt{t}$ (no lineal) [22].

3.5.3. Opciones prácticas (como hiperparámetros)

Parámetro	Efecto resumido
corrector	Correctores simplécticos de orden impar (3–17) reducen oscilaciones de energía con coste marginal si las salidas son poco frecuentes [22].
coordinates	jacobi (preferido para Kepler); también barycentric, democraticheliocentric, whds [24].
kernel	Variantes (default, modifiedkick, composition, lazy); útiles en casos específicos [24].
safe_mode	1 = sincronización y checks automáticos (más robusto); 0 = más rápido, pero el usuario debe manejar la sincronía si modifica partículas [24].
variational	Ecuaciones variacionales para LCN/MEGNO casi “gratis” en coste [7, 22].

3.5.4. Cuándo usarlo / cuándo evitarlo

Úsarlo si: hay cuerpo central dominante, órbitas casi Keplerianas, integración de millones de períodos, necesidad de conservación a largo plazo (energía, momento) y análisis de estabilidad/caos.

Evitarlo si: hay encuentros cercanos frecuentes, colisiones o no existe un centro dominante; fuerzas no conservativas/dependientes de velocidad (usar módulos externos como REBOUNDx rompen symplecticidad) [23, 25].

3.6. REBOUND: simulador N-cuerpos modular y abierto para dinámica (con y sin colisiones)

REBOUND [7] es un framework C/C99 modular para simulación N-cuerpos bajo GPLv3⁷ y POSIX⁸. Nació para dinámica colisional (anillos planetarios), pero

⁷GPLv3: licencia copyleft que garantiza redistribución del código y derivados bajo los mismos términos.

⁸POSIX: estándar de interfaces del sistema (Unix-like) que facilita portabilidad en Linux/-

también cubre gravedad pura con integradores simplécticos y aceleradores de fuerza.

3.6.1. Arquitectura (resumen)

Módulos intercambiables para: integradores, gravedad, detección de colisiones y contornos. La selección se hace por configuración, sin tocar el núcleo, lo que permite armar “pipelines” específicos por problema.

3.6.2. Módulos clave

Integradores (timestep fijo):

- **Leapfrog (DKD)**: Simpléctico de 2º orden (drift–kick–drift)⁹.
- **Wisdom–Holman (WH)**: Mapeo de variables mixtas; resuelve exactamente la parte Kepler ($1/r$), requiere solver de Kepler iterativo.
- **SEI**: Integrador epiciclo simpléctico para aproximación de Hill/*shearing sheet*¹⁰, útil en anillos/láminas.

Gravedad

- **Suma directa**: Exacta, costo $O(N \cdot N_{\text{active}})$; viable con pocos cuerpos masivos.
- **Octree (Barnes–Hut)**¹¹: precisión controlada por θ_{crit} ¹² y opción de término cuadrupolar¹³. Paraleliza con MPI/OpenMP¹⁴.

Detección de colisiones (esferas duras, coef. de restitución ϵ):

- **Vecino directo**: $O(N^2)$ al final del paso.
- **Octree**: reutiliza la jerarquía para buscar solapamientos en $O(N \log N)$ promedio.
- **Barrido plano (*plane sweep*)**¹⁵: muy eficiente en geometrías delgadas (anillos, láminas).

macOS.

⁹DKD: esquema de *splitting* que alterna evolución Kepleriana (drift) y “impulsos” por interacciones (kick).

¹⁰Aproximación local de un disco en rotación diferencial; dinámica en un marco corrotante con cizalla.

¹¹Barnes–Hut: agrupa partículas lejanas en celdas; usa su centro de masa (y opcionalmente cuadrupolo) para aproximar la fuerza, reduciendo costo a $O(N \log N)$.

¹² θ_{crit} : umbral de apertura; celdas con tamaño/distancia menores al umbral se tratan como multipolos.

¹³Cuadrupolo: término de orden superior en la expansión multipolar que mejora la aproximación de fuerzas de grupos lejanos.

¹⁴MPI/OpenMP: paso de mensajes entre nodos y multihilo en un nodo, respectivamente; combinables para escalado híbrido.

¹⁵Algoritmo que desplaza un plano/línea a través del dominio, manteniendo una lista activa de trayectorias que intersecta; muy eficiente en geometrías cuasi-2D o alargadas.

Contornos Abiertos, periódicos y *shear-periodic*¹⁶.

3.6.3. Notas

- **Modelo de coste:** directo $O(N^2)$ vs. octree $O(N \log N)$; ajustar θ_{crit} y cuádrupolo como *knobs* de precisión/rendimiento.
- **Escalado fuerte/débil**¹⁷: usar MPI+OpenMP; vigilar tamaño mínimo por proceso (costes de comunicación).
- **Reproducibilidad/determinismo:** fijar compilador/flags, versión de REBOUND, `safe_mode`, semillas y orden de operaciones (paralelismo puede cambiar redondeos IEEE-754).
- **Geometría manda:** barrido plano para dominios delgados; octree con θ_{crit} más estricto en distribuciones 3D complejas.

3.6.4. Cuándo usarlo / evitarlo

Úsarlo para: anillos/discos, N grande con colisiones, o integración orbital prolongada con buena conservación. **Evitarlo** si: necesitas pasos adaptativos intensivos con encuentros cercanos (mejor híbridos o integradores no simplécticos de alto orden), o fuerzas no conservativas dominantes (rompen simplecticidad; usar módulos externos con cautela).

3.6.5. Aplicaciones típicas

Anillos planetarios, formación de planetesimales, discos de transición/escombros (con superpartículas¹⁸) y problemas clásicos del Sistema Solar [7].

3.7. Problemas de Factibilidad y Optimización

Los problemas de **optimización** y **factibilidad** son pilares en ingeniería, ciencias computacionales e investigación operativa. Un problema de optimización busca la “mejor” alternativa sujeta a restricciones; uno de factibilidad sólo determina si existe al menos una solución que cumpla dichas restricciones [26].

3.7.1. Formulación estándar de optimización

Sea $\mathbf{x} \in \mathbb{R}^n$ el *vector de decisión*. La forma canónica es [26]:

$$\min_{\mathbf{x} \in \mathbb{R}^n} f_0(\mathbf{x}) \quad (3.7)$$

$$\text{sujeto a } g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \quad (3.8)$$

$$h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p \quad (3.9)$$

¹⁶Condición periódica con cizalla: fronteras se desplazan diferencialmente para emular rotación con gradiente de velocidad.

¹⁷Escalado fuerte: reducir tiempo con mismo problema al aumentar recursos. Escalado débil: mantener tiempo al crecer problema y recursos proporcionalmente.

¹⁸“Superpartícula”: partícula representativa que agrega muchas partículas físicas para reducir costo computacional.

Definiciones básicas:

- **Dominio** $\mathcal{D}$: valores de $\mathbf{x}$ donde todas las funciones están definidas.
- **Conjunto factible** $\mathcal{X}$: puntos de $\mathcal{D}$ que satisfacen (3.8)–(3.9). Si $\mathcal{X} = \emptyset$, el problema es *infactible*.
- **Valor óptimo** $p^* = \inf\{f_0(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}\}$ y **solución óptima** $\mathbf{x}^* \in \mathcal{X}$ con $f_0(\mathbf{x}^*) = p^*$.

3.7.2. Clasificación y optimización convexa

Los problemas se tipifican por la forma de f_0, g_i, h_j :

- **LP**: todas afines.
- **QP**: f_0 cuadrática convexa; restricciones afines.
- **Convexos**: f_0 y g_i convexas; h_j afines ($A\mathbf{x} = \mathbf{b}$) [26].

En problemas **convexos**, todo óptimo local es **global**; existen algoritmos eficientes y fiables. Una f es convexa si satisface la Desigualdad de Jensen. Aplicaciones: diseño de ingeniería, ajuste de datos (regresión), gestión de riesgos (carteras) [26].

3.7.3. Problemas de factibilidad

El problema de factibilidad busca *cualquier* $\mathbf{x}$ que cumpla las restricciones:

$$\text{encontrar } \mathbf{x} \in \mathbb{R}^n \quad (3.10)$$

$$\text{sujeto a } g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \quad (3.11)$$

$$h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p \quad (3.12)$$

Hay solución si y sólo si $\mathcal{X} \neq \emptyset$.

3.7.4. Enfoque algorítmico: oráculos y planos cortantes

En factibilidad convexa es común usar un *oráculo de separación*¹⁹ junto con métodos de *planos cortantes*²⁰, reduciendo el espacio de búsqueda hasta certificar factibilidad o probar lo contrario [27].

3.7.5. Relación factibilidad–optimización y métodos de dos fases

La factibilidad es **prerrequisito** de la optimización: no hay óptimo sin conjunto factible. Además, la factibilidad es un caso particular de optimización tomando

$$\min_{\mathbf{x}} f_0(\mathbf{x}) = 0.$$

Si el problema es factible, cualquier $\mathbf{x} \in \mathcal{X}$ es óptimo y $p^* = 0$; si es infactible, $p^* = +\infty$.

¹⁹Dado $\mathcal{C}$ convexo y un punto $\mathbf{x}$, el oráculo verifica si $\mathbf{x} \in \mathcal{C}$; en caso contrario devuelve un *hiperplano separador* $\mathbf{a}^\top \mathbf{z} \leq b$ que excluye $\mathbf{x}$ pero contiene $\mathcal{C}$.

²⁰*Cutting-planes*: iterativamente refinan una región candidata añadiendo desigualdades que “cortan” puntos no factibles, hasta certificar factibilidad o infactibilidad.

Dos fases (interior/punto barrera). Muchos *métodos de punto interior*²¹ usan una **Fase I** para encontrar factibilidad: introducir $s \in \mathbb{R}$ y resolver

$$\min_{\mathbf{x}, s} \quad \text{sujeto a} \quad g_i(\mathbf{x}) \leq s, \quad i = 1, \dots, m, \quad h_j(\mathbf{x}) = 0.$$

Si $s^* \leq 0$, se obtiene un punto estrictamente factible y se procede a la **Fase II** resolviendo (3.7)–(3.9). En caso contrario, se certifica infactibilidad.

Resumen. La optimización busca el “mejor” punto dentro de $\mathcal{X}$; la factibilidad sólo decide si $\mathcal{X}$ existe. Esta relación permite reutilizar técnicas de optimización (p. ej., dos fases, planos cortantes) para resolver factibilidad con garantías teóricas y prácticas.

3.8. Algoritmos Genéticos (AGs) Simples Mono-objetivo mediante pymoo

3.8.1. Fundamentos de los AGs canónicos

Los **Algoritmos Genéticos (AGs)** son metaheurísticas de **computación evolutiva** que optimizan una *población* de soluciones con tres operadores: **selección** (mayor probabilidad para mayor aptitud), **cruce** (recombinación de padres) y **mutación** (pequeñas perturbaciones para mantener diversidad). El *AG canónico* usaba codificación binaria, ruleta, cruce de un punto y mutación por bit; hoy se emplean variantes robustas para variables continuas y problemas *no lineales/no diferenciables* con múltiples óptimos locales.

3.8.2. Arquitectura modular de pymoo

pymoo desacopla **problema**, **algoritmo** y **operadores**:

- `pymoo.core.problem`: define función objetivo, `n_var`, y límites `xl/xu`.
- `pymoo.algorithms`: incluye GA para mono-objetivo.
- `pymoo.operators`: `sampling`, `selection`, `crossover`, `mutation`.
- `pymoo.termination`: criterios de parada (p. ej., (`‘n_gen’`, 100)).
- `pymoo.optimize`: `minimize ()` orquesta la ejecución.

3.8.3. AG mono-objetivo en pymoo

La clase `pymoo.algorithms.soo.nonconvex.ga.GA` implementa un GA para *Single-Objective Optimization* (incluye funciones no convexas). **Parámetros clave de GA (...)**:

- `pop_size`: tamaño de población (exploración vs. costo).
- `sampling`: inicialización (p. ej., `RandomSampling`).

²¹ *Punto interior / barrera logarítmica*: optimización sobre el interior del conjunto factible penalizando la cercanía a las fronteras con una barrera (p. ej., suma de logaritmos). Requieren típicamente un punto estrictamente factible.

- **crossover:** recombinación (p.ej., SBX²² para continuas; SinglePointCrossover para binarias).
- **mutation:** perturbación (p.ej., PolynomialMutation²³ para continuas; BitflipMutation para binarias).
- **eliminate_duplicates:** evita individuos idénticos (más diversidad; mayor costo).

Flujo de trabajo. (1) Definir Problem; (2) instanciar GA; (3) ejecutar minimize (problem, algorithm, termination); (4) extraer res.X (mejor decisión) y res.F (mejor objetivo).

3.8.4. Operadores y terminación

- **Selección:** torneo (controla presión selectiva) o ruleta.
- **Cruce:** SBX para continuas; combina y explora alrededor de padres.
- **Mutación:** PolynomialMutation para mantener diversidad y escapar de óptimos locales.
- **Reemplazo/elitismo:** preservar el mejor individuo entre generaciones.
- **Criterios de parada:** generaciones, convergencia, tiempo, etc.

3.8.5. Ejemplo mínimo con GA

Minimizar $f(\mathbf{x}) = \sum_i x_i^2$ con límites $[-5, 5]$:

3.9. Framework de interfaz: Qt & PyQt

3.9.1. Introducción a Qt

Qt es un framework C/C99 multiplataforma para construir GUIs y apps nativas en Linux, Windows, macOS, Android y sistemas embebidos, bajo el principio “escribe una vez, ejecuta en múltiples plataformas”²⁴ [28-32]. Su madurez (KDE, industria) y evolución de licenciamiento lo posicionan como solución robusta para aplicaciones científicas e industriales [29, 32].

3.9.2. Principios fundamentales de Qt

Qt extiende C++ mediante el *Meta-Object Compiler* (moc)²⁵ [28]. El mecanismo *signals-slots* implementa un patrón observador fuertemente tipado y desacoplado para programación dirigida por eventos [33]. La arquitectura modular distingue

²²SBX: *Simulated Binary Crossover*; η controla cercanía de descendientes a los padres.

²³Mutación polinomial (PM): distribución controlada por η ; η alto $\Rightarrow$ pasos pequeños.

²⁴WORA: escribir una sola base de código y desplegar en varios SO con cambios mínimos.

²⁵moc: preprocesador que genera metainformación para *signals/slots*, introspección y dispatch dinámico.

Listing 1 Optimización mono-objetivo con `pymoo::GA`. *Nota:* minted requiere compilar con `-shell-escape`.

```

1  import numpy as np
2  from pymoo.core.problem import Problem
3  from pymoo.algorithms.soo.nonconvex.ga import GA
4  from pymoo.operators.sampling.rnd import RandomSampling
5  from pymoo.operators.crossover.sbx import SBX
6  from pymoo.operators.mutation.pm import PolynomialMutation
7  from pymoo.optimize import minimize
8
9  # 1) Definir el problema
10 class MyProblem(Problem):
11     def __init__(self):
12         # 2 variables (n_var=2), 1 objetivo (n_obj=1), 0
13         #   restricciones; límites [-5, 5]
14         super().__init__(n_var=2, n_obj=1, n_constr=0, xl=-5,
15                          xu=5)
16
17     def _evaluate(self, x, out, *args, **kwargs):
18         out["F"] = np.sum(x**2, axis=1)
19
20 problem = MyProblem()
21
22 # 2) Instanciar el algoritmo
23 algorithm = GA(
24     pop_size=20,
25     sampling=RandomSampling(),
26     crossover=SBX(prob=0.9, eta=15),
27     mutation=PolynomialMutation(prob=0.1, eta=20),
28     eliminate_duplicates=True
29 )
30
31 # 3) Ejecutar (100 generaciones)
32 res = minimize(problem, algorithm, ("n_gen", 100), verbose=False)
33
34 # 4) Resultado
35 print("Mejor X:", res.X)
36 print("Mejor F:", res.F)

```

Essentials (Qt Core/GUI/Multimedia/Network/Quick/SQL) y *Add-Ons* (OpenGL/Wayland/Sensors/WebView/Safe Renderer/SCXML), permitiendo incluir sólo lo necesario [29].

3.9.3. Widgets, layouts y Qt Quick

La construcción clásica usa *widgets* y *layouts* (QVBoxLayout, QHBoxLayout, QGridLayout) para interfaces responsivas [31, 34]. *Qt Quick* y QML²⁶ ofrecen un enfoque declarativo moderno, integrable con lógica C++/Python cuando se requiere rendimiento [28, 35].

3.9.4. Licenciamiento y versiones

Qt se distribuye con doble modelo (GPL/LGPL y comercial) [28]. Qt 5 (2012) introdujo modularidad/QPA y Qt Quick 2; Qt 6 (2020) continúa la modernización; versiones LTS están disponibles principalmente para clientes comerciales [28-30, 32].

3.9.5. Aplicaciones típicas

Qt se usa en escritorio, móvil y embebidos (automoción, salud, aeroespacial, automatización, consumo) [28, 30]. Casos conocidos: Google Earth, Skype, VirtualBox, KDE, Maya, Telegram [36]. En computación científica: visualización, front-ends de laboratorio, GUIs de simulación y *Qt Quick 3D Physics* para cuerpos rígidos [37, 38].

3.9.6. Ventajas y desventajas de Qt

Ventajas: portabilidad real, ecosistema amplio (UI, red, multimedia, 3D), rendimiento nativo C++, documentación y comunidad sólidas [29-31].

Desventajas: curva de aprendizaje (C++/*signals-slots*), coste de licencias comerciales, posible sobrecarga en apps muy pequeñas/ultra-embebidas [28, 30, 39].

3.9.7. Introducción a PyQt

PyQt son *bindings* de Python para Qt, mantenidos por Riverbank (desde 1998, generados con SIP), con >1000 clases expuestas [[pyqt.wiki](#)]. *Qt for Python* (PySide6) es la alternativa oficial de The Qt Company; la elección suele depender de licencias (PySide tiende a LGPL; PyQt GPL o comercial) [34, 35].

3.9.8. Principios fundamentales de PyQt

PyQt expone módulos (QtCore, QtGui, QtWidgets, QtNetwork, QtSql) y preserva el modelo *signals-slots* en Python para GUIs reactivas [31, 34]. Integra con Qt Designer (.ui → pyuic) para prototipado rápido [30, 34].

3.9.9. Aplicaciones típicas de PyQt (perfil IA)

GUIs multiplataforma para workflows de IA/ML: paneles de entrenamiento e inferencia, control de simulaciones, anotación/etiquetado, visores de datos e

²⁶QML: lenguaje declarativo con JavaScript embebido para UI reactivas y animadas.

imágenes, y front-ends que integran PyQtGraph/Matplotlib [35, 38]. Útil como capa de interacción para motores numéricos (C++/CUDA) y pipelines científicos.

3.9.10. Ventajas y desventajas de PyQt

Ventajas: productividad de Python + potencia de Qt, multiplataforma, ecosistema de widgets/gráficos/red/BD y soporte de diseño visual [30, 34, 38].

Desventajas: licenciamiento (GPL o comercial; no LGPL en PyQt) [pyqt·wiki]; posibles fallos duros por *wrapping* C++; curva mayor que Tkinter; documentación centrada en C++ requiere adaptación al entorno Python [39, 40].

3.10. Matplotlib

3.10.1. Introducción

Matplotlib es la biblioteca base de visualización 2D (y 3D básico) del ecosistema científico de Python. Permite generar figuras estáticas, animadas e interactivas con calidad de publicación, integrándose de forma natural con NumPy/SciPy/pandas y adoptando una interfaz familiar para usuarios de MATLAB [41-44]. Su adopción generalizada en investigación respalda su estabilidad y evolución [43].

3.10.2. Arquitectura y conceptos clave

Matplotlib separa responsabilidades en tres capas [45, 46]:

1. **Backend**²⁷: gestiona renderizado y salida a formatos/ventanas (Tk/Qt/w-x/GTK o archivos PNG, PDF, SVG).
2. **Artist (núcleo)**: jerarquía de objetos que componen la figura (Figure, Axes²⁸, líneas, textos, parches).
3. **Capa pyplot**: interfaz de alto nivel con estado para trazados rápidos [47].

3.10.3. Estilos de API: pyplot vs. OO

Tabla 3.3: Comparativa de APIs en Matplotlib.

	pyplot (con estado)	Orientada a objetos (OO)
Uso típico	Exploración y scripts simples	Apps, librerías y figuras complejas
Control	Menor control explícito	Control fino de Figure/Axes
Código	Más corto, procedural	Más verboso, reutilizable y claro

Ambos estilos coexisten; la API OO es preferible en proyectos de producción o figuras complejas [44, 47].

²⁷Backend: motor de render y salida (pantalla/archivo). Común: Agg (raster), PDF/SVG (vectorial).

²⁸Axes: área de trazado con ticks/labels/leyendas. Axis: el eje numérico en sí.

3.10.4. Capacidades de trazado y personalización

Soporta líneas, dispersión, barras, histogramas, contornos y mapas de calor; 3D básico con `mplot3d` [42]. Ofrece control detallado (colores, estilos, tipografías, anotaciones) y múltiples formatos de salida (PNG; PDF/SVG vectoriales para publicación) [45, 48, 49]. Permite animaciones (`FuncAnimation`) para procesos dependientes del tiempo [50].

3.10.5. Aplicaciones típicas

En física/astro/ingeniería se usa para trayectorias, espacios de fase, campos y series temporales [42, 51, 52]. En IA/ML: curvas de entrenamiento (loss/accuracy), matrices de confusión, ablation studies, *hyperparameter sweeps* y visualización de inferencia. Es base de librerías de mayor nivel como Seaborn [53] y se integra con Jupyter para flujos reproducibles [42, 54]. Casos emblemáticos incluyen visualizaciones del EHT y misiones de la NASA [46, 55].

3.10.6. Ventajas

- **Versatilidad:** amplio catálogo 2D y control fino de la estética [48, 56].
- **Calidad de publicación:** salida vectorial (PDF/SVG) y estilos configurables (`rcParams`, hojas de estilo; p. ej., `SciencePlots`) [41, 57, 58].
- **Ecosistema:** integración estrecha con NumPy/SciPy/pandas y soporte multiplataforma [42, 45].
- **Abierto y documentado:** comunidad amplia y documentación extensa [43, 59].

3.10.7. Limitaciones

- **Verbosidad y curva inicial:** figuras complejas requieren API OO y más código; la dualidad `pyplot`/OO puede confundir [56, 60].
- **Rendimiento con datos muy grandes:** millones de puntos pueden requerir muestreo, agregación o librerías alternativas [61].
- **Interactividad limitada web:** menor que soluciones web nativas (`Plotly`/`Bokeh`) [48].

CAPÍTULO 4

Planeación

4.1. Validez por Juicio de Expertos

La validación de un proyecto técnico mediante el juicio de expertos constituye una etapa fundamental para asegurar su rigor, pertinencia y alineación con el conocimiento y las prácticas actuales en los campos de estudio relevantes. Esta sección, “4.1 Validez por Juicio de Expertos”, se dedica a documentar y analizar este proceso crucial, buscando corroborar la pertinencia del problema abordado, la solidez de la metodología empleada y la viabilidad de la solución propuesta para el “Modelo para representar comportamientos gravitacionales con dos cuerpos”.

A continuación, se detallarán las consultas realizadas a profesionales con experiencia en áreas directamente relacionadas con los objetivos y aplicaciones de este Trabajo Terminal. Se presentará el perfil de los expertos consultados, el método empleado para la recolección de sus aportaciones, un resumen de sus observaciones y valoraciones, y un análisis de cómo estas perspectivas externas contribuyen a la robustez y credibilidad general del proyecto. La incorporación de una visión cualificada es esencial no solo para refinar el enfoque del trabajo, sino también para asegurar que sus contribuciones sean significativas y estén bien fundamentadas dentro del estado del arte y sus potenciales campos de aplicación, como la simulación científica y el desarrollo de videojuegos.

4.1.1. Consulta con Experto en Optimización Evolutiva e Investigador de agentes en Videojuegos

Con el objetivo de validar la pertinencia del problema abordado y la adecuación de la solución propuesta en este Trabajo Terminal, se realizó una consulta especializada. Esta validación externa es crucial para asegurar que el proyecto se alinea con las necesidades y desafíos actuales en los campos de la simulación y sus potenciales aplicaciones.

Perfil del Experto Consultado

Se consultó al Maestro en Ciencias Computacionales (M.CC.) José Alberto Torres León, egresado del Centro de Investigación en Cómputo (CIC) del Instituto Politécnico Nacional, y actualmente estudiante de último año del Doctorado en

Ciencias de la Computación en la misma institución. Sus áreas de especialización incluyen la generación procedimental de contenido, agentes inteligentes para videojuegos (jugadores y dinámicos), aprendizaje por refuerzo, optimización evolutiva y *Deep Learning*. Su experiencia en optimización y su profundo conocimiento del desarrollo y las limitaciones en el ámbito de los videojuegos lo convierten en un validador idóneo para el presente proyecto.

Método de Consulta

La consulta se llevó a cabo mediante una reunión formal el martes 06 de mayo de 2025, entre las 12:35 y las 13:10 horas, en las instalaciones de investigación del CIC. Previo a la discusión, el M.CC. Torres León revisó el borrador del presente reporte técnico. Durante la reunión, se le expuso verbalmente el propósito, la visión y la evolución del proyecto, desde su concepción inicial como un “videojuego que permitiera jugar con las cualidades físicas gravitacionales de objetos para atraer, repeler, o que se mantengan en un equilibrio modificando características de los objetos en tiempo de ejecución”, hasta su iteración final como un “modelo de simulación de dos cuerpos celestes que permita el cambio de características en los cuerpos para encontrar los valores idóneos que mantengan al sistema en estabilidad”.

Resumen de Opiniones y Observaciones Obtenidas

Tras la revisión del material y la discusión, el M.CC. Torres León aportó las siguientes observaciones y valoraciones:

- **Comprensión del Problema y Enfoque:**

- Inicialmente, el experto señaló que la afirmación de que el estado del arte actual en simuladores no permite la definición propia de parámetros no era del todo precisa. Se clarificó que la limitación principal que el proyecto busca abordar es la *modificación de estos parámetros en tiempo de ejecución*.
- Identificó correctamente que el problema subyacente que se trata de resolver es de naturaleza combinatoria y subrayó la necesidad de una definición formal y clara de dicho problema combinatorio dentro del reporte.
- Enfatizó la importancia de establecer un objetivo específico para la evaluación de resultados, que permita corroborar que el comportamiento simulado sea el adecuado y esperado.

- **Aplicabilidad y Relevancia en Videojuegos:**

- Confirmó que las herramientas actuales para motores físicos en el desarrollo de videojuegos “no poseen flexibilidad en el entorno de desarrollo” para el tipo de interacciones dinámicas que este proyecto plantea, limitando la implementación de mecánicas como las concebidas originalmente para el Trabajo Terminal.
- Señaló que la industria de los videojuegos busca realizar simulaciones apegadas a comportamientos físicos reales, pero explorando “casos y

resultados que no se han captado o no pasan en la realidad”, lo cual se alinea con la capacidad del modelo propuesto.

- Consideró plausible que modelos similares ya existan en videojuegos comerciales, pero como parte de “patentes o secretos industriales”, citando como ejemplo el jefe final del videojuego *Bayonetta 1*, donde cuerpos celestes son dirigidos dinámicamente. Esto resalta la dificultad de acceso a este conocimiento y la relevancia de una propuesta abierta.
- Opinó que la solución propuesta puede “dar una ilusión más realista de comportamientos gravitacionales en videojuegos”, con un “mucho más valor en juegos de tipo plataformero”.
- Prevé una “influencia positiva en la experiencia del usuario y del desarrollador” al contar con modelos de esta naturaleza.
- Destacó un “sesgo en la investigación” del área debido a que muchos avances no se documentan públicamente o están protegidos, limitando el acceso a información pública. Mencionó que la investigación en generación procedimental de contenido y videojuegos ha ganado seriedad recientemente (desde aproximadamente 2016), aunque persiste cierto escepticismo en algunos círculos académicos.
- **Planeación y Delimitación del Proyecto:**
 - El experto consideró que el proyecto presenta una “buena dirección” y una “buena delimitación”, especialmente tras las sucesivas adaptaciones del alcance.
- **Notas Adicionales:**
 - Mencionó la existencia de optimizadores basados en fenómenos gravitacionales, aunque no especificó nombres.
 - Observó que el enfoque del reporte técnico parecía más orientado hacia la Ingeniería en Sistemas Computacionales (ISC) que hacia la Ingeniería en Inteligencia Artificial (IIA), una apreciación relevante para la presentación del trabajo.

Análisis de la Validación Experta y Vinculación con el Proyecto

Las observaciones del M.CC. José Alberto Torres León proporcionan una validación significativa tanto para la relevancia del problema como para la viabilidad y el impacto potencial de la solución propuesta.

- **Validación de la Pertinencia del Problema:**
 - La confirmación de que las herramientas actuales en motores de videojuegos carecen de la flexibilidad necesaria para modificar parámetros físicos gravitacionales en tiempo de ejecución (como se propone) valida directamente la necesidad que este proyecto busca satisfacer.
 - Su comentario sobre la existencia de soluciones propietarias o no publicadas refuerza la justificación de desarrollar un modelo abierto y documentado que pueda servir como base para futuras investigaciones

o desarrollos, especialmente considerando el sesgo de investigación existente.

- La identificación del problema como combinatorio y la necesidad de su definición clara son aportes que refinan la presentación del desafío técnico que se aborda.
- **Respaldo a la Solución Propuesta:**
 - La opinión de que el modelo puede ofrecer una “ilusión más realista” y tener un “mayor valor” en ciertos géneros de videojuegos respalda el potencial de aplicación de la solución.
 - La valoración positiva sobre la “buena dirección” y “delimitación” del proyecto, tras conocer su evolución, sugiere que el enfoque actual es pertinente y manejable.
 - La necesidad de un objetivo específico de evaluación de resultados, sugerida por el experto, se alinea con la metodología de investigación y se considerará fundamental para la etapa de pruebas y validación del modelo.

Gracias a la consulta con el M.CC. Torres León validamos que el problema de la modificación dinámica de parámetros gravitacionales en simulaciones es relevante, especialmente con miras a aplicaciones innovadoras como en videojuegos, y que la solución propuesta, aunque delimitada a dos cuerpos, aborda una brecha existente y tiene una dirección metodológica sólida. Las sugerencias sobre la definición del problema combinatorio y los criterios de evaluación serán integradas para fortalecer el rigor del proyecto.

4.1.2. Consulta con Experto en Optimización Evolutiva y Mecatrónica Aplicada

Para corroborar la relevancia del problema de investigación y la solidez del enfoque metodológico propuesto en este Trabajo Terminal, se llevó a cabo una consulta especializada con un experto en el campo de la optimización evolutiva y sus aplicaciones en ingeniería.

Perfil del Experto Consultado

Se consultó al Dr. Daniel Molina Pérez, Investigador titular en el área de Mecatrónica del Centro de Innovación y Desarrollo Tecnológico en Cómputo (CIDETEC) del Instituto Politécnico Nacional (IPN). El Dr. Molina ostenta un Doctorado en Ingeniería de Sistemas Robóticos y Mecatrónica por el mismo centro. Sus líneas de investigación y especialización abarcan la algoritmia, algoritmos bioinspirados, optimización, optimización aplicada a problemas de ingeniería y optimización evolutiva. Su trayectoria y conocimiento en estas áreas lo constituyen como un validador idóneo para los aspectos centrales de este proyecto.

Método de Consulta

La consulta se efectuó mediante una reunión formal el lunes 12 de mayo de 2025, de 14:40 a 15:15 horas, en las instalaciones del departamento de investigación

de Mecatrónica del CIDETEC-IPN. Previo a la discusión, se proporcionó al Dr. Molina un borrador del presente reporte técnico para su revisión. Durante la reunión, se expuso verbalmente el propósito, los objetivos y la evolución conceptual del proyecto, desde su concepción inicial hasta la formulación actual del modelo de simulación y optimización.

Resumen de Opiniones y Observaciones Obtenidas

Tras la presentación y discusión, el Dr. Molina Pérez aportó las siguientes valoraciones y observaciones clave:

- **Naturaleza del Problema de Optimización:**
 - Confirmó que el problema, tal como está planteado (encontrar masas que conduzcan a un exponente de Lyapunov¹ indicativo de estabilidad local), puede interpretarse como un **problema de factibilidad**² si el único objetivo es encontrar *cualquier* conjunto de masas que cumpla la restricción de estabilidad. No obstante, si se buscara el *mejor* conjunto (por ejemplo, el que minimice alguna otra métrica mientras mantiene la estabilidad, o el exponente de Lyapunov más cercano a cero sin ser positivo), se trataría de un problema de optimización con una función objetivo definida.
 - Subrayó la importancia de definir claramente si el objetivo es la mera factibilidad o la optimización de un criterio específico.
- **Variables de Decisión y Aplicabilidad del Modelo:**
 - Identificó correctamente que, al ser las masas las variables de decisión, el modelo se orienta a la estabilidad de sistemas de cuerpos celestes (limitado a dos en esta etapa).
 - Hipotetizó que considerar variables adicionales como la densidad o la relación masa/volumen (kg/m^3) podría enriquecer el modelo y la resolución del problema, sugiriendo una investigación en astrofísica y aerofísica.
- **Dinámica del Sistema y la Optimización:**
 - Inicialmente, dedujo que las masas serían fijas *después* del proceso de optimización para la simulación. Se clarificó que, si bien las masas son el resultado de la optimización y se usan para *una* simulación, el concepto más amplio del proyecto implica que estas podrían cambiar dinámicamente en la simulación general (aunque no durante un ciclo de optimización individual para encontrar *un* conjunto de masas estables).
 - Respecto a la simulación con REBOUND (específicamente con WHFast), reconoció su idoneidad para determinar trayectorias.

¹El **exponente de Lyapunov** (LE) caracteriza la tasa de separación de trayectorias cercanas en sistemas dinámicos; un LE máximo positivo indica caos, mientras que uno no positivo sugiere estabilidad [62, 63]. Es crucial en el *estudio del caos* y la *teoría de la dimensión* [62, 64, 65].

²Un **problema de factibilidad** busca determinar si existe al menos una solución que satisfaga un conjunto de restricciones, sin optimizar una función objetivo particular. Cf. [66].

- Consideró que el problema de optimización, tal como se describió (encontrar un conjunto de masas), no es inherentemente dinámico en el sentido de que la función de aptitud (exponente de Lyapunov para un conjunto dado de masas iniciales) no varía entre ejecuciones idénticas. Sin embargo, reconoció el aspecto dinámico *general* del sistema si las masas se modifican en función del tiempo t durante la simulación. Esta distinción es crucial: la optimización busca un estado inicial, la simulación general podría explorar cambios en ese estado.
- Sugirió la necesidad de un **activador** (*trigger*)³ que inicie una nueva búsqueda (optimización/factibilidad) si la aptitud del sistema cambia durante una simulación extendida, implicando un chequeo continuo del fitness.
- **Justificación Metodológica y Relevancia:**
 - Consideró el modelo como “super interesante”, con aplicaciones potenciales en la simulación científica para el desarrollo de motores físicos en videojuegos, aunque de interés prioritario en el ámbito aeroespacial y astrofísico.
 - Afirmó que el uso de **algoritmos bioinspirados**⁴ se justifica en problemas de simulación donde esta actúa como una “caja negra”, desconociéndose a priori la relación explícita entre las variables de entrada y la métrica de salida.
 - Mencionó que los **Algoritmos Genéticos**⁵ son particularmente adecuados para resolver problemas con características dinámicas (refiriéndose a la capacidad de adaptar la búsqueda si el entorno o los objetivos cambian, lo cual se alinea con la idea del *trigger*).

Análisis de la Validación Experta y Vinculación con el Proyecto

Las observaciones del Dr. Daniel Molina Pérez ofrecen una validación sustancial y directrices valiosas para el proyecto:

- **Validación de la Pertinencia del Problema:**
 - La distinción entre problema de factibilidad y optimización es fundamental y será clarificada en la formulación del problema dentro del reporte, enfocándose en la búsqueda de conjuntos de parámetros que satisfagan el criterio de estabilidad del exponente de Lyapunov.
 - Su énfasis en las aplicaciones aeroespaciales y astrofísicas refuerza la relevancia del dominio elegido, aunque el modelo pueda tener implicaciones secundarias en otras áreas como la simulación para videojuegos.

³Un **activador** (*trigger*) es una condición o evento que, al cumplirse, inicia un proceso subsecuente, como la re-evaluación o un nuevo ciclo de optimización.

⁴Los **algoritmos bioinspirados** son métodos de optimización inspirados en procesos naturales como la evolución o el comportamiento de enjambres [67, 68].

⁵Los **Algoritmos Genéticos** (AGs) son algoritmos bioinspirados basados en la selección natural y la genética, que operan sobre una población de soluciones para evolucionar hacia mejores resultados [68]. Ver Sección 3.8.

- La confirmación de que los algoritmos bioinspirados son una elección justificada para problemas donde la simulación actúa como evaluador de “caja negra” respalda la metodología central del proyecto.
- **Respaldo a la Solución Propuesta y Metodología:**
 - El reconocimiento de REBOUND y WHFast como herramientas adecuadas para la simulación de trayectorias apoya la elección tecnológica.
 - La sugerencia de un *trigger* para la re-optimización basada en el chequeo continuo del fitness se alinea con la visión de un sistema adaptable y será considerada para futuras extensiones del modelo, aunque la implementación actual se centre en encontrar un conjunto inicial estable.
 - Su comentario sobre los Algoritmos Genéticos siendo aptos para problemas dinámicos valida su elección como el paradigma bioinspirado principal, especialmente si se considera la adaptación del sistema a largo plazo.
- **Consideraciones para el Desarrollo y Reporte:**
 - La necesidad de un modelo matemático claro para la trayectoria (satisfecha por REBOUND) es un punto ya cubierto.
 - La hipótesis sobre la inclusión de densidad y kg/m^3 se tomará como una posible línea de investigación futura para enriquecer el modelo, aunque excede el alcance actual de dos cuerpos puntuales con masa como única variable de decisión.
 - Se refinará la descripción del “dinamismo” del problema, distinguiendo claramente entre la naturaleza estática de una evaluación de fitness individual (para un conjunto de masas) y el comportamiento dinámico de la simulación gravitacional en sí misma a lo largo del tiempo.

4.2. Análisis de Requerimientos

La sección de requerimientos constituye el pilar fundamental para el desarrollo exitoso de cualquier modelo o software, ya que establece de manera clara y estructurada las expectativas sobre lo que el modelo debe hacer y cómo debe comportarse. En este sentido, los requerimientos se clasifican en dos grandes categorías: **funcionales** y **no funcionales**. Los requerimientos funcionales (RF) detallan las acciones específicas que el modelo debe ejecutar, como la modificación dinámica de parámetros o la simulación de interacciones gravitacionales. Por su parte, los requerimientos no funcionales (RNF) especifican las cualidades y limitaciones del modelo, incluyendo aspectos como el rendimiento, la escalabilidad, la usabilidad y la compatibilidad con hardware determinado.

En este reporte técnico, se presenta un análisis detallado del contenido textual para identificar tanto los requerimientos explícitos como los implícitos, garantizando que el modelo de simulación gravitacional cumpla con los objetivos establecidos. Entre estos objetivos se incluye la capacidad de ajustar dinámicamente la masa

de los cuerpos celestes y asegurar el cumplimiento de las leyes de la física y la mecánica celeste. Además, se han tomado en cuenta factores esenciales como la eficiencia computacional, la estabilidad de las simulaciones y la accesibilidad para usuarios sin experiencia técnica avanzada.

La definición precisa y la documentación adecuada de estos requerimientos no solo orientarán el proceso de desarrollo, sino que también facilitarán la validación y verificación del modelo una vez implementado. Esto asegura que el producto final esté alineado con las necesidades y expectativas de los usuarios, ya sean investigadores, académicos o profesionales del sector.

4.2.1. Requerimientos Funcionales (RF)

Los requerimientos funcionales describen las capacidades que el modelo debe tener:

Tabla 4.1: Requerimientos Funcionales

ID	Descripción	Prioridad	Actor	Criterios de Aceptación
RF-001	El sistema debe permitir la modificación dinámica de la masa de los cuerpos celestes durante la simulación.	Alta	Usuario/Modelo	El usuario puede modificar la masa de los cuerpos celestes en tiempo real y la simulación responde adecuadamente a estos cambios.
RF-002	El sistema debe simular las interacciones gravitacionales entre dos cuerpos celestes.	Alta	Modelo	La simulación calcula y representa correctamente las fuerzas gravitacionales entre los cuerpos según las leyes físicas establecidas.
RF-003	El sistema debe usar el método multipolar rápido (FMM) y el algoritmo de Barnes-Hut para optimizar los cálculos gravitacionales.	Media	Modelo	Los algoritmos se implementan correctamente y mejoran el rendimiento de los cálculos en comparación con métodos directos.
RF-004	El sistema debe implementar algoritmos bioinspirados para ajustar dinámicamente los parámetros y encontrar valores idóneos que garanticen la estabilidad del sistema.	Alta	Modelo	Los algoritmos bioinspirados funcionan correctamente y logran mantener la estabilidad del sistema durante los cambios de masa.
RF-005	El sistema debe incluir un módulo para visualizar gráficamente la evolución de los dos cuerpos en iteraciones limitadas.	Media	Usuario	La visualización muestra claramente las trayectorias y estados de los cuerpos celestes durante la simulación.
RF-006	El sistema debe ofrecer una interfaz básica para modificar parámetros y ver resultados.	Media	Usuario	El usuario puede interactuar con la interfaz para ajustar parámetros y visualizar los cambios en la simulación.

4.2.2. Requerimientos No Funcionales (RNF)

Los requerimientos no funcionales especifican las cualidades y restricciones del modelo, incluyendo los dos implícitos que me pediste añadir:

Tabla 4.2: Requerimientos No Funcionales

ID	Descripción	Prioridad	Actor	Criterios de Aceptación
RNF-001	El sistema debe optimizar la complejidad de las simulaciones de n-cuerpos para ser eficiente.	Alta	Modelo	La simulación se ejecuta en tiempos razonables sin consumir excesivos recursos computacionales.
RNF-002	El sistema debe permitir la extensión futura a más de dos cuerpos.	Media	Modelo	La arquitectura del sistema es escalable y puede adaptarse para simular más cuerpos sin cambios estructurales mayores.
RNF-003	El sistema debe respetar las leyes de la física y la mecánica celeste en las interacciones gravitacionales.	Alta	Modelo	Las simulaciones producen resultados coherentes con las leyes físicas establecidas.
RNF-004	El sistema debe mantener simulaciones estables al modificar parámetros dinámicamente.	Alta	Modelo	La simulación no colapsa ni presenta comportamientos erráticos cuando se modifican los parámetros durante la ejecución.
RNF-005	La interfaz debe ser intuitiva y accesible sin necesidad de conocimientos avanzados.	Media	Usuario	Usuarios sin experiencia técnica pueden utilizar el sistema sin dificultades significativas.
RNF-006	El sistema debe ejecutarse en hardware de gama media (procesadores multinúcleo, 16 GB de RAM).	Media	Modelo	El sistema funciona correctamente en equipos con las especificaciones mínimas establecidas.
RNF-007*	El sistema debe integrarse con entornos virtuales como Unreal Engine.	Baja	Modelo	La integración con Unreal Engine u otros entornos similares es posible y funcional.
RNF-008	El sistema debe ser modular para facilitar actualizaciones y correcciones.	Media	Desarrollador	Los componentes del sistema están claramente separados y pueden modificarse independientemente.
RNF-009	El sistema debe incluir documentación clara para usuarios y desarrolladores.	Media	Desarrollador	La documentación es completa, comprensible y cubre todos los aspectos relevantes del sistema.

* El requerimiento no funcional No. 07 (RNF-007), solo hace mención a la idea conceptual del requerimiento, no se plantea su elaboración en el marco temporal que se tiene para la conclusión del trabajo terminal.

4.3. Planteamiento del Problema Principal: Factibilidad de Estabilidad del Modelo

El objetivo fundamental y principal de este proyecto es determinar si existe al menos una configuración de masas (m_1, m_2) para un sistema de dos cuerpos celestes que produzca un comportamiento gravitacional localmente estable. Esta estabilidad se cuantifica principalmente a través del Exponente de Lyapunov (LE), donde valores más bajos (idealmente negativos, $LE < 0$) indican mayor estabilidad. Este es, en esencia, un *problema de factibilidad*: buscamos probar la existencia de al menos una solución que satisfaga un conjunto de criterios.

A continuación, se presenta la formulación matemática de este problema de factibilidad. Posteriormente, se detallará cómo este problema puede ser abordado mediante una formulación de optimización, una estrategia común para resolver problemas de factibilidad complejos, especialmente cuando se utilizan algoritmos bioinspirados.

4.3.1. Definiciones Preliminares: Variables, Parámetros y Función de Evaluación

Para formular el problema, primero definimos las variables de decisión, los parámetros del sistema, los conjuntos relevantes y la función de evaluación.

Variables de Decisión

Las decisiones fundamentales giran en torno a las masas de los dos cuerpos celestes:

- m_1 : Masa del primer cuerpo celeste.
 - Naturaleza: Continua.
 - Unidades: Kilogramos (kg).
- m_2 : Masa del segundo cuerpo celeste.
 - Naturaleza: Continua.
 - Unidades: Kilogramos (kg).

Parámetros del Modelo

Estos son los valores fijos o predefinidos que contextualizan el problema:

- $\mathcal{C}_{\text{ini}}$: Conjunto de condiciones iniciales del sistema. Incluye:
 - $\mathbf{r}_{1,0}, \mathbf{r}_{2,0}$: Vectores de posición inicial para el cuerpo 1 y el cuerpo 2, respectivamente (e.g., (x_0, y_0, z_0) para cada uno). Unidades: metros (m).
 - $\mathbf{v}_{1,0}, \mathbf{v}_{2,0}$: Vectores de velocidad inicial para el cuerpo 1 y el cuerpo 2, respectivamente (e.g., $(v_{x,0}, v_{y,0}, v_{z,0})$ para cada uno). Unidades: metros por segundo (m/s).
- $\mathcal{P}_{\text{sim}}$: Conjunto de parámetros de la simulación gravitacional. Incluye:

- G : Constante de gravitación universal. Unidades: $\text{N}\cdot\text{m}^2/\text{kg}^2$. (Valor aproximado: $6.674 \times 10^{-11} \text{ N}\cdot\text{m}^2/\text{kg}^2$)
- Δt : Paso de tiempo de integración numérica. Unidades: segundos (s).
- T_{max} : Tiempo máximo de simulación para cada evaluación del Exponente de Lyapunov. Unidades: segundos (s).
- I_{type} : Especificación del tipo de integrador numérico utilizado (e.g., WHFast).
- Otros parámetros relevantes para los algoritmos de cálculo de interacciones (e.g., parámetros específicos de FMM, Barnes-Hut si se usan).
- α : Constante real positiva que define la cota superior para la relación de masas (m_2/m_1). Adimensional.
- β : Constante real positiva que define la cota inferior para la relación de masas (m_2/m_1). Adimensional.
- S_{target} : Umbral de estabilidad objetivo para el Exponente de Lyapunov. Para que una configuración sea considerada factiblemente estable, se requiere $LE \leq S_{\text{target}}$.
 - Este valor es crucial para el problema de factibilidad. Puede ser un L_{target} específico (si $L_{\text{target}} < 0$ está definido como el objetivo mínimo de estabilidad).
 - Alternativamente, S_{target} puede ser $-\delta_{\text{stability}}$, donde $\delta_{\text{stability}}$ es una constante positiva pequeña (e.g., 10^{-5}), asegurando que LE sea significativamente negativo.
 - Unidades: Adimensional o s^{-1} .
- ε_m : Un valor positivo pequeño para asegurar que las masas sean estrictamente positivas (e.g., 10^{-6} kg). Unidades: kg.
- $\varepsilon_{\text{ineq}}$: Un valor positivo pequeño para manejar desigualdades estrictas en restricciones numéricas (e.g., 10^{-9}). Adimensional o unidades consistentes con la restricción.
- $m_{1,\text{min}}, m_{1,\text{max}}$: Límites absolutos inferior y superior para la masa m_1 (opcional). Unidades: kg.
- $m_{2,\text{min}}, m_{2,\text{max}}$: Límites absolutos inferior y superior para la masa m_2 (opcional). Unidades: kg.

Función de Evaluación (Exponente de Lyapunov)

El Exponente de Lyapunov, LE , es una función que depende de las masas de los cuerpos y del conjunto de parámetros de simulación y condiciones iniciales. No es una forma cerrada simple, sino el resultado de una simulación numérica:

$$LE(m_1, m_2; \mathcal{C}_{\text{ini}}, \mathcal{P}_{\text{sim}}): \mathbb{R}^+ \times \mathbb{R}^+ \rightarrow \mathbb{R} \quad (4.1)$$

Este valor, $LE(m_1, m_2; \mathcal{C}_{\text{ini}}, \mathcal{P}_{\text{sim}})$, es el Exponente de Lyapunov calculado para una configuración dada de masas m_1, m_2 , con condiciones iniciales $\mathcal{C}_{\text{ini}}$ y parámetros de simulación $\mathcal{P}_{\text{sim}}$. Sus unidades son adimensionales o s^{-1} .

4.3.2. Definición Formal Matemática del Problema de Factibilidad

Sea el espacio de decisión $\mathcal{M} = \{(m_1, m_2) \in \mathbb{R}^2 : m_1, m_2 \geq 0\}$. Definimos el conjunto factible:

$$\mathcal{F} = \left\{ (m_1, m_2) \in \mathcal{M} \left| \begin{array}{l} m_1 \geq \varepsilon_m, \\ m_2 \geq \varepsilon_m, \\ m_2 - \beta m_1 \geq 0, \\ \alpha m_1 - m_2 \geq 0, \\ 10 m_1 - m_2 \geq \varepsilon_{\text{ineq}}, \\ LE(m_1, m_2; \mathcal{C}_{\text{ini}}, \mathcal{P}_{\text{sim}}) \leq S_{\text{target}}, \\ m_{1,\text{mín}} \leq m_1 \leq m_{1,\text{máx}}, \\ m_{2,\text{mín}} \leq m_2 \leq m_{2,\text{máx}} \end{array} \right. \right\}. \quad (4.2)$$

El *problema de factibilidad* consiste en:

$$\text{Encontrar } (m_1, m_2) \in \mathcal{F}.$$

Si $\mathcal{F} \neq \emptyset$, existe al menos una configuración de masas que satisface todas las restricciones y, por tanto, el sistema puede alcanzar estabilidad bajo el criterio S_{target} .

4.3.3. Abordaje del Problema de Factibilidad mediante Formulación de Optimización

Para resolver el problema de factibilidad definido en la sección 4.3.2, especialmente cuando la función LE es compleja y el espacio de búsqueda es extenso, es ventajoso reformularlo como un problema de optimización. Esta aproximación es particularmente adecuada para la aplicación de algoritmos bioinspirados.

La estrategia consiste en buscar la minimización del Exponente de Lyapunov (LE) sujeto a las restricciones del dominio. Si el proceso de optimización encuentra una solución (m_1^*, m_2^*) cuyo valor $LE(m_1^*, m_2^*)$ satisface $LE \leq S_{\text{target}}$ y todas las demás restricciones, entonces se ha encontrado una solución al problema de factibilidad.

Descripción Verbal del Objetivo y Restricciones de la Formulación de Optimización

- **Objetivo de la Formulación de Optimización:** Se busca minimizar el valor del Exponente de Lyapunov $Z = LE(m_1, m_2; \mathcal{C}_{\text{ini}}, \mathcal{P}_{\text{sim}})$. El propósito principal de esta minimización es encontrar *alguna* configuración de masas (m_1, m_2) para la cual el valor de LE resultante sea igual o inferior al umbral de estabilidad S_{target} , satisfaciendo a su vez todas las demás restricciones (positividad, relaciones de masa, rangos).

- **Restricciones del Dominio de Búsqueda (para la optimización):**

Las mismas restricciones que definen el espacio de soluciones factibles (sección 4.3.2, puntos 1, 2, 3, 5 y 6), excluyendo la condición de $LE \leq S_{\text{target}}$ (punto 4 de la sección 4.3.2), que se convierte en el criterio de éxito de la búsqueda.

Definición Formal Matemática de la Formulación de Optimización

El problema se formula matemáticamente como:

$$\begin{aligned}
 &\underset{m_1, m_2}{\text{Minimizar}} \quad Z(m_1, m_2) = LE(m_1, m_2; \mathcal{C}_{\text{ini}}, \mathcal{P}_{\text{sim}}) \\
 &\text{sujeto a} \\
 &\quad m_1 \geq \varepsilon_m \\
 &\quad m_2 \geq \varepsilon_m \\
 &\quad m_2 - \beta \cdot m_1 \geq 0 \\
 &\quad \alpha \cdot m_1 - m_2 \geq 0 \\
 &\quad 10 \cdot m_1 - m_2 \geq \varepsilon_{\text{ineq}} \\
 &\quad m_{1,\text{min}} \leq m_1 \leq m_{1,\text{max}} \quad (\text{Opcional}) \\
 &\quad m_{2,\text{min}} \leq m_2 \leq m_{2,\text{max}} \quad (\text{Opcional}) \\
 &\quad m_1, m_2 \in \mathbb{R}
 \end{aligned} \tag{4.3}$$

Criterio de Éxito para Resolver el Problema de Factibilidad mediante esta Optimización

Se considera que se ha encontrado una solución al problema de factibilidad original (sección 4.3.2) si el proceso de optimización produce un par de masas (m_1^*, m_2^*) que:

1. Satisface todas las restricciones del problema de optimización (4.3).
2. Y para el cual el valor de la función objetivo evaluada $Z^* = LE(m_1^*, m_2^*; \mathcal{C}_{\text{ini}}, \mathcal{P}_{\text{sim}})$ cumple con la condición de estabilidad original:

$$Z^* \leq S_{\text{target}} \tag{4.4}$$

Si, tras ejecutar el algoritmo de optimización, el valor mínimo de Z encontrado (Z_{min}) es persistentemente mayor que S_{target} (o si no se puede encontrar ninguna solución que satisfaga las restricciones de la ecuación (4.3)), se concluiría que, dentro del alcance y las capacidades del método de optimización utilizado, no se ha podido encontrar una configuración de masas que resuelva el problema de factibilidad.

4.3.4. Consideraciones Adicionales

- **Optimización Pura de la Estabilidad (Paso Secundario Opcional):**

Si el objetivo secundario fuera, además de encontrar *una* configuración estable, encontrar la configuración *más* estable (el LE más negativo posible), la formulación de optimización (ecuación (4.3)) sigue siendo válida. El interés no se detendría al alcanzar S_{target} , sino que el algoritmo continuaría buscando minimizar LE tanto como sea posible.

- **Interpretación Física de Restricciones:** La relación $\beta \leq \frac{m_2}{m_1} \leq \alpha$ y la condición $m_2 < 10 \cdot m_1$ son ejemplos de restricciones que aseguran proporciones de masa físicamente significativas o relevantes para el estudio específico.

CAPÍTULO 5

Análisis

5.1. Análisis de Procesos Internos del Programa

El análisis detallado de los procesos internos del software constituye un pilar esencial para el diseño, implementación y validación del modelo de simulación gravitacional propuesto en este proyecto. Comprender la secuencia de operaciones, el flujo de datos y la interacción entre los componentes clave (como la biblioteca REBOUND, la lógica de modificación y la interfaz de usuario) es fundamental para abordar el desafío central del proyecto: habilitar la modificación dinámica de parámetros, como la masa, en un sistema de N-cuerpos (inicialmente abordado con dos cuerpos) durante la ejecución de la simulación.

Esta sección establece la base operativa sobre la cual se construirá la solución. Proporciona el mapa necesario para implementar las innovaciones clave del proyecto –la modificación dinámica de parámetros– de manera estructurada, eficiente y verificable, asegurando que el modelo final cumpla con el objetivo de superar las limitaciones de las herramientas de simulación actuales.

5.1.1. Proceso Interno 01: Captura Parámetros

Objetivo del Proceso

El propósito principal de la actividad “**Captura Parámetros**” es recopilar, validar y almacenar todos los parámetros de configuración necesarios proporcionados por el usuario a través de la interfaz gráfica (UI). Estos parámetros son esenciales para definir cómo se ejecutará tanto la simulación gravitacional como el proceso de optimización bioinspirado, asegurando que la configuración sea correcta y coherente antes de iniciar la ejecución del sistema.

Entradas Principales

- **Evento de inicio:** Interacción inicial del usuario con la UI para comenzar la configuración de parámetros.
- **Interacciones del usuario:** Acciones como clicks, selecciones e ingreso de texto o números en los campos específicos de la UI.

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 15.1.

1. Presentar Formulario/Interfaz UI

- La UI muestra una pantalla o conjunto de controles para ingresar la configuración
- Campos presentados incluyen:
 - **Parámetros a Optimizar:** Rangos (mínimo/máximo) de parámetros variables
 - **Restricciones del Sistema:** Condiciones obligatorias (ej. proporción máxima de masas)
 - **Configuración del Optimizador:**
 - Indicador de fitness preseleccionado (Exponente de Lyapunov LE)
 - Criterios de parada (n° máximo de generaciones, umbral de mejora)
 - **Parámetros del Algoritmo Bioinspirado:** Tamaño de población, tasas de mutación/crossover
 - **Configuración de la Simulación Base:**
 - Parámetros fijos: G , $T_{\max}$, dt
 - Selección de integrador de REBOUND (si aplica)
 - **Configuración de Visualización:**
 - Instantes de tiempo t_{vis} para captura de datos
 - Variables a visualizar (trayectoria 2D, energía vs tiempo)

2. Recibir Entradas del Usuario

- La UI captura activamente valores ingresados/seleccionados en cada campo

3. Iniciar Ciclo de Validación

- Validaciones incluyen:
 - 1(a) Tipo de dato correcto (valores numéricos donde corresponda)
 - 1(b) Rangos lógicos (mín < máx)
 - 1(c) Criterios de parada coherentes (n° generaciones > 0)
 - 1(d) $t_{\text{vis}} \in [0, T_{\text{máx}}]$
 - 1(e) Consistencia $T_{\text{máx}} \geq \text{último } t_{\text{vis}}$
 - 1(f) Restricciones físicamente posibles (ej. proporción de masas > 0)

4. Decisión de Validez

- **Si ocurren errores:**
 - Generación de mensajes de error específicos
 - Resaltado de campos erróneos en UI
 - Retorno al paso de entrada de datos
- **Si válido:**
 - Creación de estructura `ConfigurationData`

5. Almacenar Configuración Validada

- Valores guardados en estructura `ConfigurationData`
- Accesible por otros componentes del sistema

6. Habilitar Inicio

- Activación de botón ‘‘Iniciar Simulación/Optimización’’

Lógica Interna y Decisiones

- Ciclo de validación activado por acción de usuario
- Nodo de decisión principal: ¿Todas las entradas válidas?
 - Bucle de corrección hasta validación exitosa
 - Transición a ejecución tras validación satisfactoria

Manejo de Datos Específico

- **Datos crudos:** Valores directos de UI
- **Indicadores de validación:** Resultados booleanos por campo
- `ConfigurationData`: Estructura final con parámetros validados

Salidas Principales

- Estructura `ConfigurationData` completa
- Estado UI actualizado (mensajes de error o botón habilitado)

Interacciones Internas

- Comunicación con componentes UI (campos de texto, listas, botones)
- Preparación de `ConfigurationData` para algoritmo bioinspirado

Diagrama del Proceso

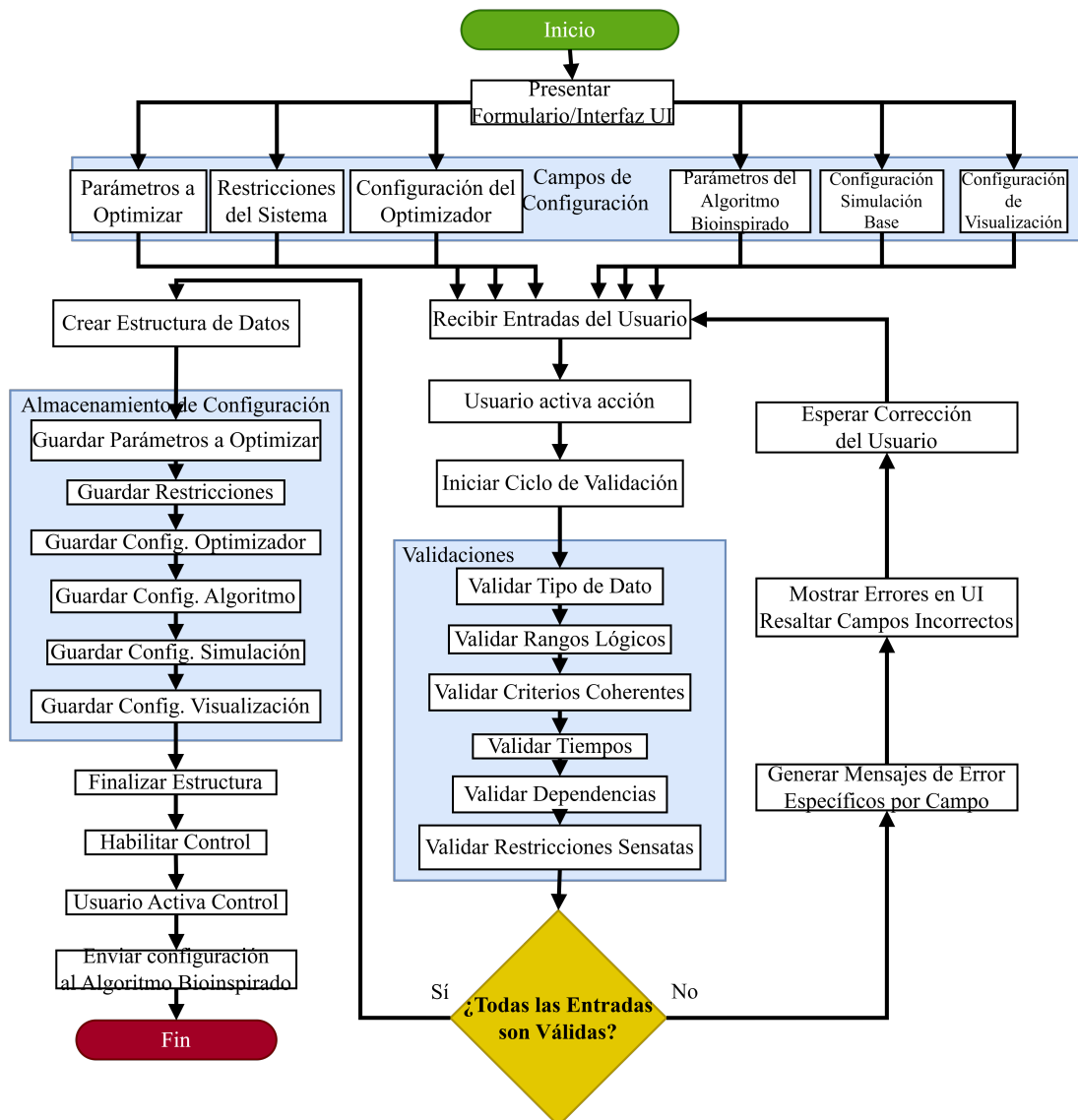


Figura 5.1: Diagrama de Proceso Interno 01: Captura de Parámetros

5.1.2. Proceso Interno 02: Inicializar Población

Objetivo del Proceso

El propósito principal de la actividad “**Inicializar Población**” es generar un conjunto inicial de N individuos que sirvan como punto de partida para el algoritmo bioinspirado del proyecto “*Modelo para representar comportamientos gravitacionales con dos cuerpos*”. Cada individuo representa un conjunto completo de parámetros a optimizar (ej masas de cuerpos), generados dentro de los rangos válidos especificados en la configuración.

Entradas Principales

- **ConfigurationData:** Estructura con:
 - Tamaño de población $N \in \mathbb{N}$
 - Rangos por parámetro: $[\text{mín}, \text{máx}]$ (ej. $[\text{masa}_{1,2 \text{ mín}}, \text{masa}_{1,2 \text{ máx}}]$)
 - *Nota:* Restricciones funcionales se manejarán posteriormente

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 25.2

1. Leer Configuración Relevante

- Extraer de ConfigurationData:
 - N
 - Lista de parámetros a optimizar
 - Rangos asociados
- Verificar coherencia: $\forall \text{parámetro}, \text{mín} < \text{máx}$

2. Inicializar Contenedor de Población

- Crear estructura vacía (lista/array) para N individuos

3. Iniciar Bucle de Creación de Población

- (a) Inicializar contador $i = 0$
- (b) **Mientras** $i < N$:
 - Generar parámetros candidatos:
 - $\forall \text{parámetro: valor aleatorio} \in [\text{min}, \text{max}]$
 - Agregar candidato a la población
 - $i \leftarrow i + 1$

4. Verificar Población Generada

- Confirmar $|población| = N$
- (*Opcional*) Analizar distribución/diversidad

5. Finalizar y Devolver Población Inicial

- Retornar estructura poblada lista para optimización

Lógica Interna y Decisiones

- **Control del bucle:** Garantiza generación exacta de N individuos
- **Generación en rangos:**
 - Aleatorización restringida por $[mín, máx]$
 - Elimina necesidad de validación posterior
- **Verificación crítica:**
 - Precondición: $mín < máx \forall$ parámetro
- **Diversidad opcional:**
 - Métricas estadísticas no requeridas en flujo principal

Manejo de Datos Específico

- **Entrada:** ConfigurationData
- **Intermedios:**
 - Valores aleatorios $\in$ rangos
 - Estructuras temporales por individuo
- **Salida:** Lista/array de N individuos

Salidas Principales

- **Población inicial:**
 - N individuos con parámetros $\in$ rangos
 - Entrada para evaluación de LE

Interacciones Internas

- **Con ConfigurationData:** Lectura de parámetros
- **Con generador aleatorio:** Producción de valores válidos
- **Con estructura de población:** Almacenamiento y gestión

Diagrama del Proceso

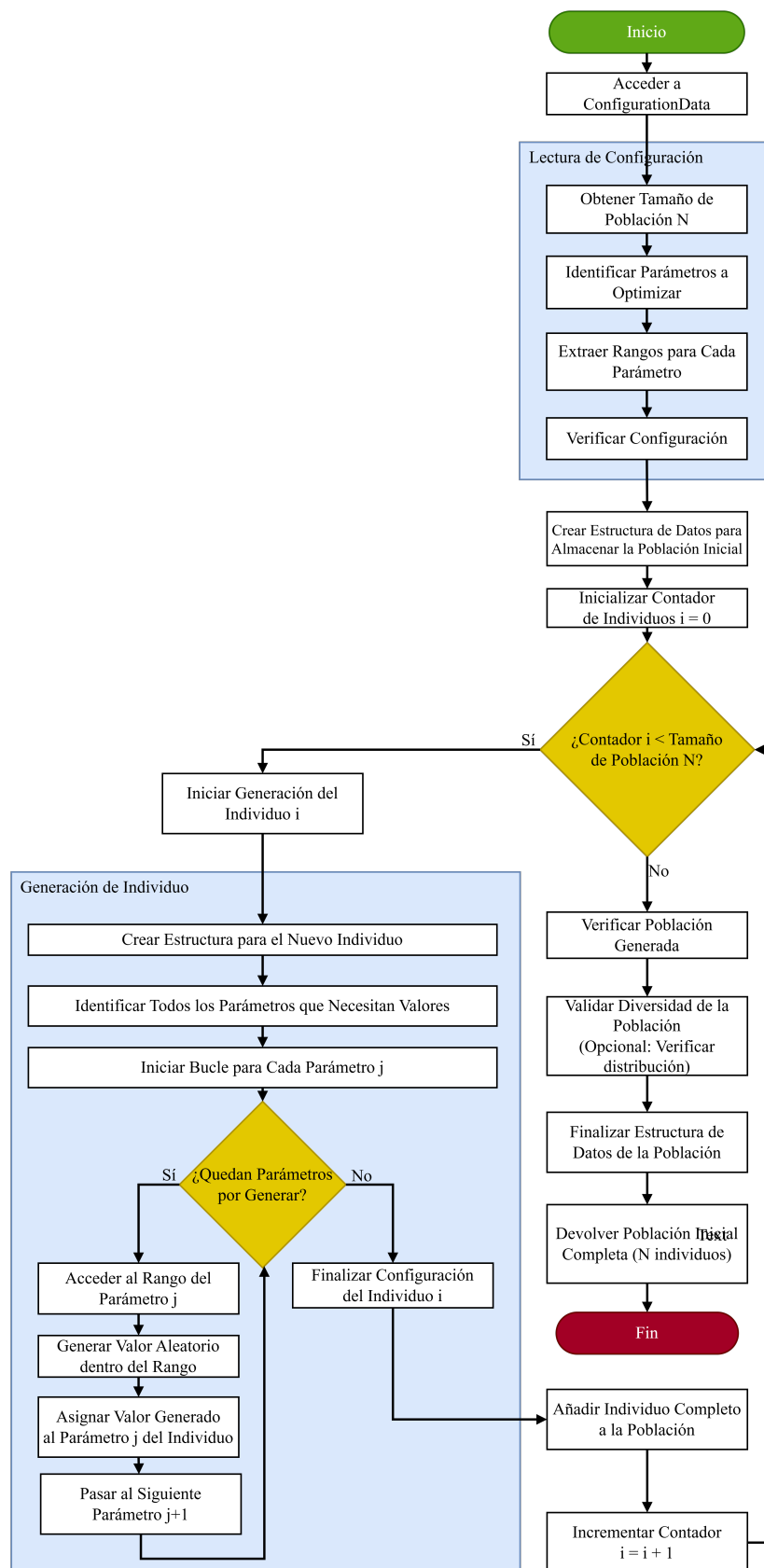


Figura 5.2: Diagrama de Proceso Interno 02: Inicializar Población

5.1.3. Proceso Interno 03: Generar Nueva Generación

Objetivo del Proceso

El propósito principal de la actividad “**Generar Nueva Generación**” es crear una nueva población de N individuos mediante operadores genéticos (Selección por Torneo, Cruzamiento SBX y Mutación Polinomial), garantizando que los parámetros permanezcan dentro de los rangos [mín, máx]. La evaluación de fitness se realiza posteriormente.

Entradas Principales

- **Población actual:**
 - N individuos con parámetros y F_p (fitness penalizado)
- **ConfigurationData:**
 - $N, P_c, P_m, \eta_c, \eta_m, k$
 - Rangos [mín _{i} , máx _{i}] por parámetro

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 35.3

1. Inicializar Nueva Población

- Crear new_population vacío (capacidad N)

2. Iniciar Bucle de Generación

(a) **While** |new_population| < N :

Paso 2a: Selección de padres

- Parent1 = Torneo3(k, F_p)
- Parent2 = Torneo3(k, F_p)

Paso 2b: Cruzamiento SBX

- Generar $r_1 \sim U[0, 1]$
- **If** $r_1 < P_c$:
 - {Offspring1, Offspring2} $\leftarrow$ SBX(Parent1, Parent2, η_c)
- **Else**:
 - Clonar padres

Paso 2c: Mutación Polinomial

Para cada descendiente en {Offspring1, Offspring2}:

Para cada parámetro x_i del descendiente:

- Generar $r_2 \sim U[0, 1]$
- **If** $r_2 < P_m$:
 - $x_i \leftarrow$ MutaciónPolinomial($x_i, \eta_m, \min_i, \max_i$)

Paso 2d: Ajuste de límites

Para cada parámetro x_i en cada descendiente:

- $x_i \leftarrow \text{clip}(x_i, \min_i, \max_i)$

Paso 2e: Añadir descendientes

- Agregar `Offspring1` a `new_population`
- Si $|\text{new_population}| < N$, agregar `Offspring2`

3. Devolver Nueva Población

- Retornar `new_population` con N individuos

Lógica Interna y Decisiones

- **Control del bucle:**
 - Condición de término: $|\text{new_population}| = N$
 - Manejo de N impar con adición condicional de `Offspring2`
- **Operadores probabilísticos:**
 - Cruzamiento: $P_c \in [0, 1]$
 - Mutación: $P_m \in [0, 1]$ por parámetro
- **Preservación de restricciones:**
 - Ajuste post-operadores: $x_i \in [\min_i, \max_i]$

Manejo de Datos Específico

- **Entradas:**
 - Población actual: Lista de estructuras $\{\text{parámetros}, F_p\}$
 - Parámetros operadores: $\{P_c, P_m, \eta_c, \eta_m, k\}$
- **Intermedios:**
 - Padres seleccionados
 - Descendientes pre-ajuste
- **Salida:**
 - `new_population`: Lista de N individuos no evaluados

Salidas Principales

- **Nueva población:**
 - N individuos con parámetros en rangos válidos
 - Listos para evaluación de fitness

Interacciones Internas

- **Con población actual:** Lectura de F_p para torneos
- **Con módulos genéticos:**
 - Torneo3(k), SBX3(η_c), MutaciónPolinomial3(η_m)
- **Con gestor de restricciones:** Aplicación de clip()

Diagrama del Proceso

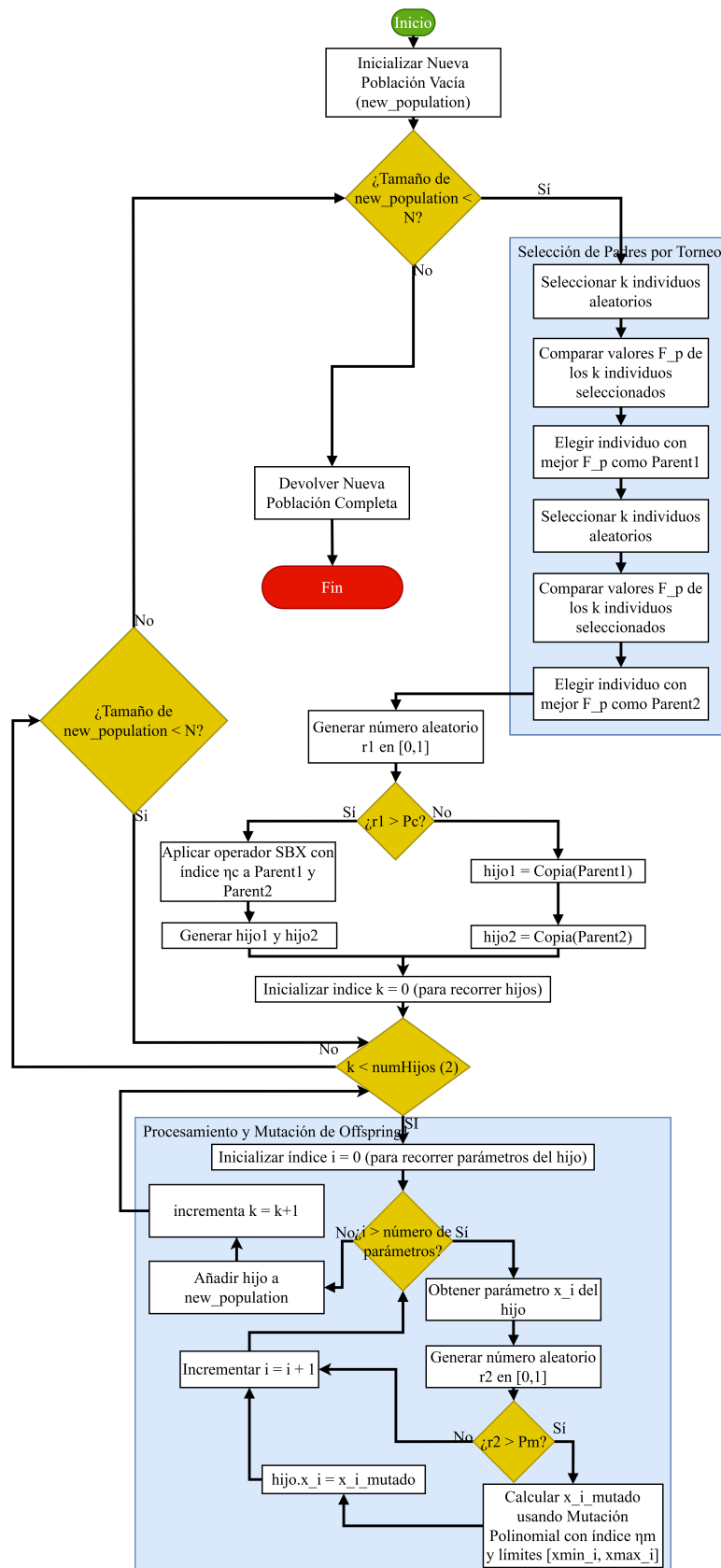


Figura 5.3: Diagrama de Proceso Interno 03: Generar Nueva Generación

5.1.4. Proceso Interno 04: Crear Nueva Simulación

Objetivo del Proceso

Instanciar un entorno de simulación vacío usando REBOUND, preparado para:

- Configuración posterior de parámetros
- Simulación de dinámica gravitacional de 2 cuerpos
- Cálculo del Exponente de Lyapunov (LE)

Entradas Principales

- **Trigger:** Señal de solicitud de creación
- *Nota:* No se requieren datos específicos de entrada

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 45.4

1. Invocar API de REBOUND

```
sim = rebound.Simulation()
```

2. Inicialización Interna

REBOUND ejecuta:

- Asignación de memoria para estructura `Simulation`
- Creación de contenedor vacío para partículas
- Configuración de valores por defecto:
 - $t_0 = 0$
 - $G = 6.674 \times 10^{-11} \text{ N} \cdot \text{m}^2/\text{kg}^2$
 - Integrador: `WHFast`

3. Generar Referencia

- Retorna handler/puntero: `Simulation*`
- Ejemplo: `0x7ffe3c3d8b50`

4. Transferir Control

- Pasa referencia al proceso “Definir Condiciones”

Lógica Interna y Decisiones

- **Flujo lineal:** Sin bifurcaciones condicionales
- **Manejo de errores:**
 - Excepciones de API manejadas en nivel superior
- **Defaults:**
 - Valores modificables en configuración posterior

Manejo de Datos Específico

- **Entrada:** Solo trigger de activación
- **Intermedios:**
 - Estructura `Simulation` en memoria
 - Estado inicial: $t = 0$, $partículas = \emptyset$
- **Salida:** Referencia a `Simulation`

Salidas Principales

- **`Simulation*`:** Puntero a instancia REBOUND vacía
 - Lista para configuración de parámetros
 - Integrable en flujo de optimización

Interacciones Internas

- **Con API REBOUND:**
 - Constructor `rebound.Simulation4()`
 - Gestión interna de memoria
- **Con subsistema de memoria:**
 - Asignación dinámica (4KB a MB según complejidad)
- **Con flujo de control:**
 - Paso de referencia a siguiente módulo

Diagrama del Proceso

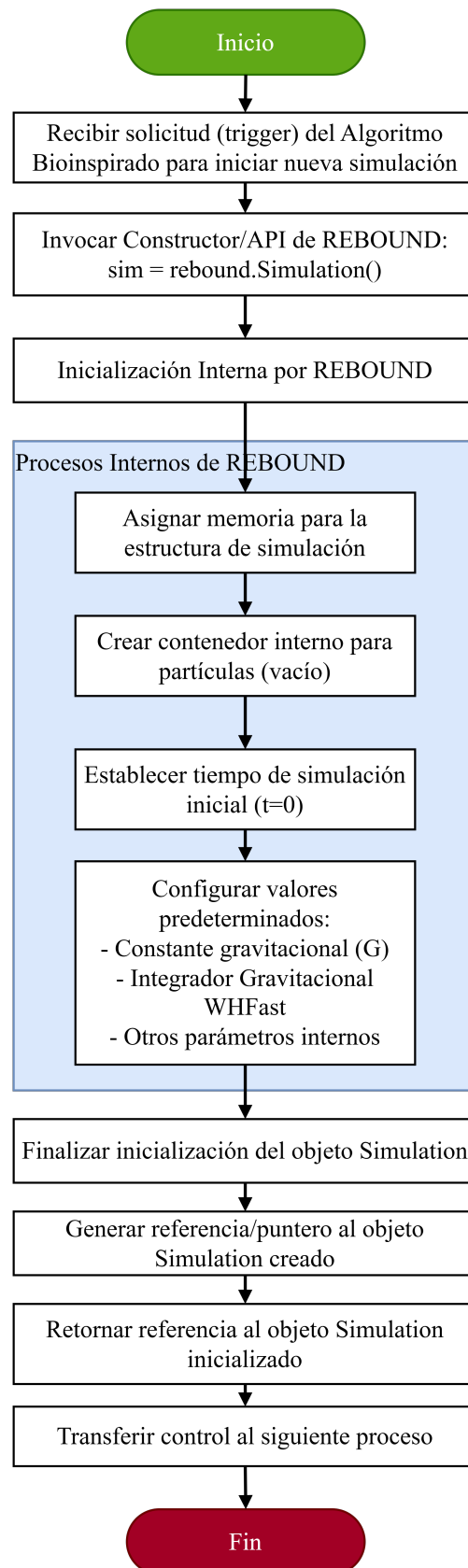


Figura 5.4: Diagrama de Proceso Interno 04: Crear Simulación

5.1.5. Proceso Interno 05: Agregar Cuerpos

Objetivo del Proceso

Incorporar un cuerpo celeste con propiedades físicas ($m, \mathbf{x}, \mathbf{v}$) a una simulación REBOUND existente, permitiendo:

- Configuración de sistemas de 2 cuerpos
- Cálculo posterior de estabilidad dinámica (LE)
- Integración en el ciclo de optimización

Entradas Principales

- **sim**: Referencia a objeto `Simulation` de REBOUND
- **body_params**: Estructura con:
 - $m \in \mathbb{R}^+$ (masa)
 - $\mathbf{x} = [x, y, z]$ (posición inicial)
 - $\mathbf{v} = [v_x, v_y, v_z]$ (velocidad inicial)

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 5.12

1. Recibir Parámetros

- Capturar `sim` y `body_params`

2. Desempaquetar Parámetros

- Extraer: $m, x, y, z, v_x, v_y, v_z$

3. Validación Opcional

- Chequear:
 - $\forall v \in \{m, x, y, z, v_x, v_y, v_z\} : v \neq \text{NaN}, \infty$
 - $m > 0$

4. Invocar API REBOUND

```
sim.add(m=m, x=x, y=y, z=z,
        vx=vx, vy=vy, vz=vz)
```

5. Procesamiento Interno REBOUND

- Asignar memoria para nueva partícula
- Actualizar contador: $n_{\text{partículas}} \leftarrow n_{\text{partículas}} + 1$

- Recalcular propiedades del sistema (opcional)

6. Verificación de Estado (Opcional)

- Confirmar $n_{\text{partículas}} = n_{\text{prev}} + 1$

Lógica Interna y Decisiones

- **Flujo lineal:** Sin bifurcaciones principales
- **Validaciones:**
 - Opcionales, dependientes de implementación
 - Manejo de errores delegado a REBOUND

Manejo de Datos Específico

- **Entradas:**
 - Referencia a simulación existente
 - Parámetros físicos estructurados
- **Intermedios:**
 - Estructura interna de partícula en REBOUND
- **Salida:**
 - Simulación modificada in-place

Salidas Principales

- `sim` actualizado con nuevo cuerpo
 - Listo para integración numérica
 - Configuración adicional posible

Interacciones Internas

- **Con API REBOUND:**
 - Método `add5()` para gestión de partículas
 - Administración interna de memoria
- **Con subsistema físico:**
 - Actualización de propiedades del sistema

Diagrama del Proceso

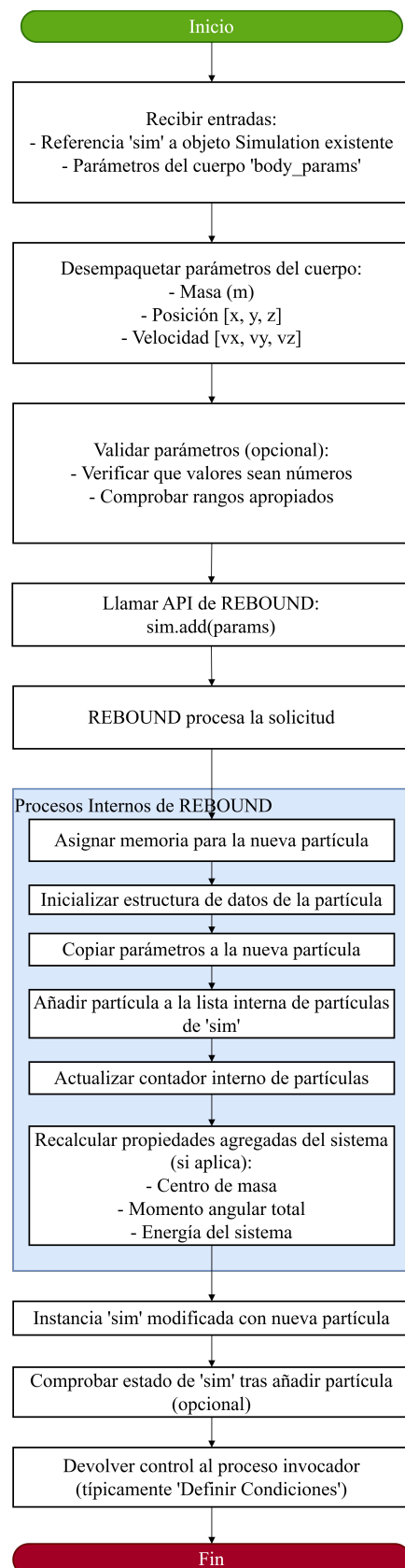


Figura 5.5: Diagrama de Proceso Interno 05: Agregar Cuerpos

5.1.6. Proceso Interno 06: Definir Condiciones

Objetivo del Proceso

Configurar parámetros operativos clave en una simulación REBOUND para:

- Establecer condiciones de integración numérica
- Definir constantes físicas fundamentales
- Preparar simulación para cálculo de LE

Entradas Principales

- **sim**: Referencia a objeto `Simulation` configurado
- **sim_params**: Estructura con:
 - $dt \in \mathbb{R}^+$ (paso temporal)
 - $G \in \mathbb{R}^+$ (constante gravitacional)
 - $T_{\max} \in \mathbb{R}^+$ (tiempo máximo)

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 65.6

1. Recibir Instancia y Parámetros

- Capturar `sim` y `sim_params`

2. Desempaquetar Parámetros

- Extraer: $dt, G, T_{\max}$

3. Validación Opcional

$$\begin{cases} dt > 0 \\ G > 0 \\ T_{\max} > \text{sim.t} \end{cases}$$

4. Establecer Parámetros en REBOUND

```
sim.dt = dt # Paso temporal
sim.G = G   # Constante gravitacional
```

5. Procesar Tiempo Máximo

- Almacenar $T_{\max}$ para control externo
- No modifica estado interno de `sim`

6. Verificación de Compatibilidad

- Chequear coherencia integrador/parámetros
- Ejemplo: `sim.integrador == IAS15`

Lógica Interna y Decisiones

- **Validaciones:**
 - Condicionales según política del sistema
 - Manejo de errores por valores inválidos
- **Compatibilidad:**
 - Verificación implícita de estabilidad numérica

Manejo de Datos Específico

- **Entradas:**
 - Parámetros de configuración temporal/física
 - Instancia REBOUND preconfigurada
- **Intermedios:**
 - Estado modificado de `sim`
 - $T_{\max}$ almacenado para ejecución
- **Salida:**
 - Simulación lista para ejecución
 - $T_{\max}$ disponible como metadato

Salidas Principales

- `sim` configurado con:
 - dt para integración numérica
 - G para interacciones gravitatorias
- $T_{\max}$ como parámetro de control

Interacciones Internas

- **Con API REBOUND:**
 - Modificación directa de propiedades `sim`
 - Consulta de estado del integrador
- **Con flujo de control:**
 - Transmisión de $T_{\max}$ a siguiente etapa

Diagrama del Proceso

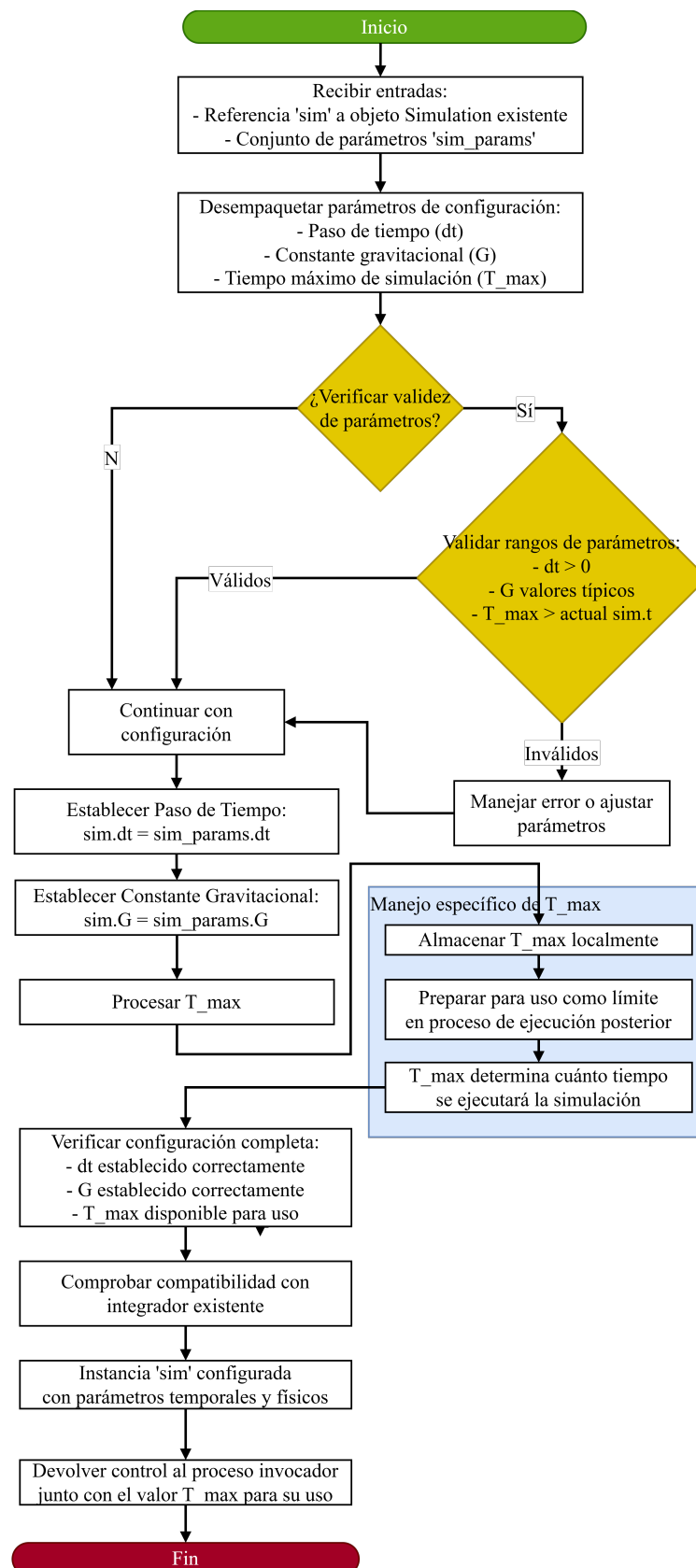


Figura 5.6: Diagrama de Proceso Interno 06: Definir Condiciones

5.1.7. Proceso Interno 07: Iniciar Simulación

Objetivo del Proceso

Ejecutar integración numérica en REBOUND para:

- Generar trayectoria completa del sistema
- Almacenar estados temporales
- Activar recolección de datos en $t_{\text{vis}} \in t_{\text{vis_list}}$
- Preparar datos para cálculo de LE

Entradas Principales

- **sim:** Objeto `Simulation` configurado
- $T_{\text{max}} \in \mathbb{R}^+$ (tiempo máximo)
- $t_{\text{vis_list}} \subset [0, T_{\text{max}}]$ (instantes de visualización)

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 75.7

1. Inicializar Almacenamiento

- Crear estructuras:
 - `time_steps = []`
 - `position_history = []`
 - `velocity_history = []`
- Capturar estado inicial en $t = 0$

2. Bucle Principal de Simulación

Paso 1: Condición: `sim.t < Tmax`

Paso 2: Integración:

`sim.integrate(sim.t + sim.dt)`

Paso 3: Almacenamiento:

- Actualizar `time_steps`
- Registrar posiciones/velocidades

Paso 4: Verificación Visualización:

$$\exists t_{\text{vis}} \in t_{\text{vis_list}} \quad | \quad |\text{sim.t} - t_{\text{vis}}| < \epsilon$$

Paso 5: Recolección Condicional:

- Si coincide: Activar módulo de visualización
- Si no: Continuar integración

Lógica Interna y Decisiones

- **Control del bucle:** Condición $\text{sim.t} < T_{\max}$
- **Tolerancia visualización:** $\epsilon = dt/2$
- **Manejo de errores:** Excepciones REBOUND
- **Almacenamiento:** Registro completo vs. muestreo selectivo

Manejo de Datos Específico

- **Entradas:**
 - Estado inicial de `sim`
 - Parámetros temporales
- **Intermedios:**
 - Estructuras de almacenamiento dinámicas
 - Estados intermedios del sistema
- **Salida:**
 - `SimulationResult`: $\{\text{time_steps}, \text{position_history}, \text{velocity_history}\}$

Salidas Principales

- **SimulationResult:** Estructura con:
 - Serie temporal completa
 - Datos de posición/velocidad
 - Metadatos de ejecución

Interacciones Internas

- **Con API REBOUND:**
 - Llamadas a `sim.integrate7()`
 - Acceso a `sim.particles`
- **Con módulo de visualización:**
 - Activación en t_{vis}
 - Transferencia de datos parciales
- **Con sistema de almacenamiento:**
 - Gestión eficiente de grandes datasets

Diagrama del Proceso

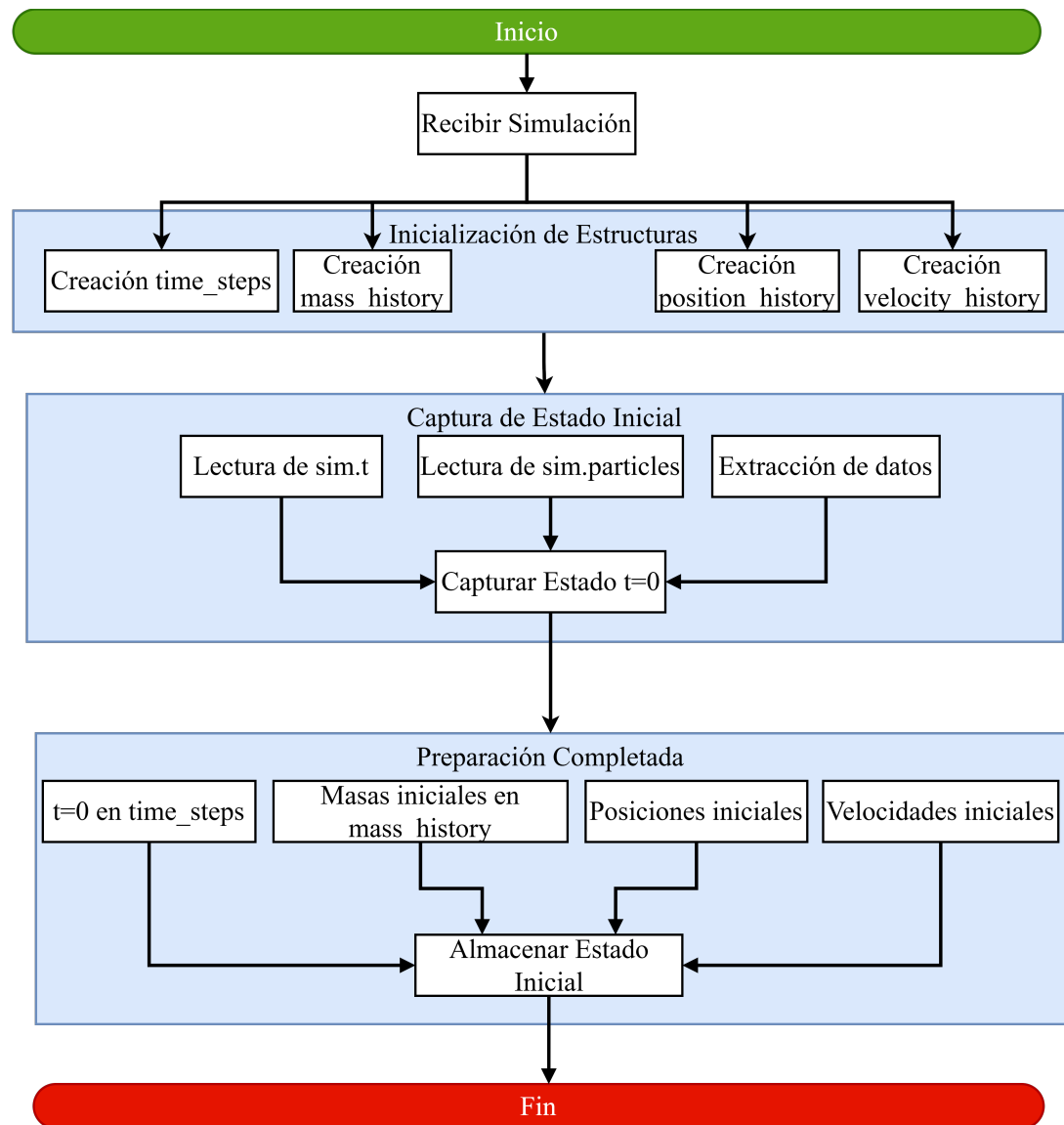


Figura 5.7: Diagrama de Proceso Interno 07: Iniciar Simulación

5.1.8. Proceso Interno 08: Avanzar un Paso de Simulación Usando al Integrador WHFast (Resolver Ecuaciones)

Objetivo del Proceso

Actualizar el estado del sistema gravitacional usando el esquema Drift-Kick-Drift (DKD) de WHFast:

$$H = H_{\text{Kepler}} + H_{\text{Interaction}}$$

Garantizando precisión en el cálculo de posiciones y velocidades para la evaluación del LE.

Entradas Principales

- **sim:** Objeto Simulation con:
 - $\mathbf{r}_i(t) = [x_i, y_i, z_i]$
 - $\mathbf{v}_i(t) = [v_{x_i}, v_{y_i}, v_{z_i}]$
 - m_i, dt , sistema coordenadas
- $t_{\text{target}} = t + dt$

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 85.8

1. Preparación de Coordenadas

- Transformación a coordenadas Jacobi si es necesario:

$$\mathbf{r}_{\text{Jacobi}} = T(\mathbf{r}_{\text{inercial}})$$

- Verificación modo seguro

2. Primer Drift ($dt/2$)

- Integración Kepleriana:

$$E - e \sin E = M \quad (\text{Ecuación de Kepler})$$

- Solucionador mejorado (Newton/Laguerre-Conway)
- Actualización posiciones: $\mathbf{r}_i(t + dt/2)$

3. Transformación para Interacción

- Conversión a coordenadas baricéntricas si es necesario

4. Kick (dt)

- Cálculo de aceleraciones:

$$\mathbf{a}_i = G \sum_{j \neq i} \frac{m_j (\mathbf{r}_j - \mathbf{r}_i)}{|\mathbf{r}_j - \mathbf{r}_i|^3}$$

- Actualización velocidades:

$$\mathbf{v}_i(t + dt) = \mathbf{v}_i(t) + \mathbf{a}_i \cdot dt$$

- Kernel modificado (si aplica)

5. Segundo Drift ($dt/2$)

- Integración Kepleriana final
- Posiciones finales: $\mathbf{r}_i(t + dt)$

6. Sincronización Final

- Transformación a coordenadas de reporte
- Actualización tiempo: $\text{sim.t} = t_{\text{target}}$
- Verificaciones modo seguro

Lógica Interna y Decisiones

- **Transformaciones coordenadas:**
 - Optimizadas para precisión numérica
 - Jacobi $\leftrightarrow$ Baricéntricas
- **Modo seguro:** Verificaciones de estabilidad
- **Kernel:** Selección entre precisión/rendimiento

Manejo de Datos Específico

- **Entradas:** Estado del sistema en t
- **Intermedios:**
 - Estados parciales DKD
 - Coordenadas transformadas
- **Salida:** Estado del sistema en $t + dt$

Salidas Principales

- **sim** actualizado con:
 - $\mathbf{r}_i(t + dt)$
 - $\mathbf{v}_i(t + dt)$
 - $t = t + dt$

Interacciones Internas

- **API REBOUND:**
 - Módulo WHFast y solucionador Kepler
 - Transformaciones coordenadas
- **Gestor de memoria:** Almacenamiento estados intermedios
- **Sistema fíísico:** Conservación propiedades dinámicas

Diagrama del Proceso

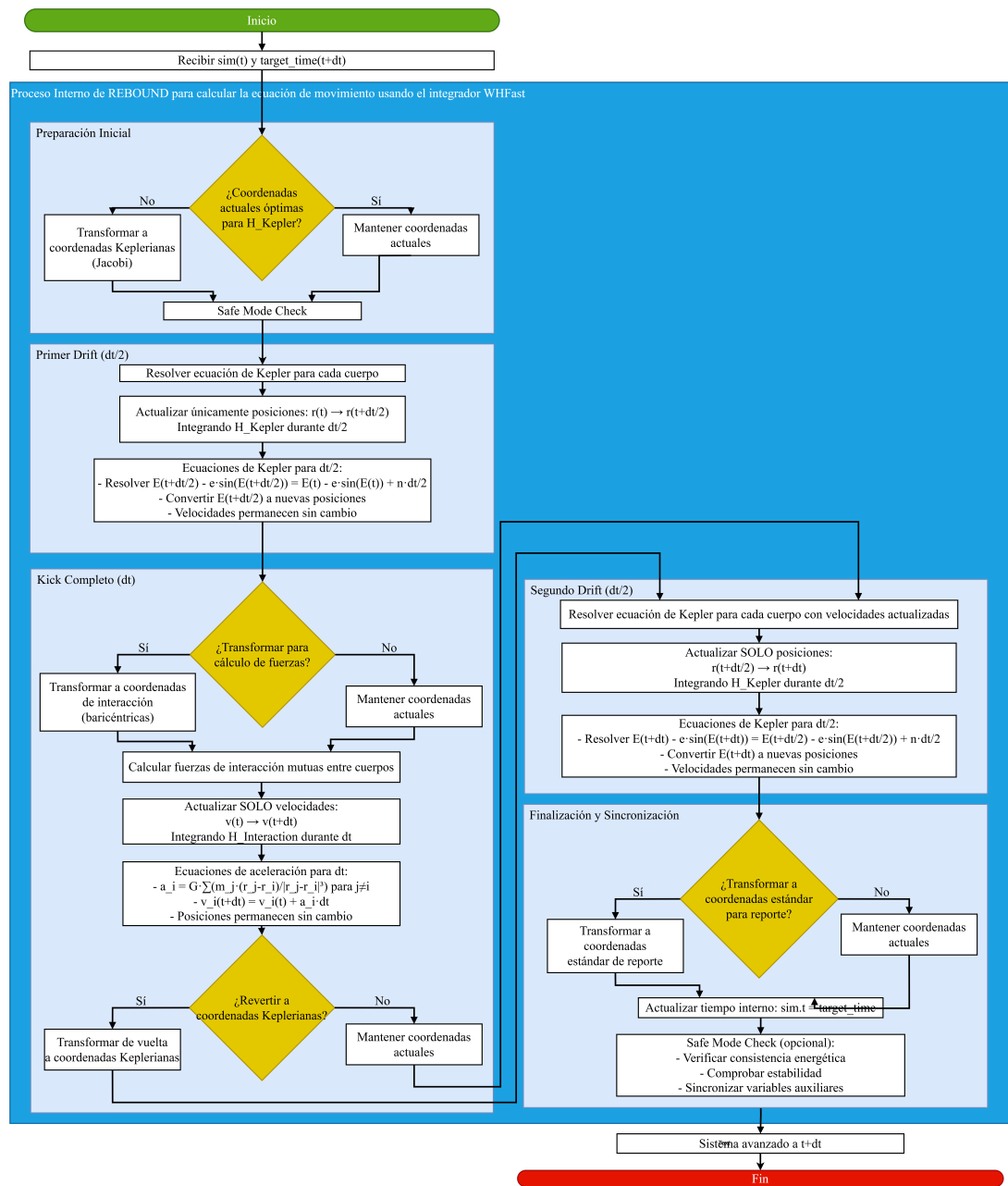


Figura 5.8: Diagrama de Proceso Interno 08: Resolver Ecuaciones

5.1.9. Proceso Interno 09: Actualizar Estado de las Partículas

Objetivo del Proceso

El propósito principal de la actividad “Actualizar Estado de las Partículas” es consolidar y reflejar oficialmente dentro del objeto **Simulation** de REBOUND (**sim**) el nuevo estado físico del sistema gravitacional, incluyendo las posiciones, velocidades, y tiempo de todas las partículas, tras la ejecución de un paso de integración numérica (dt) realizado por el integrador seleccionado (por ejemplo, WHFast). Esta actividad asegura que el objeto **sim** represente de manera coherente el estado del sistema en el nuevo instante de tiempo, preparándolo para las siguientes operaciones en el bucle de simulación, como almacenamiento o visualización.

Entradas Principales

- **Referencia al Objeto Simulation (sim):**
 - Un puntero o referencia a la instancia del objeto **Simulation** de REBOUND, recibida inmediatamente después de que la función de integración (por ejemplo, `sim.integrate9(t_nuevo)`) haya concluido.
 - Esta instancia contiene implícitamente:
 - Los nuevos valores de posición ($\mathbf{r}_{\text{nuevo}} = [x, y, z]$) y velocidad ($\mathbf{v}_{\text{nuevo}} = [v_x, v_y, v_z]$) de cada partícula, calculados por el integrador y almacenados en `sim.particles`.
 - El nuevo tiempo de simulación (`sim.t = t_nuevo = t_anterior + dt`), actualizado por el integrador.
 - Las masas de las partículas (m), que permanecen sin cambios en el contexto estándar del proyecto.

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 95.9

1. Finalización del Paso de Integración

- La actividad comienza justo después de que la función de integración de REBOUND (por ejemplo, `sim.integrate9(t_nuevo)`) retorna, habiendo completado el cálculo del nuevo estado del sistema para el tiempo t_{nuevo} .

2. Validar Tiempo de Simulación

- Se verifica que el tiempo interno de la simulación (`sim.t`) sea igual al tiempo objetivo ($t_{\text{nuevo}} = t_{\text{anterior}} + dt$), asegurando que el integrador ha avanzado correctamente el sistema al instante esperado.
- Esta validación confirma la sincronización temporal del estado del sistema.

3. Verificar Actualización del Estado Físico de las Partículas

- **Posiciones:**
 - Se confirma que las posiciones de todas las partículas en `sim.particles` han sido actualizadas por el integrador a los nuevos valores ($\mathbf{r}_{\text{nuevo}} = [x, y, z]$), correspondientes al tiempo t_{nuevo} .
- **Velocidades:**
 - Se verifica que las velocidades de todas las partículas en `sim.particles` han sido actualizadas a los nuevos valores ($\mathbf{v}_{\text{nuevo}} = [v_x, v_y, v_z]$), calculados por el integrador.
- **Masas:**
 - En el contexto de este proyecto, donde no se implementa la modificación dinámica de masas durante la integración estándar, se confirma que las masas de las partículas (m) en `sim.particles` permanecen sin cambios respecto al paso anterior.
- Estas verificaciones aseguran que el estado físico del sistema, almacenado en `sim.particles`, refleja correctamente los resultados del paso de integración.

4. Validación de Coherencia (Opcional)

- **Conservación de Energía:**
 - Opcionalmente, se calcula la energía total del sistema (cinética más potencial) en el nuevo estado y se compara con el estado anterior para verificar una conservación aproximada, dentro de los límites de precisión del integrador (por ejemplo, WHFast).
- **Conservación de Momento Angular:**
 - Opcionalmente, se calcula el momento angular total y se verifica su conservación aproximada, asegurando que el integrador no ha introducido errores significativos.
- **Coherencia General:**
 - Se realiza una comprobación general para asegurar que el estado físico es consistente (por ejemplo, posiciones y velocidades son valores numéricos válidos, no NaN ni infinitos).
- Estas validaciones son opcionales y pueden omitirse en implementaciones estándar si se confía en la robustez del integrador.

5. Actualización del Estado Computacional

- **Variables Auxiliares (Si Existen):**
 - Se actualizan cualquier variable auxiliar interna de `sim` que pueda ser utilizada por REBOUND (por ejemplo, acumuladores para métricas de error o estados intermedios del integrador).

- **Sincronización de Coordenadas (Si Aplica):**

- Si el integrador utiliza múltiples sistemas de coordenadas (por ejemplo, Jacobi para cálculos internos y baricéntricas para reporte), se sincronizan las representaciones para asegurar que el estado reportado en `sim.particles` esté en el sistema esperado por los procesos posteriores.
- Estas operaciones garantizan que el estado computacional de `sim` esté alineado con el estado físico.

6. Estado Coherente del Sistema

- En este punto, el objeto `sim` representa de manera consistente el estado del sistema en el tiempo t_{nuevo} , con `sim.t` actualizado y `sim.particles` conteniendo las nuevas posiciones, velocidades, y masas (sin cambios).

7. Listo para la Siguiete Acción

- La instancia `sim`, con su estado interno completamente actualizado, está preparada para las operaciones siguientes en el bucle principal de simulación, como almacenar el estado, verificar tiempos de visualización, o ejecutar otro paso de integración.

Lógica Interna y Decisiones

- **Validación del Tiempo:**

- La verificación de que `sim.t = tnuevo` es una decisión implícita que asegura la sincronización temporal. Si no se cumple, podría indicar un error en el integrador, pero esto es manejado por REBOUND internamente.

- **Validaciones Opcionales:**

- La decisión de realizar verificaciones de conservación (energía, momento angular) o coherencia general depende de los requisitos de robustez del sistema. Si se implementan, introducen bifurcaciones donde un fallo (por ejemplo, energía no conservada) podría desencadenar un manejo de errores, como lanzar una excepción.

- **Sincronización de Coordenadas:**

- La necesidad de sincronizar sistemas de coordenadas depende de la configuración del integrador y los requisitos de los procesos posteriores. Esta decisión es condicional y se basa en el estado interno de `sim`.

- **Ausencia de Transformación Directa:**

- No hay transformación de datos en esta actividad, ya que el integrador ya ha actualizado el estado en `sim`. La lógica se centra en verificar y consolidar ese estado, con decisiones orientadas a la validación y sincronización.

Manejo de Datos Específico

- **Datos de Entrada:**
 - Referencia al objeto `Simulation` (`sim`) post-integración, que contiene implícitamente:
 - Nuevo tiempo (`sim.t = tnuevo`).
 - Nuevas posiciones y velocidades en `sim.particles`.
 - Masas sin cambios en `sim.particles`.
- **Datos Intermedios:**
 - Resultados de las verificaciones (por ejemplo, valores de energía o momento angular, si se calculan).
 - Estado de variables auxiliares o sistemas de coordenadas sincronizados, si aplica.
- **Datos de Salida:**
 - La instancia `sim` con su estado interno consolidado, representando el sistema en `tnuevo`.

Salidas Principales

- **Instancia `sim` Actualizada:**
 - La referencia al objeto `Simulation` (`sim`), con su estado interno (tiempo `sim.t`, posiciones y velocidades en `sim.particles`) consolidado y coherente para el tiempo `tnuevo`, lista para las siguientes acciones en el bucle de simulación.

Interacciones Internas

- **Con la API de REBOUND:**
 - Depende directamente de la función de integración (por ejemplo, `sim.integrate9()`), que modifica el estado interno de `sim` (`sim.t`, `sim.particles`) antes de que esta actividad comience.
 - Accede a las propiedades de `sim` (como `sim.t` y `sim.particles`) para verificar el estado actualizado.
- **Con el Estado Interno de `sim`:**
 - Inspecciona y, si es necesario, sincroniza las estructuras internas de `sim`, incluyendo el tiempo, las partículas, y posibles variables auxiliares.
- **Con el Flujo de Simulación:**
 - Se integra en el bucle principal de simulación, actuando como el punto donde se consolida el estado tras cada paso de integración, preparando `sim` para procesos como almacenamiento o visualización.

Diagrama del Proceso

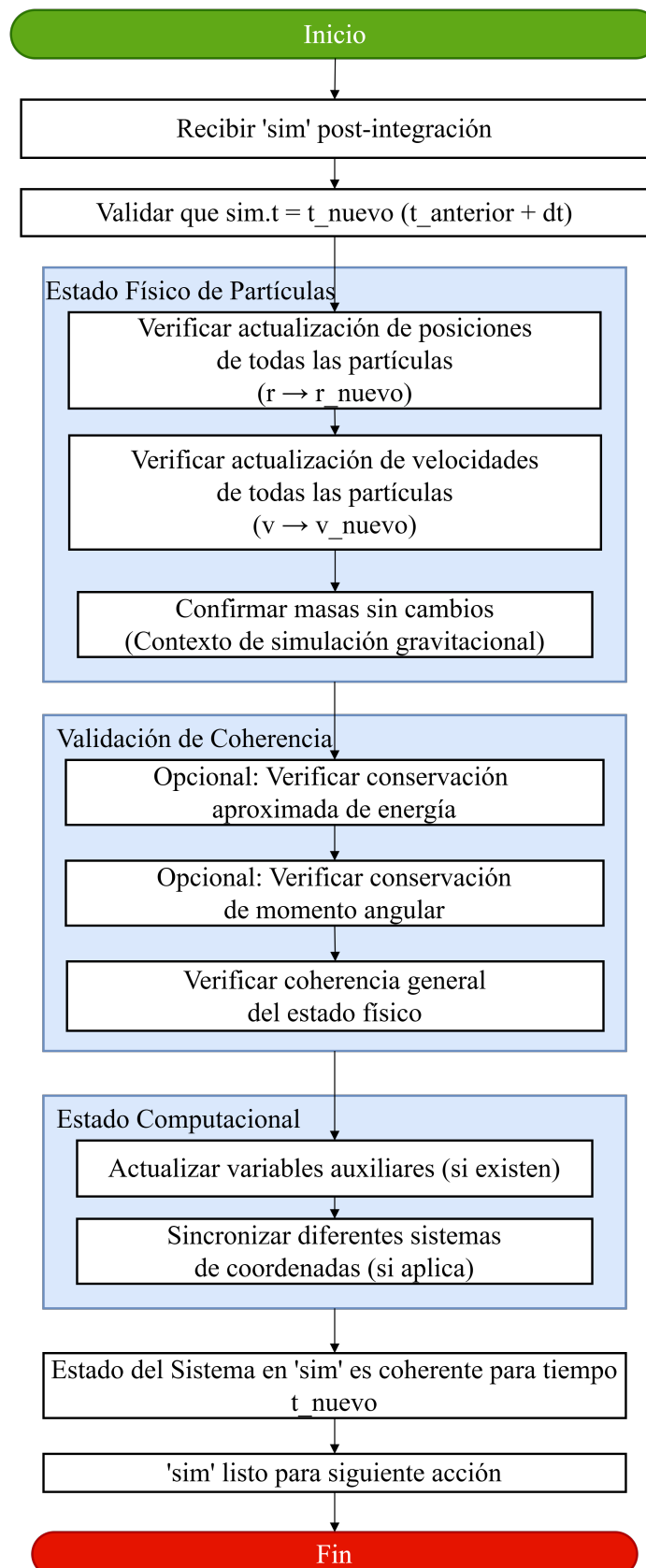


Figura 5.9: Diagrama de Proceso Interno 09: Actualizar Estado

5.1.10. Proceso Interno 10: Recolectar Datos para Visualización

Objetivo del Proceso

El propósito principal de la actividad “Recolectar Datos para Visualización” es extraer el estado f3sico actual del sistema gravitacional (tiempo, masas, posiciones y, opcionalmente, velocidades) desde la instancia de simulaci3n de REBOUND (`sim`) cuando se recibe una se1al indicando que el tiempo actual coincide con un instante designado para la visualizaci3n (t_{vis}). Los datos recolectados se organizan en una estructura coherente para ser procesados por el siguiente paso del m3dulo de visualizaci3n, facilitando la representaci3n gr1fica del sistema en el contexto de la simulaci3n de dos cuerpos.

Entradas Principales

- **Se1al de Activaci3n:** Un evento o llamada originada desde el bucle “Iniciar Simulaci3n”, que indica que el tiempo actual (`sim.t`) coincide con un instante de visualizaci3n (t_{vis}) de la lista `t_vis_list`.
- **Acceso al Estado Actual:** Una referencia a la instancia del objeto `Simulation` (`sim`) de REBOUND, que contiene el estado actual del sistema, incluyendo:
 - Tiempo actual (`sim.t`).
 - Masas (m), posiciones ($\mathbf{r} = [x, y, z]$), y velocidades ($\mathbf{v} = [v_x, v_y, v_z]$) de todas las part3culas en `sim.particles`.
- Alternativamente, una copia o extracto del estado actual (tiempo, masas, posiciones, velocidades) proporcionada directamente por el proceso llamador.

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura05.10

1. Recepci3n de Activaci3n

- La actividad es invocada por el bucle principal de simulaci3n cuando se detecta que `sim.t` est1 suficientemente cerca de un tiempo de visualizaci3n (t_{vis}).
- Se recibe la referencia al objeto `sim` o los datos del estado actual del sistema, que ser1n utilizados para la recolecci3n de datos.

2. Lectura del Estado B1sico

- **Leer Tiempo Actual:** Se obtiene el valor del tiempo actual t desde `sim.t`, que representa el instante exacto de la simulaci3n en el que se realiza la visualizaci3n.

- **Determinar Número de Partículas:** Se lee el número total de partículas en la simulación ($n = \text{sim.N}$), que indica cuántos cuerpos deben procesarse para extraer sus propiedades físicas.

3. Extracción de Datos por Partícula

- **Inicializar Arrays de Almacenamiento:** Se crean arrays o listas temporales para almacenar los datos recolectados:
 - `masas[]`: Para almacenar la masa de cada partícula.
 - `posiciones[] []`: Para almacenar los vectores de posición ($\mathbf{r} = [x, y, z]$) de cada partícula.
 - `velocidades[] []` (opcional): Para almacenar los vectores de velocidad ($\mathbf{v} = [v_x, v_y, v_z]$) si son requeridos por la visualización.
- **Bucle de Extracción:** Se itera sobre las n partículas (i de 0 a $n - 1$):
 - **Leer Masa:** Se extrae la masa de la partícula i (`masas[i] = sim.particles[i].m`).
 - **Leer Posición:** Se extrae el vector de posición de la partícula i (`posiciones[i] = [sim.particles[i].x, sim.particles[i].y, sim.particles[i].z]`).
 - **Verificar Requerimiento de Velocidades:** Se evalúa si la visualización requiere velocidades (por ejemplo, para dibujar vectores de velocidad en la interfaz gráfica).
 - Si se requieren: Se extrae el vector de velocidad de la partícula i (`velocidades[i] = [sim.particles[i].vx, sim.particles[i].vy, sim.particles[i].vz]`).
 - Si no se requieren: Se omite la lectura de velocidades, dejando velocidades vacía o sin inicializar.
- El bucle continúa hasta procesar todas las partículas ($i < n$).

4. Empaquetado de Datos

- **Crear Estructura VisualizationState:** Se inicializa una estructura de datos temporal (por ejemplo, un diccionario o un objeto `VisualizationState`) para agrupar los datos recolectados.
- **Añadir Tiempo:** Se asigna el tiempo actual t a la estructura (`VisState.t = t`).
- **Añadir Masas:** Se asigna el array de masas a la estructura (`VisState.masas = masas[]`).
- **Añadir Posiciones:** Se asigna el array de posiciones a la estructura (`VisState.posiciones = posiciones[] []`).
- **Verificar Velocidades:** Se evalúa si se recolectaron velocidades.

- Si se recolectaron: Se asigna el array de velocidades a la estructura (`VisState.velocidades = velocidades[] []`).
- Si no se recolectaron: Se omite la inclusión de velocidades en la estructura.
- **Verificar Metadatos Adicionales:** Se evalúa si son necesarios metadatos adicionales para la visualización (por ejemplo, nombres de partículas, colores, tamaños u otros parámetros gráficos).
 - Si son necesarios: Se añaden los metadatos a la estructura (por ejemplo, `VisState.metadata = {nombres, colores, tamaños}`). Estos pueden provenir de configuraciones predefinidas o datos asociados a `sim`.
 - Si no son necesarios: Se omite la inclusión de metadatos.
- **Finalizar Empaquetado:** La estructura `VisualizationState` se completa, representando una instantánea coherente del estado del sistema en el tiempo t_{vis} .

5. Pasar Datos a Procesamiento

- La estructura `VisualizationState` se envía al siguiente paso del pipeline de visualización, el proceso “Procesar Datos”, que se encargará de transformar los datos para su renderización en la interfaz gráfica.

Lógica Interna y Decisiones

- **Requerimiento de Velocidades:** La decisión de recolectar velocidades introduce una bifurcación condicional. Si la visualización no requiere velocidades (por ejemplo, si solo se muestran posiciones de partículas), se omite su extracción, optimizando el uso de recursos.
- **Metadatos Adicionales:** La inclusión de metadatos depende de los requisitos específicos del módulo de visualización. Esta decisión permite flexibilidad para soportar diferentes estilos de visualización (por ejemplo, partículas con colores o tamaños personalizados).
- **Bucle de Extracción:** El bucle sobre las partículas es controlado por la condición $i < n$, asegurando que se procesen todas las partículas en la simulación. No hay bifurcaciones dentro del bucle más allá de la decisión sobre velocidades.
- **Manejo de Errores Implícito:** La lectura de datos desde `sim.particles` podría generar errores si el estado de `sim` es inválido (por ejemplo, valores NaN). Estos casos son manejados por REBOUND o por validaciones previas en el bucle de simulación, fuera del alcance de esta actividad.

Manejo de Datos Específico

- **Datos de Entrada:**
 - Señal de activación desde el bucle de simulación.

- Referencia a `sim` o extracto del estado actual (tiempo, masas, posiciones, velocidades).
- **Datos Intermedios:**
 - Arrays temporales para almacenar masas (`masas[]`), posiciones (`posiciones[][]`), y opcionalmente velocidades (`velocidades[][]`).
 - Valores individuales extraídos de `sim.particles` ($t, m, x, y, z, v_x, v_y, v_z$).
 - Metadatos adicionales, si se requieren.
- **Datos de Salida:**
 - Estructura `VisualizationState` (o equivalente), que contiene la instantánea del sistema: tiempo (t), masas, posiciones, velocidades (si aplica), y metadatos (si aplica).

Salidas Principales

- **Estructura `VisualizationState`:** Una estructura de datos que encapsula el estado relevante del sistema en el instante de visualización (t_{vis}), incluyendo el tiempo, las masas, las posiciones, y opcionalmente las velocidades y metadatos, lista para ser procesada por el módulo de visualización.

Interacciones Internas

- **Con el Bucle de Simulación:** La actividad es activada por el proceso “Iniciar Simulación” cuando `sim.t` coincide con un tiempo de visualización.
- **Con la API de REBOUND:** Accede al estado interno del objeto `Simulation` (`sim.t`, `sim.N`, `sim.particles`) para extraer los datos físicos del sistema.
- **Con Estructuras de Datos Temporales:** Crea y llena arrays temporales (`masas`, `posiciones`, `velocidades`) y una estructura `VisualizationState` para organizar los datos recolectados.
- **Con el Pipeline de Visualización:** Transfiere la estructura `VisualizationState` al proceso “Procesar Datos”, integrándose en el flujo de renderización gráfica.

Diagrama del Proceso

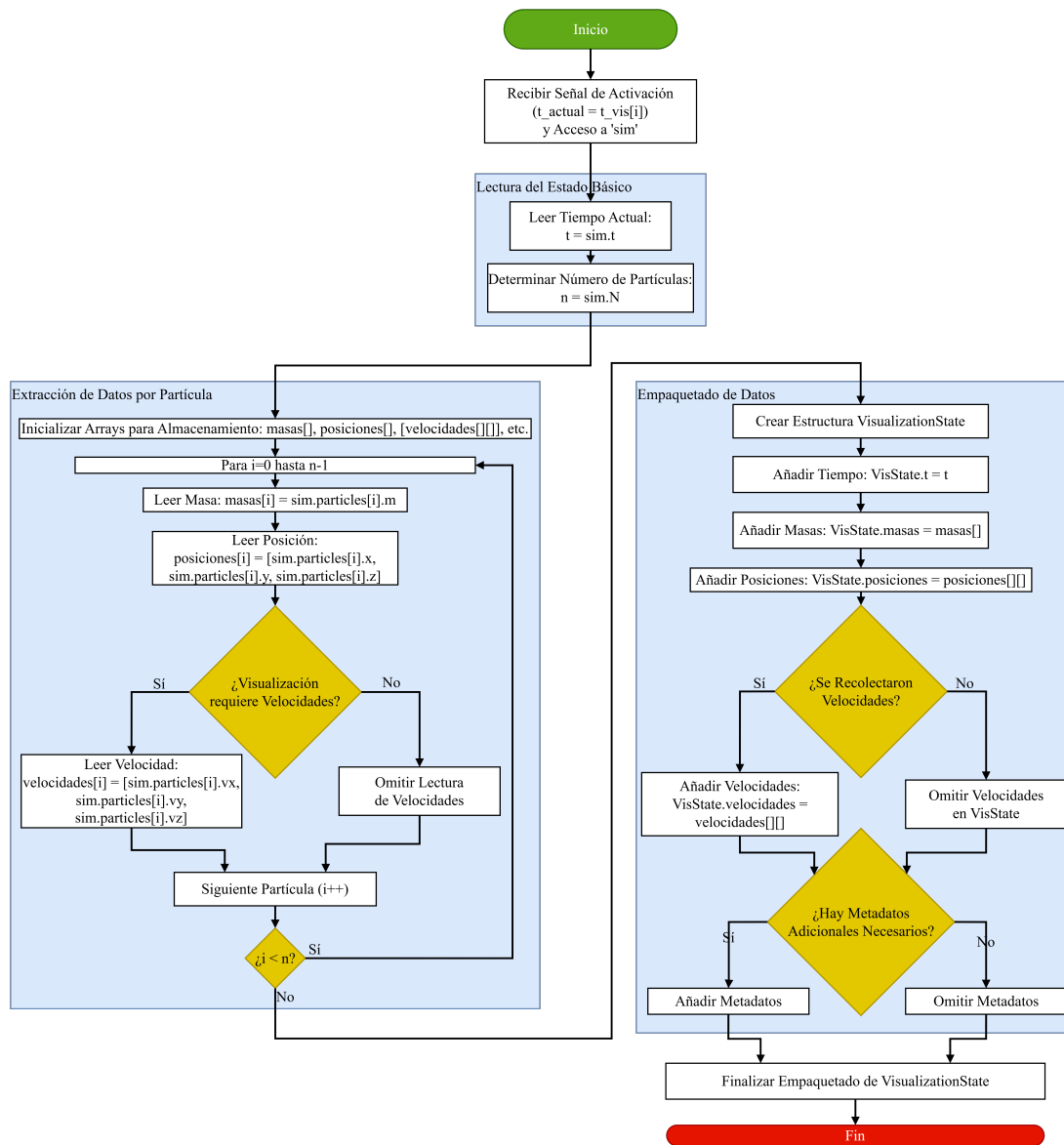


Figura 5.10: Diagrama de Proceso Interno 10: Recolectar Datos

5.1.11. Proceso Interno 11: Procesar Datos para Visualización

Objetivo del Proceso

El propósito principal de la actividad “Procesar Datos para Visualización” es transformar y formatear los datos brutos del estado del sistema gravitacional, recolectados en el proceso “Recolectar Datos”, para que sean compatibles y óptimos para la biblioteca o herramienta gráfica específica utilizada en la visualización. Esta actividad asegura que los datos estén en el formato adecuado y contengan solo la información relevante para generar gráficos como trayectorias 2D/3D o gráficos de energía, soportando la representación visual del comportamiento dinámico en el contexto de la simulación de dos cuerpos.

Entradas Principales

- **Estructura VisualizationState:** Una estructura de datos (por ejemplo, un diccionario o objeto) que contiene la instantánea del sistema en un instante de visualización, incluyendo:
 - Tiempo (t).
 - Masas (`masas []`).
 - Posiciones (`posiciones [] []`: $\mathbf{r} = [x, y, z]$ por partícula).
 - Velocidades (`velocidades [] []`: $\mathbf{v} = [v_x, v_y, v_z]$ por partícula, si se recolectaron).
 - Metadatos opcionales (por ejemplo, nombres, colores, tamaños).
- **Configuración de Visualización:** Información que define el tipo de gráfico a generar (por ejemplo, trayectoria 2D XY, trayectoria 3D, gráfico de energía vs tiempo) y parámetros de transformación, como:
 - Límites de la ventana gráfica (por ejemplo, $x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$).
 - Factores de escala (`scale_factor`) y traslación (`offset`).
 - Requerimientos de proyección (por ejemplo, de 3D a 2D).

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 1??

1. Recibir Datos Recolectados

- La actividad recibe la estructura **VisualizationState** con los datos brutos del estado del sistema en el instante de visualización.

2. Verificación de Configuración

- **Leer Configuración de Visualización:** Se accede a la configuración de visualización para determinar:

- El tipo de gráfico (por ejemplo, trayectoria 2D, trayectoria 3D, energía vs. tiempo).
- Parámetros de transformación (escala, traslación, proyección).
- Límites y escalas de la ventana gráfica.
- **Determinar Tipo de Gráfico:** Se identifica el tipo de gráfico especificado:
 - Trayectoria 2D/3D: Configura el procesamiento para extraer coordenadas espaciales.
 - Energía vs. Tiempo: Configura el procesamiento para incluir cálculos de energía.
 - Otro: Configura el procesamiento según los requerimientos específicos del gráfico.

3. Transformación de Coordenadas (Condicional)

- **Verificar Requerimiento de Transformación:** Se evalúa si las coordenadas físicas en `VisualizationState.posiciones` (potencialmente a gran escala) necesitan transformarse para adaptarse a la ventana gráfica o al formato de la biblioteca gráfica.
- **Si se requiere transformación:**
 - **Inicializar Arrays:** Se crean arrays para almacenar las coordenadas transformadas.
 - **Bucle por Partícula:** Para cada partícula:
 - **Escalado:** Se aplica un factor de escala para mapear las coordenadas físicas a la escala gráfica:

$$\mathbf{r}_{\text{scaled}} = (\mathbf{r}_{\text{original}} - \mathbf{r}_{\text{min}}) \times \text{scale_factor}$$

donde $\mathbf{r}_{\text{min}}$ y scale_factor provienen de la configuración.

- **Traslación:** Se aplica un desplazamiento para centrar las coordenadas en la ventana gráfica:

$$\mathbf{r}_{\text{transformed}} = \mathbf{r}_{\text{scaled}} + \text{offset}$$

- **Proyección (Si Aplica):** Se evalúa si es necesaria una proyección de 3D a 2D (por ejemplo, para gráficos 2D).
 - Si se requiere: Se aplica una matriz de proyección $\mathbf{P}$:

$$\mathbf{r}_{\text{2D}} = \mathbf{P} \times \mathbf{r}_{\text{transformed}}$$

donde $\mathbf{P}$ es la matriz de proyección definida por la configuración.

- Si no se requiere: Se mantienen las coordenadas transformadas sin proyección.
- El bucle continúa hasta procesar todas las partículas.

- **Si no se requiere transformación:** Se mantienen las coordenadas originales ($\mathbf{r}_{\text{transformed}} = \mathbf{r}_{\text{original}}$).

4. Selección de Datos Relevantes

- **Iniciar Selección:** Se identifican los datos necesarios según el tipo de gráfico especificado.
- **Según Tipo de Gráfico:**
 - Trayectoria 2D: Se extraen las componentes x e y de las posiciones transformadas, almacenándolas en arrays como $\mathbf{x_coords}[]$ y $\mathbf{y_coords}[]$.
 - Trayectoria 3D: Se extraen las componentes x , y , y z , almacenándolas en $\mathbf{x_coords}[]$, $\mathbf{y_coords}[]$, y $\mathbf{z_coords}[]$.
 - Energía vs. Tiempo: Se extraen el tiempo (t), masas (m), posiciones ($\mathbf{r}$), y velocidades ($\mathbf{v}$, si disponibles) para cálculos posteriores de energía.
 - Otro: Se extraen los componentes relevantes según la configuración específica del gráfico.
- Los datos seleccionados se almacenan para el siguiente paso.

5. Cálculos Derivados (Condicional)

- **Verificar Requerimiento de Cálculos:** Se evalúa si el tipo de gráfico requiere cálculos derivados (por ejemplo, energía o momento angular).
- **Si se requieren cálculos:**
 - **Energía:** Se calculan:
 - Energía cinética:

$$E_k = \sum_i (0.5 \times m_i \times |\mathbf{v}_i|^2)$$

- Energía potencial:

$$E_p = -G \sum_{i < j} \frac{m_i m_j}{|\mathbf{r}_i - \mathbf{r}_j|}$$

- Energía total:

$$E_{\text{total}} = E_k + E_p$$

- **Momento Angular:** Se calcula el momento angular total:

$$\mathbf{L} = \sum_i m_i (\mathbf{r}_i \times \mathbf{v}_i)$$

- **Otros Cálculos:** Se realizan cálculos específicos según la configuración (por ejemplo, distancias relativas o velocidades relativas).
- **Si no se requieren cálculos:** Se omiten los cálculos derivados, manteniendo solo los datos seleccionados.

6. Formateo para API Gráfica

- **Iniciar Formateo:** Se prepara la conversión de los datos seleccionados y derivados al formato requerido por la biblioteca gráfica (por ejemplo, Matplotlib, Pygame, Plotly).
- **Identificar Formato Requerido:** Se determina el formato esperado por la API:
 - Listas Separadas: Crear arrays individuales por eje (por ejemplo, `x_list = [x1, x2, ...]`, `y_list = [y1, y2, ...]`).
 - Lista de Tuplas: Crear una lista de puntos (por ejemplo, `points = [(x1, y1), (x2, y2), ...]`).
 - Arrays NumPy: Convertir los datos a arrays NumPy (por ejemplo, `X = np.array([x1, x2, ...])`, `Y = np.array([y1, y2, ...])`).
 - Diccionario/Objeto: Crear una estructura con metadatos (por ejemplo, `data = {'x': [x1, x2, ...], 'y': [y1, y2, ...], 'tipo': 'scatter'}`).
- Los datos se convierten al formato seleccionado, incluyendo cualquier valor derivado (como energías).

7. Finalizar Estructura de Datos

- Se completa la estructura de datos formateada, que contiene los datos transformados, seleccionados, y posiblemente derivados, en el formato exacto requerido por la biblioteca gráfica.

8. Pasar Datos Formateados

- La estructura de datos formateada se envía al proceso “Generar Gráficos” del pipeline de visualización, donde se utilizará para renderizar la representación gráfica.

Lógica Interna y Decisiones

- **Tipo de Gráfico:** La selección del tipo de gráfico (trayectoria 2D, 3D, energía, otro) determina las ramas de procesamiento, afectando la selección de datos y los cálculos derivados.
- **Transformación de Coordenadas:** La decisión de transformar coordenadas depende de la configuración de visualización. Si es necesaria, se aplican escalado, traslación, y proyección condicionalmente (por ejemplo, proyección 3D a 2D solo para gráficos 2D).
- **Cálculos Derivados:** La necesidad de cálculos como energía o momento angular introduce una bifurcación. Gráficos como trayectorias omiten estos cálculos, mientras que gráficos de energía los requieren.

- **Formato de la API:** La elección del formato (listas, tuplas, NumPy, diccionario) depende de la biblioteca gráfica, permitiendo flexibilidad para diferentes herramientas.
- **Manejo de Errores Implícito:** Errores como datos inválidos (por ejemplo, divisiones por cero en el cálculo de energía potencial) son manejados por validaciones previas o excepciones en la biblioteca gráfica, fuera del alcance directo de esta actividad.

Manejo de Datos Específico

- **Datos de Entrada:**
 - Estructura `VisualizationState` con tiempo, masas, posiciones, velocidades (si aplica), y metadatos.
 - Configuración de visualización (tipo de gráfico, parámetros de transformación).
- **Datos Intermedios:**
 - Coordenadas transformadas (escaladas, trasladadas, proyectadas: $\mathbf{r}_{\text{transformed}}$, $\mathbf{r}_{2D}$).
 - Subconjuntos de datos seleccionados (por ejemplo, `x_coords`, `y_coords`).
 - Valores derivados (por ejemplo, energía cinética E_k , potencial E_p , total E_{total} , momento angular $\mathbf{L}$).
- **Datos de Salida:**
 - Estructura de datos formateada (por ejemplo, `x_list`, `y_list`, arrays NumPy, o diccionario) específica para la API de la biblioteca gráfica.

Salidas Principales

- **Datos Formateados para Visualización:** Una estructura de datos que contiene los datos del estado del sistema, seleccionados, transformados, y formateados según los requisitos de la biblioteca gráfica, lista para ser utilizada directamente en el proceso “Generar Gráficos”.

Interacciones Internas

- **Con el Proceso Anterior:** Recibe la estructura `VisualizationState` del proceso “Recolectar Datos”.
- **Con la Configuración de Visualización:** Lee los parámetros de configuración para determinar el tipo de gráfico, transformaciones, y formato de salida.
- **Con Cálculos Matemáticos:** Realiza operaciones como escalado, traslación, proyección, y cálculos de energía o momento angular, según sea necesario.

- **Con el Pipeline de Visualización:** Transfiere los datos formateados al proceso “Generar Gráficos”, integrándose en el flujo de renderización gráfica.

Diagrama del Proceso

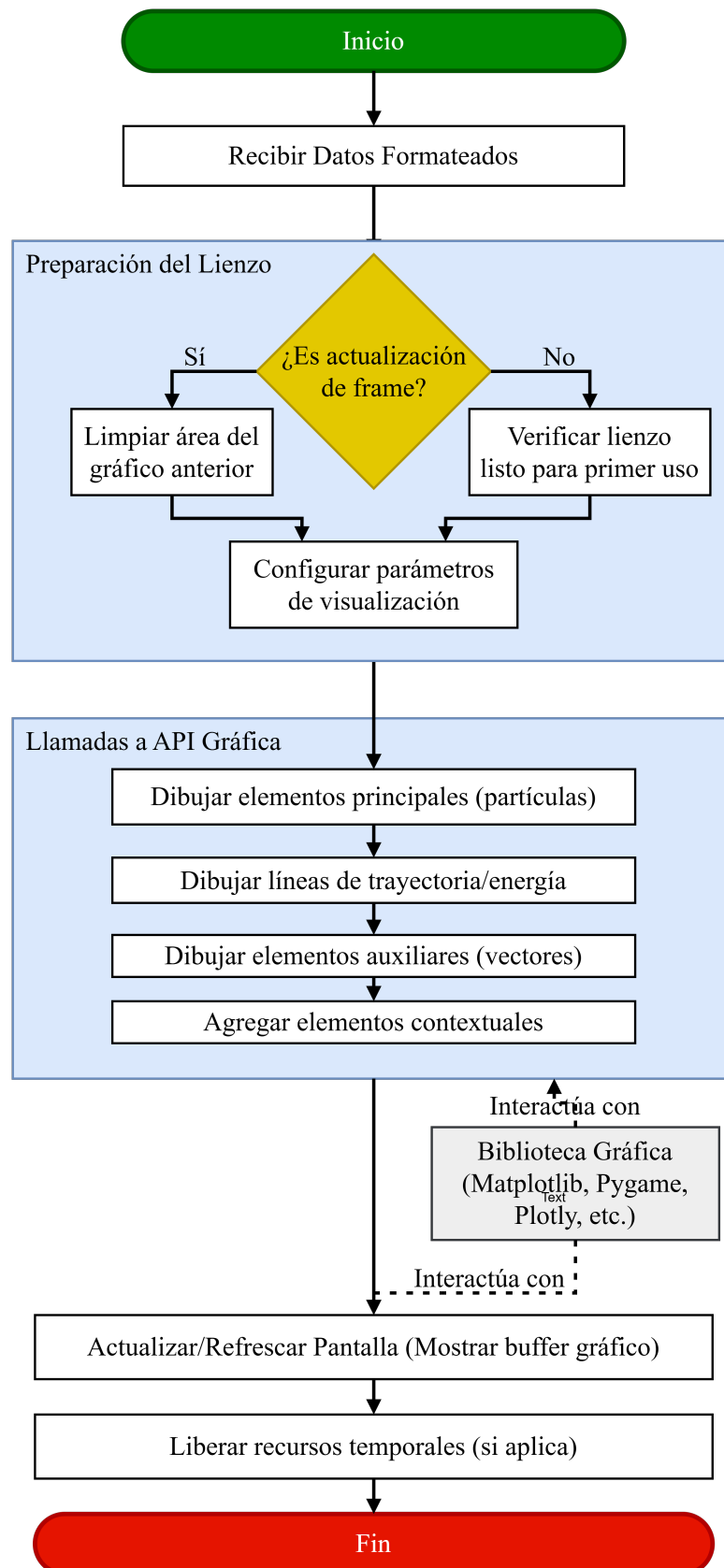


Figura 5.11: Diagrama de Proceso Interno 12: Generar Gráfico

5.1.12. Proceso Interno 12: Generar Gráficos

Objetivo del Proceso

El propósito principal de la actividad “Generar Gráficos” es utilizar los datos preprocesados y formateados del estado del sistema gravitacional para crear o actualizar una representación visual en la pantalla, empleando las funciones específicas de la biblioteca gráfica seleccionada (por ejemplo, `Matplotlib`, `Pygame`, `Plotly`). Esta actividad asegura que el estado del sistema, como las posiciones de las partículas o los valores de energía, se visualice de manera clara y precisa para el usuario, soportando el análisis y monitoreo de la simulación en el contexto del proyecto.

Entradas Principales

- **Datos Formateados:** Una estructura de datos generada por el proceso “Procesar Datos”, que contiene información lista para graficar, como:
 - Coordenadas para trayectorias (por ejemplo, `x_list`, `y_list` para 2D, o `x_list`, `y_list`, `z_list` para 3D).
 - Valores escalares para gráficos de energía (por ejemplo, `[t, E_total]`).
 - Configuración visual adicional (por ejemplo, colores, tamaños, tipos de gráfico).
- **Contexto Gráfico:** Acceso al lienzo, ventana, o superficie de dibujo proporcionada por la biblioteca gráfica, que sirve como el área donde se renderizará la visualización.

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 25.11

Recibir Datos Formateados

- La actividad recibe la estructura de datos formateada, que contiene las coordenadas, valores derivados (como energía), y parámetros visuales necesarios para generar el gráfico.

Preparar Lienzo

- **Verificar si es Actualización de Frame:**
 - Se evalúa si el gráfico es una actualización de un frame anterior (como en una animación) o el primer dibujo en el lienzo.
- **Si es una actualización:**
 - Se limpia el área del gráfico anterior para evitar superposiciones. Esto puede implicar:
 - Rellenar el fondo con un color sólido (por ejemplo, blanco o negro).

- Eliminar objetos gráficos previos (por ejemplo, puntos o líneas) del buffer.
- **Si es la primera vez o no es animación:**
 - Se verifica que el lienzo esté inicializado y listo para dibujar, sin necesidad de limpieza.
- Esta preparación asegura que el lienzo esté en un estado adecuado para el nuevo dibujo.

Configurar Parámetros de Visualización

- Se establecen los parámetros gráficos definidos en la configuración, como el estilo de los puntos, colores, tamaños, o propiedades del gráfico (por ejemplo, escala de ejes, títulos).
- Estos parámetros se pasan a las funciones de dibujo de la biblioteca gráfica.

Llamar Funciones de Dibujo de la API Gráfica

- **Dibujar Elementos Principales (Partículas):**
 - Se utilizan los datos formateados para dibujar los elementos primarios, como las partículas. Por ejemplo:
 - Para una trayectoria 2D, se llama a funciones como `draw_circle2()` (Pygame) o `scatter2()` (Matplotlib) para cada partícula, usando coordenadas `x_list` y `y_list`.
 - El tamaño o color de los puntos puede reflejar la masa de las partículas, según la configuración.
- **Dibujar Líneas de Trayectoria o Energía:**
 - Se dibujan líneas o curvas que conectan puntos para representar trayectorias o series temporales. Por ejemplo:
 - Para trayectorias, se usa `plot2()` o `draw_line2()` para conectar posiciones consecutivas.
 - Para gráficos de energía, se usa `plot_line2()` para añadir un punto (`[t, E_total]`) a la serie temporal.
- **Dibujar Elementos Auxiliares (Vectores):**
 - Si los datos incluyen velocidades y la configuración lo requiere, se dibujan vectores de velocidad usando funciones como `draw_line2()` o `arrow2()` para cada partícula.
- **Agregar Elementos Contextuales:**
 - Se añaden elementos gráficos como ejes, etiquetas, títulos, y leyendas, utilizando funciones como `set_xlabel2()`, `set_title2()`, o `add_legend2()` en la API gráfica.

- Cada llamada a la API utiliza los datos formateados como argumentos, asegurando que la representación visual sea precisa.

Actualizar/Refrescar Pantalla

- Una vez que todos los elementos gráficos han sido dibujados en el buffer gráfico, se llama a la función de la biblioteca gráfica que hace visible el buffer en la pantalla. Por ejemplo:
 - `pygame.display.flip2()` en Pygame.
 - `plt.draw2()` o `fig.show2()` en Matplotlib.
 - `plotly.render2()` en Plotly.
- Esto asegura que el usuario vea la representación gráfica actualizada.

Liberar Recursos Temporales (Si Aplica)

- Se liberan recursos temporales utilizados durante el dibujo, como buffers intermedios o estructuras de datos auxiliares, para optimizar el uso de memoria.
- En la mayoría de los casos, las bibliotecas gráficas manejan esta limpieza automáticamente.

Lógica Interna y Decisiones

- **Actualización vs Primer Dibujo:** La decisión de limpiar el lienzo depende de si el gráfico es una actualización en una animación o un dibujo inicial. Esta bifurcación asegura que las animaciones no acumulen artefactos visuales de frames anteriores.
- **Tipo de Elementos Gráficos:** La selección de funciones de dibujo (por ejemplo, `scatter` para partículas, `plot` para líneas) depende implícitamente del tipo de gráfico especificado en los datos formateados (trayectoria, energía, etc.).
- **Elementos Auxiliares:** La inclusión de vectores de velocidad, etiquetas, o leyendas es condicional, basada en la configuración visual y la disponibilidad de datos (por ejemplo, velocidades).
- **Manejo de Errores Implícito:** Errores como formatos de datos incompatibles con la API gráfica o fallos en el lienzo son manejados por la biblioteca gráfica, que puede lanzar excepciones. Esta actividad asume que los datos formateados son válidos, gracias al procesamiento previo.

Manejo de Datos Específico

- **Datos de Entrada:**
 - Estructura de datos formateada (por ejemplo, `x_list`, `y_list`, `[t, E_total]`) con coordenadas, valores escalares, y parámetros visuales.

- Contexto gráfico (lienzo o ventana de la biblioteca gráfica).
- **Datos Intermedios:**
 - Ningún dato numérico nuevo se genera; los datos formateados se consumen directamente como parámetros para las funciones de dibujo.
- **Datos de Salida:**
 - No se generan datos numéricos como salida; el resultado es la modificación del buffer gráfico, que produce una representación visual en la pantalla.

Salidas Principales

- **Representación Gráfica Actualizada:** La visualización del estado del sistema (por ejemplo, posiciones de partículas, trayectorias, o gráficos de energía) renderizada y visible en la interfaz gráfica, proporcionando al usuario una representación clara del comportamiento dinámico.

Interacciones Internas

- **Con el Proceso Anterior:** Recibe los datos formateados del proceso “Procesar Datos”.
- **Con la Biblioteca Gráfica:** Interactúa extensamente con la API de la biblioteca gráfica (por ejemplo, `Matplotlib`, `Pygame`, `Plotly`) para dibujar elementos y actualizar la pantalla.
- **Con el Lienzo Gráfico:** Modifica el estado del lienzo o ventana gráfica, ya sea limpiándolo, dibujando nuevos elementos, o refrescando el buffer para mostrar la visualización.
- **Con el Pipeline de Visualización:** Completa el flujo de visualización, transformando los datos formateados en una salida gráfica que el usuario puede observar.

Diagrama del Proceso

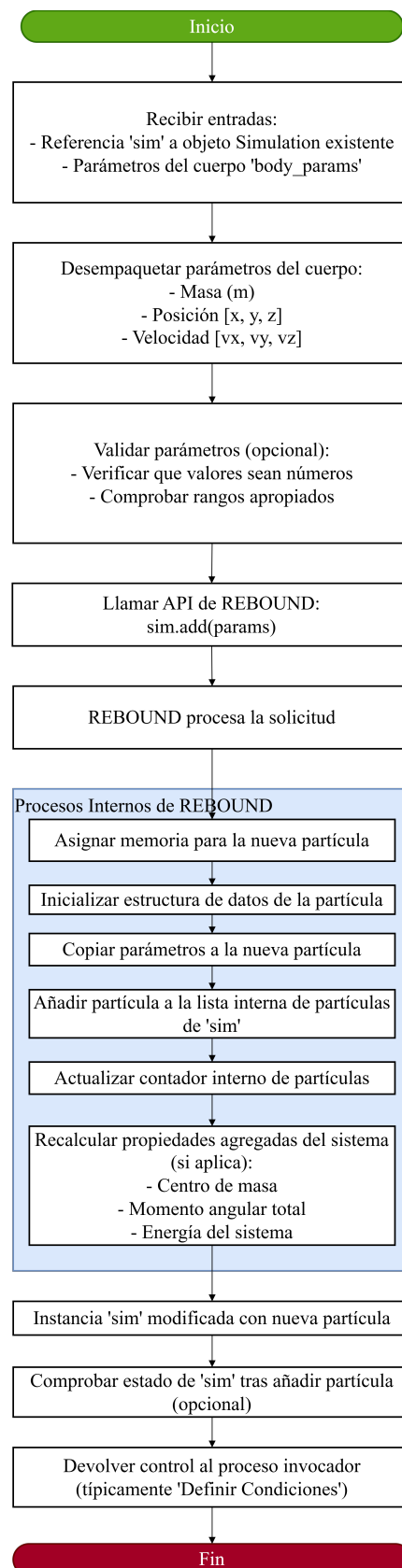


Figura 5.12: Diagrama de Proceso Interno 05: Agregar Cuerpos

5.1.13. Proceso Interno 13: Evaluar Fitness

Objetivo del Proceso

El propósito principal es calcular el *fitness penalizado* $F_p(x) = f(x) + \lambda_p P(x)$, donde:

- $f(x)$: Exponente de Lyapunov (LE)
- $P(x) = \sum \text{máx}(g_k(x), 0)^2 + \sum |h_k(x)|^2$
- λ_p : Parámetro de penalización

Entradas Principales

- **Parámetros x** : Vector en $\mathbb{R}^n$ (ej. $[m_1, m_2]$)
- **Datos REBOUND**: $\{\text{pos}(t), \text{vel}(t), \text{mass}(t)\}_{t=0}^{T_{\text{máx}}}$
- **Restricciones**:
 - $g_k(x) \leq 0$ (desigualdad)
 - $h_k(x) = 0$ (igualdad)
- $\lambda_p \in \mathbb{R}^+$

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 35.13

1. Recibir Resultados de Simulación

- Validar integridad de datos y $T_{\text{máx}}$

2. Calcular $f(x)$

- Cálculo de LE :

$$f(x) = \begin{cases} LE_{\text{calc}} & \text{si converge} \\ \text{INF} & \text{si falla} \end{cases}$$

3. Calcular $P(x)$

$$P(x) = \sum_{k=1}^K \text{máx}(g_k(x), 0)^2 + \sum_{j=1}^J |h_j(x)|^2 + \begin{cases} \text{PENALTY_LARGE} & \text{si simulación inviable} \end{cases}$$

4. Calcular $F_p(x)$

$$F_p(x) = f(x) + \lambda_p \cdot P(x)$$

- Verificar $F_p \in \mathbb{R}$

Lógica Interna y Decisiones

- **Convergencia LE:**
 - $\text{Fallo} \Rightarrow f(x) \leftarrow \text{INF}$
- **Violaciones:**
 - g_k : $\max(g_k(x), 0) \Rightarrow$ solo violaciones positivas
 - h_k : L^2 -norm del desvío
- **Penalización catastrófica:**
 - Activa para simulaciones físicamente inviables

Manejo de Datos Específico

- **Entradas:**
 - $x \in [\min_i, \max_i]^n$
 - $\lambda_p > 0$
- **Intermedios:**
 - $LE_{\text{calc}} \in \mathbb{R}$
 - violaciones $\in \mathbb{R}^+$
- **Salida:**
 - $F_p(x) \in \mathbb{R}$

Salidas Principales

- **Fitness penalizado:** $F_p(x)$ ordenable para selección

Interacciones Internas

- **REBOUND:** Provee $\{\text{pos}(t), \text{vel}(t)\}$
- **Módulo LE:** Calcula $f(x)$
- **Gestor de restricciones:** Evalúa $g_k(x)$, $h_j(x)$

Diagrama del Proceso

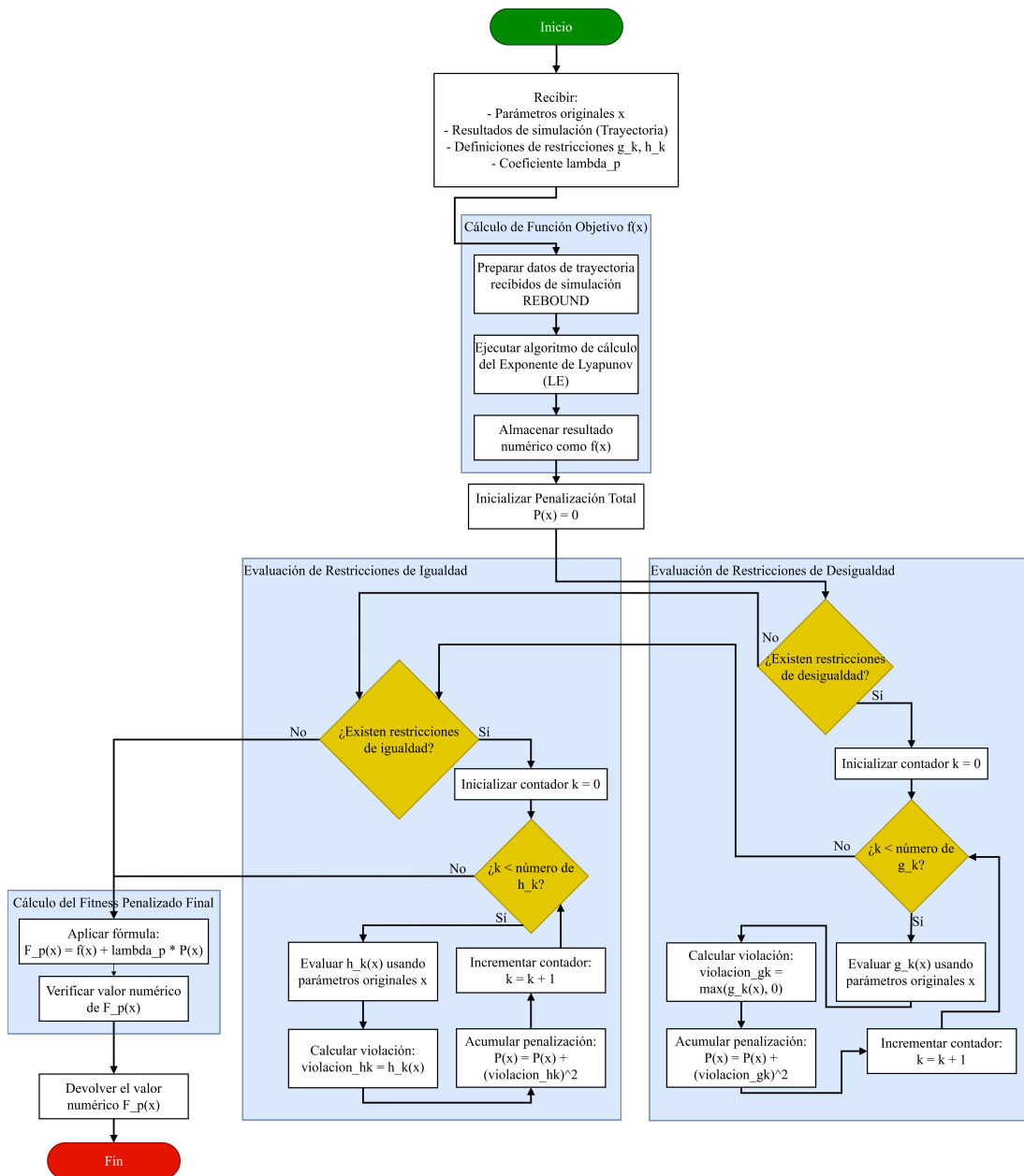


Figura 5.13: Diagrama de Proceso Interno 13: Calcular Fitness

5.1.14. Proceso Interno 14: Formar Siguiete Población

Objetivo del Proceso

El propósito principal es construir la población para la próxima iteración del algoritmo genético, combinando:

- Los k mejores individuos (élite) de `Poblacion.Actual`
- Los $(N - k)$ mejores descendientes

Manteniendo el tamaño poblacional N .

Entradas Principales

- **Población Actual:**
 - N individuos $\{x_i, F_p^i\}_{i=1}^N$
- **Descendencia:**
 - N individuos nuevos $\{x'_j, F_p^j\}_{j=1}^N$
- **Parámetros:**
 - $k \in \mathbb{N}$ (tamaño de élite)
 - $N \in \mathbb{N}$ (tamaño población)

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 45.14

1. Ordenar Poblaciones

- `Poblacion.Actual` $\leftarrow$ `sort_asc`($\{F_p^i\}$)
- `Descendencia` $\leftarrow$ `sort_asc`($\{F_p^j\}$)

2. Inicializar Población Siguiete

- `Crear Poblacion.Siguiete` $= \emptyset$

3. Aplicar Elitismo

$$\text{Poblacion.Siguiete} \leftarrow \begin{cases} \text{primeros } k \text{ de Poblacion.Actual} & \text{si } k > 0 \\ \emptyset & \text{si } k = 0 \end{cases}$$

4. Completar con Descendencia

$$\begin{aligned} \text{num.a.copiar} &= N - |\text{Poblacion.Siguiete}| \\ \text{Poblacion.Siguiete} &\leftarrow \text{Poblacion.Siguiete} \cup \{\text{primeros num.a.copiar de Descendencia}\} \end{aligned}$$

5. Verificación Final

- Asegurar $|\text{Poblacion_Siguiete}| = N$
- Manejo de errores (relleno adicional si es necesario)

Lógica Interna y Decisiones

- **Elitismo condicional:**
 - $k > 0 \Rightarrow$ preservar élite
 - $k = 0 \Rightarrow$ reemplazo completo
- **Selección por fitness:**
 - Ordenamiento basado en F_p ascendente
 - $\text{num_a_copiar} = N - k$ garantiza tamaño poblacional

Manejo de Datos Específico

- **Entradas:**
 - Dos poblaciones ordenables por F_p
 - Parámetros k, N de configuración
- **Intermedios:**
 - Poblaciones ordenadas
 - Contador num_a_copiar
- **Salida:**
 - $\text{Poblacion_Siguiete}$ con N individuos

Salidas Principales

- **Población siguiente:**
 - $\{\text{élite}\} \cup \{\text{mejores descendientes}\}$
 - Lista para próxima iteración del algoritmo

Interacciones Internas

- **Con módulo de ordenación:** Clasificación por F_p
- **Con gestor de poblaciones:** Fusión controlada de individuos
- **Con ConfigurationData:** Lectura de k y N

Diagrama del Proceso

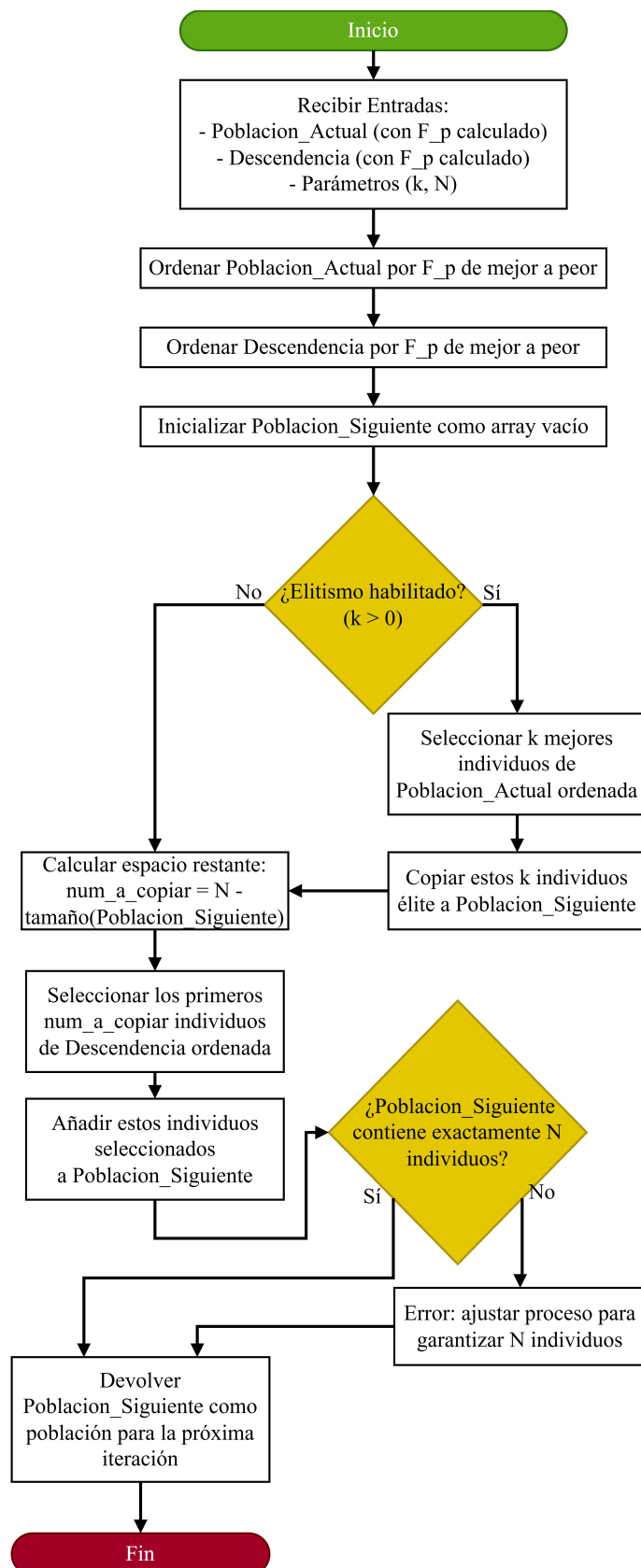


Figura 5.14: Diagrama de Proceso Interno 14: Selección de Individuos

5.1.15. Proceso Interno 15: Mostrar Resultados

Objetivo de la Actividad

El propósito principal de la actividad “**Mostrar Resultados**” es presentar de manera clara y comprensible al usuario final, a través de la interfaz gráfica (UI), la solución óptima encontrada por el Algoritmo Bioinspirado al concluir el proceso de optimización. Esto incluye el mejor conjunto de parámetros (ej. masas finales optimizadas) y su correspondiente valor de fitness, definido como el Exponente de Lyapunov (LE) minimizado. Opcionalmente, puede incluir una visualización final estática de la trayectoria asociada a la solución óptima.

Entradas Principales

- **Señal de finalización:** Evento que indica término del proceso de optimización
- **Parámetros óptimos:** Valores numéricos finales (ej. masas de cuerpos)
- **Valor de fitness:** LE asociado a la solución óptima
- **Datos de trayectoria (opcional):** Posiciones/velocidades de la simulación REBOUND
- **Resumen de optimización (opcional):** Detalles del proceso de optimización

Sub-pasos Secuenciales

Este apartado es proporcionado para profundizar y describir de forma textual cada paso contenido dentro del diagrama del proceso descrito en la figura 5.15.

1. Recibir Señal de Finalización

- UI detecta notificación de término del Algoritmo Bioinspirado

2. Recibir Datos Finales

- Datos recibidos incluyen:
 - Parámetros óptimos (valores numéricos)
 - LE final
 - Bandera de existencia de datos de trayectoria
 - Resumen de optimización (si está disponible)

3. Formatear Datos para Presentación

- Conversión a formatos legibles:
 - Parámetros: valor $\rightarrow$ cadena (ej. ‘‘0.1234’’)
 - $LE \rightarrow$ ‘‘LE: -0.567’’

- Generación de etiquetas descriptivas (ej. ‘‘Masa Cuerpo 1 Final:~’’)
- Resumen $\rightarrow$ texto descriptivo (ej. ‘‘50 generaciones’’)

4. Manejar Visualización Final (Condicional)

Condición 1: Verificar existencia de datos de trayectoria

Condición 2: Confirmar configuración de visualización habilitada

Si ambas condiciones se cumplen:

- Preparación de datos:
 - Extracción de posiciones/velocidades
 - Determinación de escalas (ej. rango de coordenadas)
 - Generación de metadatos (etiquetas ejes, título)
- Llamado al Módulo de Visualización
- Integración del gráfico en UI

Si falla alguna condición:

- Mostrar solo resultados textuales

5. Presentar Resultados en Pantalla

- Actualización de elementos UI:
 - Parámetros óptimos formateados con etiquetas
 - *LE* con interpretación (ej. ‘‘indica estabilidad’’)
 - Resumen de optimización (si existe)
 - Gráfico estático (si se generó)
- Habilitación de controles:
 - Botones Guardar, Reiniciar, Cerrar

6. Esperar Interacción del Usuario

- UI permanece en estado receptivo para acciones posteriores

Lógica Interna y Decisiones

- **Decisión clave:** Generar visualización gráfica
 - Depende de:
 - 5(a) Datos de trayectoria disponibles
 - 5(b) Configuración de visualización activada
- **Manejo de opcionales:**

- Omisión automática de secciones sin datos

Manejo de Datos Específico

- **Entradas:**
 - Parámetros: float/double
 - *LE*: float/double
 - Trayectoria: arrays/listas (opcional)
- **Intermedios:**
 - Cadenas formateadas
 - Datos gráficos procesados
- **Salidas:**
 - Texto UI formateado
 - Gráfico estático (opcional)

Salidas Principales

- UI actualizada con:
 - Resultados numéricos y textuales
 - Visualización gráfica (condicional)
 - Controles interactivos habilitados

Interacciones Internas

- **Con Algoritmo Bioinspirado:**
 - Recepción asincrónica de datos finales
- **Con UI:**
 - Actualización dinámica de componentes visuales
- **Con Módulo de Visualización:**
 - Integración gráfica bajo demanda

Diagrama del Proceso

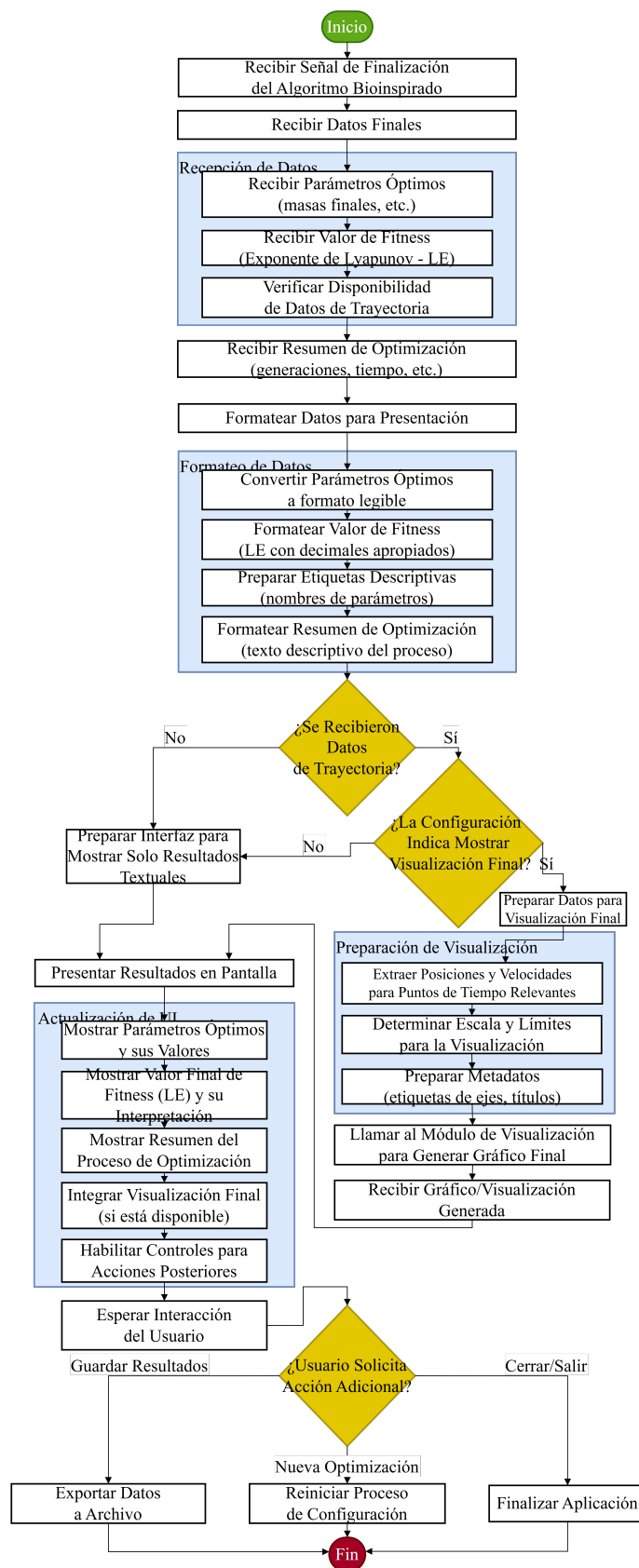


Figura 5.15: Diagrama de Proceso Interno 15: Mostrar Resultados

5.1.16. Matriz de Procesos

La Matriz de Resumen de Procesos presentada en esta sección ofrece un resumen consolidado y estructurado de los procesos clave del proyecto “Modelo para representar comportamientos gravitacionales con dos cuerpos”. Cada fila de la tabla describe un proceso específico, incluyendo su nombre, objetivo, los actores o componentes involucrados, el evento que lo inicia, las entradas y salidas principales, y un resumen de los pasos esenciales. Esta disposición permite a los usuarios obtener una visión clara y rápida del propósito y funcionamiento de cada proceso dentro del sistema, facilitando la comprensión y el análisis sin necesidad de consultar descripciones más extensas.

Los procesos detallados en la tabla son fundamentales para el desarrollo del proyecto, cubriendo desde la captura de parámetros ingresados por el usuario hasta la creación de gráficos que ilustran los comportamientos gravitacionales simulados. Estos procesos trabajan en conjunto para alcanzar el objetivo principal del proyecto: minimizar el Exponente de Lyapunov (LE) mediante algoritmos bioinspirados, apoyándose en herramientas como la biblioteca REBOUND para simulaciones y métodos de optimización como FMM/Barnes-Hut.

Esta matriz está diseñada para ser una herramienta práctica tanto para analistas de sistemas como para diseñadores de software y otros stakeholders involucrados. Su estructura, con columnas específicas como “*Trigger* (Evento)” y “Pasos Clave”, destaca las dependencias y el flujo de datos entre los procesos, ofreciendo una perspectiva integral del sistema. En esencia, la tabla no solo actúa como una referencia rápida, sino que también ilustra cómo los distintos elementos del proyecto —interfaz, simulación, optimización y visualización— se integran para cumplir con los objetivos establecidos.

Tabla 5.1: Matriz de Procesos

Nombre del Proceso	Objetivo	Actores Involucrados	Inicio (Evento)	Entradas Principales	Salidas Principales	Pasos Clave
Captura Parámetros	Recopilar, validar y almacenar parámetros de configuración del usuario.	UI, Módulo de Validación	Usuario interactúa con la UI para configurar.	Evento de inicio, interacciones del usuario (clicks, selecciones, texto).	Estructura <code>ConfigurationData</code> validada, estado de UI actualizado.	1. Presentar formulario en UI. 2. Recibir y validar entradas. 3. Almacenar configuración validada. 4. Habilitar control de inicio.
Mostrar Resultados	Presentar solución óptima y visualización final al usuario.	UI, Módulo de Visualización, Algoritmo Bioinspirado	Señal de finalización del Algoritmo Bioinspirado.	Parámetros óptimos, valor de fitness, datos de trayectoria (opcional).	Actualización visual de la UI con resultados finales.	1. Recibir datos finales. 2. Formatear datos para presentación. 3. (Condicional) Generar visualización final. 4. Presentar resultados en UI.
Inicializar Población	Generar conjunto inicial de N individuos dentro de rangos válidos.	Algoritmo Bioinspirado, Generador Aleatorio	Inicio del proceso de optimización.	Tamaño de población (N), rangos de parámetros.	Lista de N individuos (conjuntos de parámetros).	1. Leer configuración (N, rangos). 2. Generar parámetros dentro de rangos. 3. Crear lista de población inicial.

Continúa en la siguiente página

Tabla 5.1 – continuación de la página anterior

Nombre del Proceso	Objetivo	Actores Involucrados	Inicio (Evento)	Entradas Principales	Salidas Principales	Pasos Clave
Generar Nueva Generación	Crear nueva población de N individuos usando operadores genéticos.	Algoritmo Bioinspirado, Operadores Genéticos	Inicio de cada iteración del algoritmo genético.	Población actual, parámetros de configuración (Pc, Pm, η_c , η_m , k).	Lista de N nuevos individuos (conjuntos de parámetros).	1. Seleccionar padres (Torneo). 2. Aplicar cruzamiento (SBX) condicional. 3. Aplicar mutación (Polinomial) condicional. 4. Corregir límites de parámetros.
Evaluar Fitness	Calcular fitness penalizado (F_p) combinando LE y violaciones.	R E B O U N D, Módulo de Cálculo de LE, Restricciones	Evaluación de cada individuo en el algoritmo.	Parámetros del individuo, resultados de simulación, restricciones, lambda_p.	Valor numérico de $F_p(x)$.	1. Calcular LE ($f(x)$). 2. Calcular penalización $P(x)$ por violaciones. 3. Calcular $F_p(x) = f(x) + \lambda_p \cdot P(x)$.
Formar Siguiete Población	Construir la población para la siguiente iteración con elitismo.	Algoritmo Bioinspirado	Después de generar descendencia y evaluar fitness.	Población actual, descendencia, tamaño de élite (k).	Lista de N individuos para la siguiente generación.	1. Ordenar poblaciones por F_p. 2. Copiar k mejores individuos (<i>élite</i>). 3. Completar con mejores descendientes.

Continúa en la siguiente página

Tabla 5.1 – continuación de la página anterior

Nombre del Proceso	Objetivo	Actores Involucrados	Inicio (Evento)	Entradas Principales	Salidas Principales	Pasos Clave
Crear Nueva Simulación	Instanciar un nuevo entorno de simulación vacío en REBOUND.	REBOUND, Algoritmo Bioinspirado	Solicitud del Algoritmo Bioinspirado para nueva simulación.	Solicitud de creación (trigger).	Referencia a nuevo objeto <code>Simulation</code> .	1. Invocar constructor de REBOUND. 2. Inicialización interna por REBOUND. 3. Retornar referencia a ‘Simulation’.
Agregar Cuerpos	Añadir una partícula con propiedades físicas a la simulación.	REBOUND, Algoritmo Bioinspirado	Configuración de la simulación antes de ejecutar.	Referencia a <code>simulation</code> , parámetros del cuerpo (masa, posición, velocidad).	Instancia <code>simulation</code> modificada con nueva partícula.	1. Recibir parámetros del cuerpo. 2. Invocar API de REBOUND para añadir partícula. 3. Actualización interna por REBOUND.
Definir Condiciones	Configurar parámetros operativos de la simulación (dt , G , $T_{\text{máx}}$).	REBOUND, Algoritmo Bioinspirado	Antes de iniciar la ejecución de la simulación.	Referencia a <code>simulation</code> , parámetros de configuración (dt , G , $T_{\text{máx}}$).	Instancia <code>simulation</code> configurada, valor $T_{\text{máx}}$ para ejecución.	1. Recibir y desempaquear parámetros. 2. Establecer dt y G en <code>simulation</code>. 3. Procesar $T_{\text{máx}}$ para uso posterior.

Continúa en la siguiente página

Tabla 5.1 – continuación de la página anterior

Nombre del Proceso	Objetivo	Actores Involucrados	Inicio (Evento)	Entradas Principales	Salidas Principales	Pasos Clave
Iniciar/Ejecutar Simulación	Ejecutar la integración numérica paso a paso hasta $T_{\text{máx}}$.	REBOUND, Módulo de Visualización	Inicio de la simulación para un individuo.	Referencia a <code>sim</code> , $T_{\text{máx}}$, lista de tiempos de visualización.	Estructura <code>SimulationResult</code> con trayectoria completa.	1. Inicializar almacenamiento de resultados. 2. Bucle: avanzar un paso, almacenar estado, verificar visualización. 3. Empaquetar y devolver resultados.
Avanzar un Paso de Simulación	Avanzar el estado del sistema un paso dt usando WH-Fast (DKD).	REBOUND (Integrador WH-Fast)	Cada iteración del bucle de simulación.	Estado actual de <code>sim</code> , <code>target_time</code> ($t + dt$).	Instancia <code>sim</code> actualizada al tiempo <code>target_time</code> .	1. Primer Drift ($dt/2$). 2. Kick (dt). 3. Segundo Drift ($dt/2$). 4. Finalización y sincronización.
Actualizar Estado	Consolidar el nuevo estado del sistema tras la integración.	REBOUND	Después de cada paso de integración.	Referencia a <code>sim</code> post-integración.	Instancia <code>sim</code> con estado consolidado.	1. Validar tiempo de simulación. 2. Verificar actualización de posiciones y velocidades. 3. Confirmar masas sin cambios.

Continúa en la siguiente página

Tabla 5.1 – continuación de la página anterior

Nombre del Proceso	Objetivo	Actores Involucrados	Inicio (Evento)	Entradas Principales	Salidas Principales	Pasos Clave
Recolectar Datos	Extraer estado actual del sistema en instantes de visualización.	Módulo de Visualización, REBOUND	Coincidencia de <code>sim.t</code> con <code>t_vis</code> .	Referencia a <code>sim</code> o estado actual.	Estructura <code>VisualizationState</code> con instantánea del sistema.	1. Leer tiempo, masas, posiciones (y velocidades si aplica). 2. Empaquetar datos en <code>VisualizationState</code> .
Procesar Datos	Transformar y formatear datos para la biblioteca gráfica.	Módulo de Visualización	Recepción de <code>VisualizationState</code> .	<code>VisualizationState</code> , configuración de visualización.	Datos formateados para la API gráfica.	1. Verificar configuración de visualización. 2. (Condicional) Transformar coordenadas. 3. Seleccionar datos relevantes. 4. (Condicional) Calcular valores derivados. 5. Formatear para API gráfica.
Generar Gráficos	Dibujar o actualizar la representación visual en la pantalla.	Módulo de Visualización, Biblioteca Gráfica	Recepción de datos formateados.	Datos formateados, contexto gráfico.	Representación gráfica actualizada en la UI.	1. Preparar lienzo (limpiar si es actualización). 2. Llamar funciones de dibujo de la API gráfica. 3. Actualizar/refrescar pantalla.

5.2. Diagrama de Actividades

La representación explícita del flujo de trabajo operativo es fundamental para el diseño, la implementación y la validación del sistema propuesto en este trabajo. Dicho sistema integra técnicas de optimización bioinspiradas con simulaciones gravitacionales de N-cuerpos, utilizando la biblioteca REBOUND y abordando inicialmente el caso de dos cuerpos. La comprensión detallada de la secuencia de actividades, las bifurcaciones lógicas y la interacción entre los componentes clave —*Usuario*, *Interfaz de Usuario*, *Motor de Optimización* (Algoritmo Bioinspirado), *Simulador* (REBOUND) y *Módulo de Visualización*— resulta esencial para gestionar la complejidad inherente a esta integración metodológica.

El diagrama de actividades presentado en la Sección 5.2.1 sirve como una representación formal de este proceso dinámico. Modela cómo el algoritmo bioinspirado gestiona la generación de conjuntos de parámetros, inicia ejecuciones de simulación en REBOUND y evalúa los resultados para refinar iterativamente la búsqueda de soluciones que satisfagan criterios predefinidos (p. ej., estabilidad orbital).

Mediante la descomposición del proceso global en pasos discretos y la asignación de responsabilidades a través de carriles (swimlanes), el diagrama clarifica la lógica operativa y establece una base para una implementación estructurada y verificable. Asegura que la integración del ciclo de optimización con el motor de simulación sea coherente con los objetivos del proyecto, facilitando el desarrollo de una solución robusta para la exploración de escenarios de N-cuerpos bajo criterios de optimización específicos.

5.2.1. Diagrama de Actividades

A continuación, se presenta el diagrama de actividades que modela el flujo operativo del sistema, abarcando la optimización de parámetros mediante algoritmos bioinspirados y la ejecución de simulaciones N-cuerpos con REBOUND.

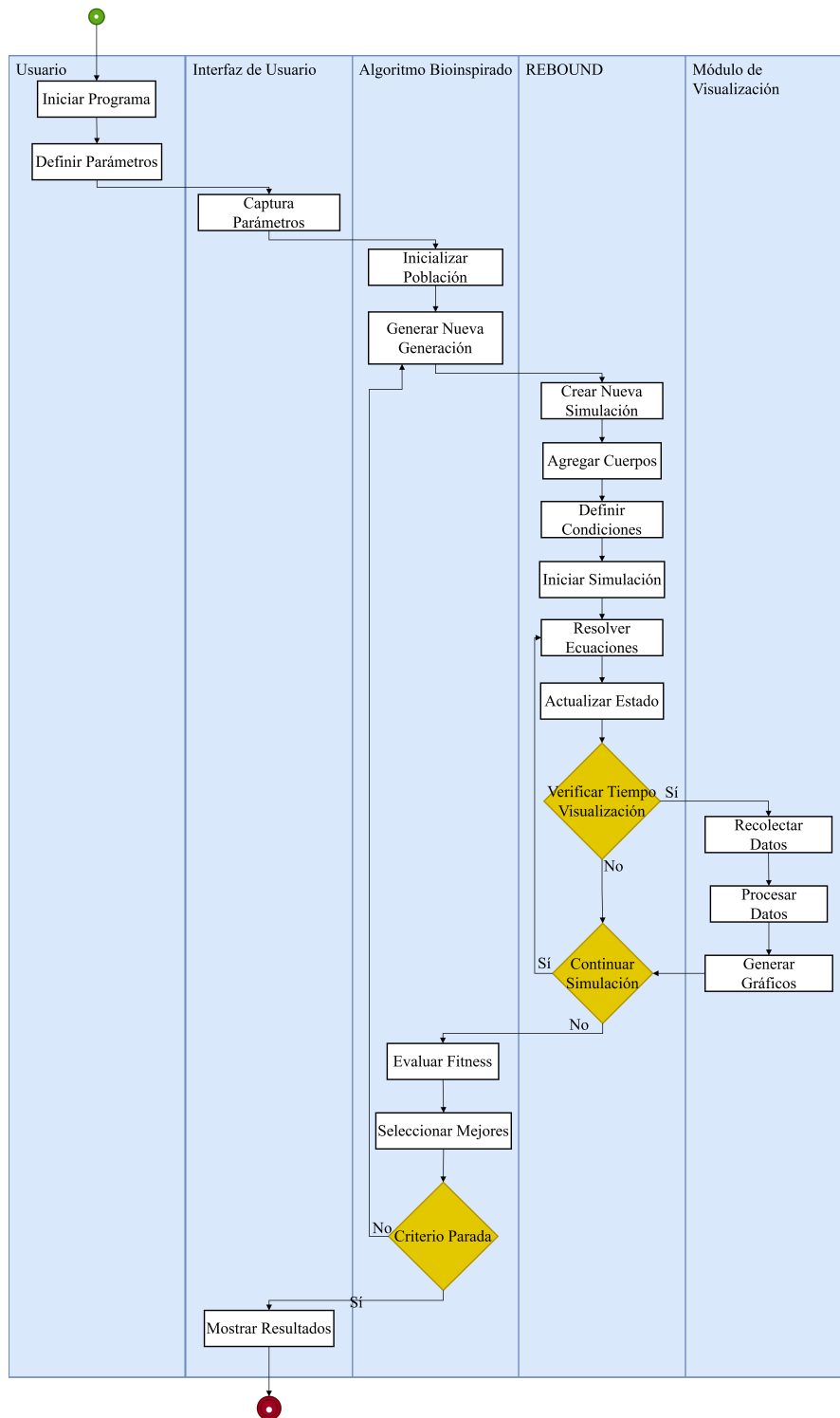


Figura 5.16: Diagrama de Actividades UML del proceso de simulación y optimización.

5.2.2. Descripción del Diagrama de Actividades

Estructura del Diagrama: Carriles (Swimlanes)

El diagrama se organiza en cinco carriles (swimlanes), cada uno representando un actor o componente lógico del sistema, delineando sus responsabilidades específicas:

1. **Usuario:** Representa al actor humano que interactúa con el sistema. Es responsable de iniciar la ejecución y proporcionar la configuración inicial requerida.
2. **Interfaz de Usuario (UI):** Componente software que media la interacción entre el *Usuario* y el núcleo del sistema. Gestiona la captura de parámetros de entrada y la presentación de resultados.
3. **Algoritmo Bioinspirado:** Componente central responsable de la optimización. Administra la población de soluciones candidatas (conjuntos de parámetros), aplica operadores evolutivos (generación, evaluación, selección) y determina la terminación del proceso según criterios definidos.
4. **REBOUND:** Representa la biblioteca de simulación gravitacional. Ejecuta simulaciones de N-cuerpos (inicialmente, 2 cuerpos) basadas en los parámetros proporcionados por el *Algoritmo Bioinspirado* para cada individuo evaluado. Cabe destacar que se instancia una nueva simulación para cada conjunto de parámetros a evaluar.
5. **Módulo de Visualización:** Componente encargado de generar representaciones gráficas del estado del sistema en instantes de tiempo específicos, definidos por el *Usuario*, para el monitoreo de la evolución dinámica.

Flujo Detallado de Actividades

El proceso modelado en el diagrama sigue la secuencia lógica que se describe a continuación:

1. **Inicialización y Configuración:**
 - El *Usuario* inicia la ejecución de la aplicación (**Iniciar Programa**).
 - El *Usuario* especifica los parámetros de configuración inicial, incluyendo rangos de búsqueda para la optimización, la función objetivo, criterios de parada, y los instantes discretos de tiempo para la visualización (**Definir Parámetros**).
 - La *Interfaz de Usuario* captura y valida esta información (**Captura Parámetros**).
2. **Ciclo Iterativo de Optimización y Simulación:**
 - La *UI* transmite los parámetros de configuración al *Algoritmo Bioinspirado*, el cual inicializa su población de soluciones candidatas (**Inicializar Población**).
 - **Inicio del Bucle de Optimización:** El algoritmo genera una nueva generación de conjuntos de parámetros candidatos aplicando sus operadores definidos (**Generar Nueva Generación**).

- **Ejecución de Simulaciones por Individuo:** Para cada conjunto de parámetros (individuo) en la generación actual:
 - Se instruye a *REBOUND* para crear una nueva instancia de simulación (**Crear Nueva Simulación**).
 - Se añaden los cuerpos celestes al entorno de simulación con las propiedades especificadas por el conjunto de parámetros actual (**Agregar Cuerpos**).
 - Se configuran las condiciones restantes de la simulación, como el integrador numérico y el paso de tiempo (**Definir Condiciones**).
 - *REBOUND* inicia la simulación (**Iniciar Simulación**).
 - **Bucle Interno de Simulación (REBOUND):**
 - *REBOUND* resuelve las ecuaciones de movimiento para avanzar un paso de tiempo (**Resolver Ecuaciones**).
 - Actualiza el estado del sistema (posiciones, velocidades) (**Actualizar Estado**).
 - Verifica si el tiempo de simulación actual (t_{sim}) corresponde a uno de los instantes de visualización solicitados (t_{vis}) (**Verificar Tiempo Visualización**).
 - Si $t_{\text{sim}} = t_{\text{vis}}$ (Condición Verdadera): Se activa el *Módulo de Visualización*, que recolecta los datos relevantes del estado actual (**Recolectar Datos**), los procesa (**Procesar Datos**) y genera la salida gráfica correspondiente (**Generar Gráficos**). La simulación continúa.
 - Si $t_{\text{sim}} \neq t_{\text{vis}}$ (Condición Falsa): La simulación continúa directamente al siguiente paso.
 - *REBOUND* verifica si se ha alcanzado la condición de término de la simulación (p. ej., tiempo final T_{max}) (**Continuar Simulación**). Si no se ha alcanzado (Condición Falsa), retorna a **Resolver Ecuaciones**. Si se ha alcanzado (Condición Verdadera: **Simulación Terminada**), la ejecución para este individuo finaliza.
- **Evaluación:** Tras la finalización de la simulación para un individuo, el *Algoritmo Bioinspirado* utiliza los resultados (p. ej., una métrica derivada del estado final o la trayectoria) para calcular su valor de aptitud (fitness) (**Evaluar Fitness**). Este paso se repite para todos los individuos de la generación.
- **Selección y Criterio de Parada:** El *Algoritmo Bioinspirado* aplica sus mecanismos de selección basados en la aptitud calculada (**Seleccionar Mejores**) y verifica si se cumple el criterio global de parada (p. ej., número máximo de generaciones, convergencia de la aptitud) (**Criterio Parada**).

- Si el criterio no se cumple (Condición Falsa): Se procede a generar la siguiente generación (**Generar Nueva Generación**), continuando el ciclo de optimización.
- Si el criterio se cumple (Condición Verdadera): El ciclo de optimización concluye.

3. Finalización:

- El *Algoritmo Bioinspirado* transfiere la mejor solución encontrada (conjunto de parámetros óptimos y su aptitud asociada) a la *Interfaz de Usuario*.
- La *Interfaz de Usuario* presenta los resultados finales al *Usuario* (**Mostrar Resultados**).
- El proceso finaliza (nodo final de actividad).

5.2.3. Aspectos Clave del Diseño Reflejados en el Diagrama

El diagrama de actividades pone de manifiesto varios aspectos fundamentales del diseño del sistema:

- **Modularidad y Separación de Responsabilidades:** La estructura en carriles delimita claramente las funciones del componente de optimización (*Algoritmo Bioinspirado*) y del componente de simulación física (*REBOUND*), promoviendo un diseño modular.
- **Naturaleza Iterativa del Proceso:** El bucle principal que engloba **Generar Nueva Generación**, la ejecución de simulaciones en *REBOUND*, **Evaluar Fitness** y **Criterio Parada** representa visualmente el núcleo iterativo característico de los algoritmos evolutivos y otros métodos de optimización bioinspirados.
- **Simulación como Evaluación de Aptitud:** Desde la perspectiva del optimizador, el módulo *REBOUND* actúa como una función objetivo (o parte de ella), evaluando la calidad (aptitud) de un conjunto de parámetros candidato mediante la ejecución de una simulación física.
- **Instanciación Independiente de Simulaciones:** El flujo muestra que cada evaluación de un individuo dentro del ciclo de optimización requiere la creación y ejecución de una nueva instancia de simulación en *REBOUND*, configurada con los parámetros específicos de dicho individuo.
- **Control Discreto de la Visualización:** La generación de visualizaciones está integrada en el bucle de simulación de *REBOUND* pero se activa únicamente en puntos temporales discretos especificados por el *Usuario*, evitando una sobrecarga continua.

5.3. diccionario de datos

En el desarrollo de este proyecto, la representación adecuada de los datos es funda-

mental para garantizar la consistencia, comprensión y el correcto funcionamiento del sistema, buscando facilitar tanto la interpretación del modelo, así como su mantenimiento y expansión futura.

Por lo cual, en esta sección se mostrará el diccionario de datos, donde se describirán los atributos de los datos, sus significados, sus restricciones y ejemplos que ilustran su uso.

Tabla 5.2: Cuerpo celeste.

Nombre del atributo	Descripción	Tipo de dato	Rango	Ejemplo
masa	Masa del cuerpo celeste que influye en su atracción gravitacional.	float	> 0 (positivos) para la primera masa, para la segunda masa mayor a 0 y menor a 10 veces la primera masa	1.0
a	Semieje mayor de la órbita, define el tamaño de la órbita elíptica, es la mitad del eje largo de la elipse y en caso de ser circular es el radio.	float	> 0 (positivos)	1.0
e	Excentricidad orbital, indica cuán alargada es la órbita siendo un círculo perfecto si la excentricidad es 0 y mientras mas estirada, mas cercana a 1	float	$[0, 1)$	0.1
inc_deg	Inclinación orbital respecto al plano de referencia, en grados.	float	$[0^\circ, 180^\circ]$	30.0
perturba	Indica si se aplica una perturbación inicial pequeña a los parámetros.	bool	True o False	True

Tabla 5.3: Simulación.

Nombre del atributo	Descripción	Tipo de dato	Rango	Ejemplo
t_max	Tiempo total de simulación en años.	float	> 0 (positivos)	100.0
N_steps	Número de pasos a almacenar, define la resolución temporal de la simulación. A mayor número de pasos, mayor resolución.	entero	> 0 (positivos)	1000
sim.units	Unidades de la simulación, siendo en orden unidad de distancia, unidad de tiempo y unidad de masa.	texto	AU, yr, Msun	AU, yr, Msun
sim.integrator	Indica el integrador numérico que se va a utilizar para avanzar la simulación en el tiempo.	texto	ias15, whfast, BS, mercurius	ias15
x, y, z	Guarda las posiciones de nuestros cuerpos	array(float)	> 0 (positivos)	[5.0, 230.0, 20.0]

Tabla 5.4: Métricas.

Nombre del atributo	Descripción	Tipo de dato	Rango	Ejemplo
times	Array que guarda los tiempos en la simulación.	array(float)	> 0 (positivos)	[0.0, 100.0, 200.0]
energy	Energía total del sistema, incluye la energía cinética y potencial.	float	Valor real	-0.5
a_arr, a_pert	Array que guarda el semieje mayor de la órbita en cada paso de la simulación y el array de semiejes mayores perturbados.	array(float)	> 0 (positivos)	[1.0, 1.5, 2.0]
e_arr, e_pert	Array que guarda la excentricidad orbital en cada paso de la simulación y el array de excentricidades perturbadas.	array(float)	[0, 1)	[0.1, 0.2, 0.3]
Exponente de Lyapunov (λ)	Indica la tasa de crecimiento exponencial de las perturbaciones, relacionada con la estabilidad del sistema.	float	Valor real	0.01

5.3.1. Relacion de datos

En esta sección explicaremos como se interconectan los distintos atributos descritos previamente para tener un mayor entendimiento del sistema, así como el flujo de datos que se tiene.

1. times

Para conocer los tiempos en la simulación, lo que hacemos es generar un array de `N_steps` elementos, empezando por el 0 hasta `t_max`, cada elemento va a estar a una distancia de `t_max` entre `N_steps`

2. x, y, z, energy, a_arr, a_pert, e_arr, e_pert

Nos apoyamos de las funciones que nos brinda REBOUND el cual con su funcion de `simulation` nos brinda todos esos datos pero para ello necesitamos conocer los tiempos en la simulación (`times`), asi como las masas de los cuerpos involucrados, sus semiejes mayores, su excentricidad y el integrador que se va a utilizar

3. Exponente de Lyapunov

Para poder calcular el exponente de Lyapunov necesitamos calcular primero lo siguiente: `a_arr`, `e_arr`, `a_pert`, `e_pert`, para poder asi obtener nuestro delta de orbital

CAPÍTULO 6

Diseño

6.1. Diseño Arquitectónico

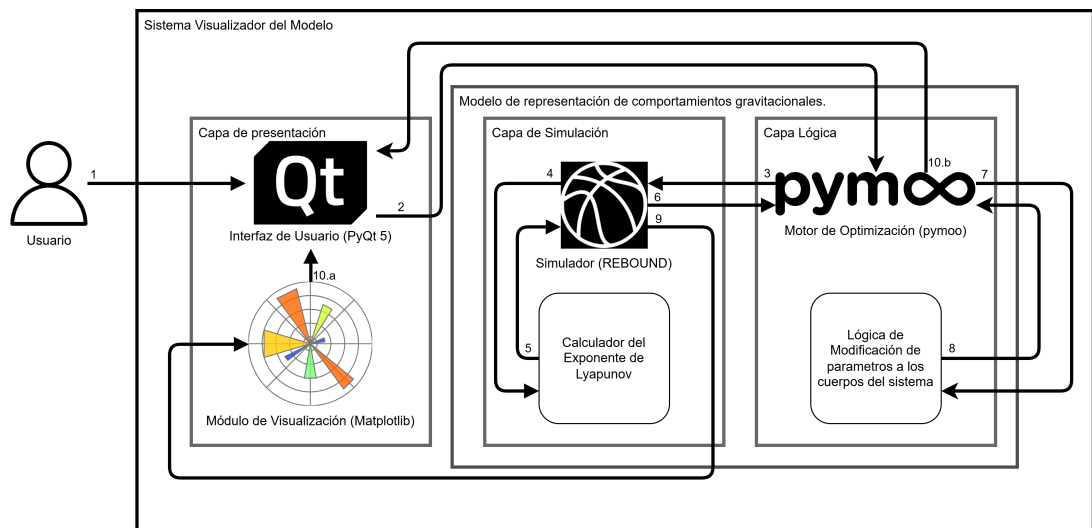


Figura 6.1: Diagrama de Arquitectura.

6.1.1. Introducción y Objetivos de Diseño

El presente diseño arquitectónico describe la estructura y organización del “Sistema Visualizador del Modelo”, concebido para la simulación, optimización y análisis de comportamientos gravitacionales, con un enfoque inicial en sistemas de dos cuerpos. La arquitectura se fundamenta en principios de **modularidad, encapsulamiento y separación de incumbencias**, con el fin de garantizar la extensibilidad, mantenibilidad y robustez del sistema. Se ha adoptado un paradigma arquitectónico híbrido que combina un enfoque basado en componentes, una estructuración en capas lógicas y una comunicación parcialmente dirigida por eventos.

Los objetivos primordiales que guían este diseño son:

1. **Modularidad y Cohesión:** Facilitar el desarrollo, prueba y mantenimiento independientes de los distintos módulos funcionales.

2. **Bajo Acoplamiento:** Minimizar las dependencias entre componentes para permitir la sustitución o mejora de tecnologías individuales (e.g., motor de simulación, biblioteca de optimización) con un impacto reducido en el resto del sistema.
3. **Capacidad de Modificación Dinámica:** Soportar la alteración de parámetros clave del sistema (ej. masas) durante la ejecución del ciclo de optimización.
4. **Eficiencia Computacional:** Integrar herramientas especializadas como REBOUND para la simulación N-cuerpos y pymoo para la optimización bioinspirada, buscando un balance entre precisión y tiempo de ejecución.
5. **Visualización Interactiva:** Proveer retroalimentación visual clara y comprensible del proceso de simulación y optimización al usuario a través de PyQt5 y Matplotlib.

La arquitectura propuesta se articula en torno a tres capas principales: Presentación, Lógica y Simulación/Datos, cada una con componentes especializados que interactúan mediante interfaces bien definidas.

6.1.2. Arquitectura en Capas y Componentes Principales

El sistema se organiza en las siguientes capas lógicas, cada una agrupando componentes con responsabilidades cohesivas:

Capa de Presentación

Propósito General: Gestionar toda la interacción con el usuario final, desde la captura de datos de entrada hasta la visualización de resultados y el estado del sistema. Esta capa abstrae los detalles de la interfaz gráfica y la representación de datos del resto de la lógica del sistema.

1. 2.1.1. Interfaz de Usuario (UI)

- **Tecnología:** PyQt5.
- **Responsabilidades Fundamentales:**
 - **Recolección y Validación de Parámetros:** Presenta formularios y controles para que el usuario ingrese los parámetros iniciales de la simulación (e.g., masas, posiciones, velocidades iniciales de los cuerpos) y los parámetros de configuración del algoritmo de optimización (e.g., tamaño de la población, número de generaciones, criterios de parada, rangos de los parámetros a optimizar). Implementa rutinas de validación para asegurar la coherencia y admisibilidad de los datos antes de iniciar cualquier proceso.
 - **Iniciación y Control de Procesos:** Actúa como el punto de entrada para disparar el ciclo de optimización invocando al Motor de Optimización. Permite al usuario iniciar, pausar o detener los procesos.
 - **Presentación de Resultados y Retroalimentación:** Despliega

los resultados finales de la optimización (mejor conjunto de parámetros, valor de fitness correspondiente) y provee retroalimentación en tiempo real sobre el progreso del algoritmo de optimización (e.g., número de generación actual, mejor fitness de la generación).

- **Datos Manejados:**

- *Entrada:* Parámetros de configuración de simulación y optimización, eventos de control del usuario.
- *Salida:* Representación textual y gráfica de resultados, estado del sistema.

2. 2.1.2. Módulo de Visualización

- **Tecnología:** Matplotlib, integrado en la UI de PyQt5.

- **Responsabilidades Fundamentales:**

- **Generación de Representaciones Gráficas:** Transforma los datos numéricos provenientes de la Capa de Simulación (trayectorias, estados energéticos) y de la Capa Lógica (progreso de la optimización) en visualizaciones comprensibles.
- **Renderizado y Actualización Dinámica:** Es responsable de renderizar los gráficos (e.g., trayectorias orbitales 2D/3D, evolución de la energía del sistema, convergencia del fitness) dentro de los widgets designados en la UI./ Soporta la actualización dinámica de estos gráficos para reflejar el estado cambiante durante la simulación y la optimización.
- **Procesamiento de Datos para Visualización:** Realiza las transformaciones necesarias sobre los datos crudos (e.g., escalado, selección de subconjuntos de datos, cálculo de métricas derivadas para visualización) para adecuarlos a los requisitos de Matplotlib y a las preferencias del usuario.
- **Datos Manejados:**
 - *Entrada:* Series temporales de posiciones y velocidades, valores de energía, métricas de fitness, configuración de visualización.
 - *Salida:* Objetos gráficos de Matplotlib (figuras, ejes, primitivas de dibujo).

Capa Lógica

Propósito General: Orquestar el proceso de optimización de parámetros. Contiene la inteligencia para guiar la búsqueda de soluciones óptimas y adaptar los parámetros del sistema basándose en el rendimiento observado.

1. 2.2.1. Motor de Optimización

- **Tecnología:** pymoo.

- **Responsabilidades Fundamentales:**
 - **Gestión del Algoritmo Bioinspirado:** Implementa el ciclo de vida de un algoritmo genético (u otro paradigma bioinspirado soportado por `pymoo`). Esto incluye la inicialización de una población de soluciones candidatas (conjuntos de parámetros), la aplicación iterativa de operadores evolutivos (selección, cruzamiento, mutación) para generar nuevas poblaciones, y la gestión de criterios de parada.
 - **Evaluación de la Función Objetivo (Fitness):** Para cada individuo (conjunto de parámetros) de la población, coordina su evaluación. Esto implica invocar a la Capa de Simulación para ejecutar una simulación con dichos parámetros y obtener las trayectorias, y luego utilizar el Calculador del Exponente de Lyapunov para obtener un valor escalar de fitness que cuantifique la “calidad” de esa solución.
 - **Orquestación del Flujo de Optimización:** Controla el flujo iterativo, decidiendo cuándo generar una nueva población, cuándo evaluar, y cuándo terminar el proceso.
- **Datos Manejados:**
 - *Entrada:* Configuración del algoritmo de optimización (desde la UI), valores de fitness (desde el Calculador LE).
 - *Salida:* Conjuntos de parámetros para evaluación (hacia la Capa de Simulación), mejor solución encontrada y métricas de progreso (hacia la UI).

2. 2.2.2. Lógica de Modificación de Parámetros

- **Responsabilidades Fundamentales:**
 - **Aplicación de Variaciones Paramétricas:** Actúa como intermediario para aplicar los conjuntos de parámetros generados por el Motor de Optimización (`pymoo`) a las instancias de simulación en REBOUND. Asegura que los parámetros propuestos por los operadores genéticos (e.g., masas, componentes de velocidad/posición inicial) se traduzcan correctamente en la configuración de cada simulación individual.
 - **Retroalimentación y Adaptación del Proceso de Optimización (Potencial):** Aunque el flujo principal es que `pymoo` genere los parámetros, este módulo podría incorporar lógicas para adaptar dinámicamente el propio proceso de optimización. Por ejemplo, basándose en la convergencia observada o la diversidad de la población, podría ajustar hiperparámetros de `pymoo` (como tasas de mutación/cruzamiento) o refinar los rangos de búsqueda de los parámetros, interactuando para ello con el Motor de Optimización.
- **Datos Manejados:**

- *Entrada:* Nuevos conjuntos de parámetros (individuos) generados por pymoo.
- *Salida:* Parámetros configurados para instancias específicas del Simulador REBOUND.

Capa de Simulación y Datos

Propósito General: Ejecutar las simulaciones físicas y realizar los cálculos necesarios para evaluar la calidad de las configuraciones paramétricas. Esta capa encapsula la física del problema y las herramientas numéricas para su resolución.

1. 2.3.1. Simulador

- **Tecnología:** REBOUND.
- **Responsabilidades Fundamentales:**
 - **Modelado del Sistema Físico:** Configura y ejecuta simulaciones del problema de N-cuerpos (inicialmente dos cuerpos) de acuerdo con los parámetros proporcionados (masas, posiciones iniciales, velocidades iniciales, constante gravitacional G , paso de tiempo dt , tiempo máximo de simulación T_{max}).
 - **Integración Numérica:** Utiliza los integradores numéricos provistos por REBOUND (e.g., WHFast, IAS15) para avanzar el estado del sistema en el tiempo, calculando las trayectorias de los cuerpos bajo la influencia de sus interacciones gravitacionales mutuas.
 - **Provisión de Datos de Trayectoria:** Exporta los resultados de la simulación, típicamente como series temporales de las posiciones y velocidades de cada cuerpo, que son consumidos por el Calculador del Exponente de Lyapunov y, opcionalmente, por el Módulo de Visualización.
- **Datos Manejados:**
 - *Entrada:* Parámetros físicos y de simulación (masas, estado inicial, G , dt , T_{max} , integrador a usar).
 - *Salida:* Series temporales de vectores de posición y velocidad para cada cuerpo.

2. 2.3.2. Calculador del Exponente de Lyapunov (LE)

- **Responsabilidades Fundamentales:**
 - **Cuantificación de la Estabilidad Dinámica:** Implementa el algoritmo para calcular el Exponente de Lyapunov Máximo (MLE) a partir de las trayectorias generadas por el Simulador. El LE es una métrica fundamental en la teoría de sistemas dinámicos que mide la tasa promedio de separación exponencial de trayectorias inicialmente cercanas. Un LE positivo indica comportamiento caótico, mientras que un LE cercano a cero o negativo sugiere estabilidad u orbitalidad regular.

- **Función de Fitness:** Provee el valor del LE calculado como la función objetivo (o un componente principal de ella) al Motor de Optimización. El objetivo de la optimización es, típicamente, encontrar conjuntos de parámetros que minimicen el LE, buscando configuraciones de mayor estabilidad.
- **Procesamiento de Trayectorias:** Toma como entrada los datos de trayectoria (series temporales de estado) de dos o más simulaciones ligeramente perturbadas (o utiliza métodos basados en la evolución de vectores tangentes) para estimar la tasa de divergencia.
- **Datos Manejados:**
 - *Entrada:* Datos de trayectoria (posiciones, velocidades a lo largo del tiempo) de REBOUND.
 - *Salida:* Un valor escalar representando el Exponente de Lyapunov.

6.1.3. Interacciones Principales y Flujos de Datos y Control

El funcionamiento coordinado del sistema se basa en una serie de interacciones clave entre sus componentes. A continuación, se detallan estos flujos de control y datos, numerados según la descripción original y expandidos para mayor claridad:

1. Usuario → Interfaz de Usuario (Inicio y Configuración Inicial):

- **Disparador:** Acción del operador humano.
- **Acción:** El usuario interactúa con la Interfaz de Usuario (UI) para introducir los parámetros fundamentales de la simulación (e.g., valores nominales o rangos para las masas de los dos cuerpos, sus posiciones y velocidades iniciales), la constante gravitacional G , el paso de tiempo Δt para la integración numérica, y el tiempo máximo de simulación $T_{\max}$ para cada evaluación individual. Adicionalmente, se configuran los parámetros del Motor de Optimización (**pymoo**), tales como el tamaño de la población de soluciones, el número máximo de generaciones, los criterios de parada, y la especificación de qué parámetros físicos (e.g., masas) serán optimizados junto con sus rangos de búsqueda permitidos. La activación de un control en la UI (e.g., botón “Iniciar Simulación/Optimización”) formaliza la entrada de estos datos.
- **Datos Intercambiados:** Estructuras de datos o variables conteniendo los valores numéricos y selecciones de configuración ingresados por el usuario.
- **Resultado Esperado:** La UI valida internamente la coherencia y admisibilidad de los parámetros (e.g., rangos lógicos, tipos de datos correctos) y los prepara para su posterior procesamiento y distribución a las capas correspondientes.

2. Interfaz de Usuario → Capa de Simulación (Configuración de Simulación Base/Visualización Directa - *Contexto Específico*):

- **Disparador:** Podría ser una acción del usuario para una simulación directa sin optimización, o un paso de configuración previo a la optimización.
- **Acción:** En un escenario donde el usuario desee ejecutar una simulación única con parámetros fijos (sin pasar por el ciclo de optimización bioinspirada) o para establecer una simulación base, la UI enviaría los parámetros validados (masas, posiciones, Δt , G , T_{max}) directamente al subsistema de simulación (específicamente al Simulador REBOUND a través de una lógica de control simplificada).
- **Datos Intercambiados:** Conjunto completo de parámetros físicos y de simulación necesarios para una ejecución de REBOUND.
- **Resultado Esperado:** El Simulador REBOUND se configura y está listo para ejecutar una integración numérica. *Nota: En el flujo principal de optimización, esta interacción directa es menos prominente, ya que el Motor de Optimización gestiona las invocaciones a REBOUND.*

3. Interfaz de Usuario → Motor de Optimización (pymoo) (Inicio del Ciclo Evolutivo):

- **Disparador:** Evento de “inicio” generado por la UI tras la validación de los parámetros de configuración (como se describe en la Interacción 1), específicamente cuando el objetivo es la optimización.
- **Acción:** La UI transfiere la configuración completa del problema de optimización al Motor de Optimización (pymoo). Esto incluye la definición de las variables de diseño (qué parámetros físicos se optimizan), sus límites (rangos de búsqueda), la función objetivo (implícitamente, minimizar el LE), y los parámetros del algoritmo genético (tamaño de población, número de generaciones, etc.).
- **Datos Intercambiados:** Objeto o estructura de datos que encapsula la definición del problema de optimización y los hiperparámetros del algoritmo genético.
- **Resultado Esperado:** pymoo se inicializa, creando la primera población de individuos (conjuntos de parámetros candidatos) de acuerdo con la estrategia de inicialización configurada (e.g., muestreo aleatorio dentro de los rangos).

4. Motor de Optimización (pymoo) → Simulador (REBOUND) (Solicitud de Evaluación de Individuo):

- **Disparador:** Necesidad del Motor de Optimización de evaluar el fitness de cada individuo (conjunto de parámetros) en la población actual.
- **Acción:** Para cada individuo, pymoo instruye (a través de la Lógica de Modificación de Parámetros si actúa como intermediaria, o directamente) al Simulador REBOUND para que ejecute una simulación. Los

parámetros específicos del individuo (e.g., masas `m1`, `m2` propuestas por `pymoo`) se utilizan para configurar esta instancia particular de la simulación, junto con los parámetros de simulación globales (`G`, `dt`, `T_max`).

- **Datos Intercambiados:** Vector de parámetros del individuo actual y parámetros de simulación fijos.
- **Resultado Esperado:** El Simulador `REBOUND` ejecuta una integración numérica completa para el conjunto de parámetros dado, generando una trayectoria.

5. Simulador (`REBOUND`) → Calculador del Exponente de Lyapunov (Provisión de Trayectorias):

- **Disparador:** Conclusión exitosa de una simulación en `REBOUND` para un individuo específico.
- **Acción:** `REBOUND` hace disponibles los datos de la trayectoria resultante. Esto típicamente comprende una serie temporal de los vectores de estado (posiciones y velocidades) de los dos cuerpos a lo largo del tiempo de simulación `T_max`.
- **Datos Intercambiados:** Estructura de datos (e.g., array multidimensional, lista de tuplas) que representa la evolución temporal del sistema.
- **Resultado Esperado:** El Calculador `LE` recibe estos datos y está preparado para realizar el cómputo del Exponente de Lyapunov.

6. Motor de Optimización (`pymoo`) → Simulador (`REBOUND`) (Invocaciones Múltiples para Evaluación):

- **Clarificación:** Esta interacción es una generalización de la Interacción 4. No es una interacción separada en el flujo secuencial, sino que enfatiza que el Motor de Optimización (`pymoo`) realizará múltiples solicitudes de simulación a `REBOUND`, una por cada individuo en la población actual que necesita ser evaluado, y esto se repetirá para cada nueva generación de individuos cuyos parámetros han sido “mutados” (modificados por operadores genéticos como cruzamiento y mutación).
- **Acción y Resultado:** Idénticos a la Interacción 4, pero repetidos para cada individuo de la población que `pymoo` necesita evaluar en una generación dada.

7. Motor de Optimización (`pymoo`) → Interfaz de Usuario (Retroalimentación de Progreso y Resultados):

- **Disparador:** Típicamente al final de cada generación del algoritmo evolutivo, o al alcanzar un criterio de parada.
- **Acción:** `pymoo` comunica al UI el estado actual del proceso de optimización. Esto incluye métricas como el número de la generación actual, el mejor valor de fitness (`LE` mínimo) encontrado en esa generación

o hasta el momento, y opcionalmente, los parámetros del individuo que logró dicho mejor fitness. Si el proceso ha concluido, se envían los resultados finales.

- **Datos Intercambiados:** Valores numéricos de métricas de optimización, y potencialmente el vector de parámetros del mejor individuo.
- **Resultado Esperado:** La UI actualiza sus elementos visuales (e.g., campos de texto, barras de progreso, gráficos de convergencia) para informar al usuario sobre el avance y los resultados de la optimización.

8. Lógica de Modificación de Parámetros → Motor de Optimización (pymoo) (Adaptación Dinámica del Proceso de Optimización):

- **Disparador:** Podría ser activado por hitos en el proceso de optimización (e.g., después de un cierto número de generaciones, o si se detecta estancamiento en la convergencia del fitness).
- **Acción:** La Lógica de Modificación de Parámetros, basándose en el análisis del progreso de la optimización (e.g., diversidad de la población, tasa de mejora del fitness), podría proponer ajustes a los hiperparámetros del Motor de Optimización (pymoo). Esto podría incluir la modificación dinámica de los rangos de búsqueda para ciertos parámetros físicos (si se observa que las mejores soluciones se concentran en subregiones) o el ajuste de las tasas de los operadores genéticos (e.g., aumentar la mutación si la diversidad es baja).
- **Datos Intercambiados:** Nuevos valores para hiperparámetros de pymoo o nuevos límites para las variables de diseño.
- **Resultado Esperado:** pymoo incorpora estos ajustes en las subsiguientes generaciones, potencialmente mejorando la eficiencia o la capacidad de exploración del algoritmo de optimización.

9. Simulador (REBOUND) → Motor de Optimización (pymoo) (Provisión de Datos Adicionales para Análisis):

- **Disparador:** Finalización de una simulación en REBOUND, concurrente con la Interacción 5.
- **Acción:** Además de las trayectorias completas necesarias para el cálculo del LE, REBOUND podría reenviar al Motor de Optimización (posiblemente a través de la Lógica de Modificación de Parámetros) datos intermedios o estados finales específicos (e.g., energía total final del sistema, momento angular, distancia mínima entre cuerpos durante la simulación).
- **Datos Intercambiados:** Valores escalares o vectores representando métricas adicionales de la simulación.
- **Resultado Esperado:** pymoo o la Lógica de Modificación de Parámetros pueden utilizar estos datos para análisis de convergencia más sofisticados, para la implementación de restricciones en el problema de

optimización (e.g., penalizar soluciones que no conservan la energía), o para refinar la función de fitness más allá del simple LE.

10. Visualización y Actualizaciones de UI:

- (a) **Módulo de Visualización → Interfaz de Usuario (Renderizado de Gráficos):**
 - **Disparador:** Solicitud de la UI para visualizar una trayectoria específica (e.g., la del mejor individuo de la generación actual) o datos de simulación.
 - **Acción:** El Módulo de Visualización, utilizando `Matplotlib`, procesa los datos de trayectoria o estado recibidos (que originalmente provinieron de `REBOUND` y fueron gestionados por la UI o el Motor de Optimización) y genera las representaciones gráficas correspondientes (e.g., órbitas, gráficos de energía vs. tiempo). Estos gráficos se renderizan o actualizan dentro de los widgets designados en la UI.
 - **Datos Intercambiados:** Objetos gráficos de `Matplotlib`.
 - **Resultado Esperado:** El usuario observa una representación visual actualizada del comportamiento del sistema o del progreso de la optimización.
- (b) **Motor de Optimización → UI / Indicadores (Actualización de Estado en Tiempo Real):**
 - **Disparador:** Emisión de eventos de estado por parte de `pymoo` (como se mencionó en la Interacción 7).
 - **Acción:** La UI recibe estos eventos o datos de estado (e.g., número de generación, mejor fitness actual) y actualiza los componentes visuales correspondientes, como barras de progreso, etiquetas de texto o gráficos de convergencia de fitness.
 - **Datos Intercambiados:** Métricas de estado de la optimización.
 - **Resultado Esperado:** El usuario obtiene una retroalimentación continua y en tiempo (casi) real sobre el estado del proceso de optimización.

6.1.4. Estilo y Paradigmas Arquitectónicos

La arquitectura del sistema se adhiere a los siguientes estilos y paradigmas para alcanzar los objetivos de diseño:

- **Arquitectura Basada en Componentes (Component-Based Architecture):**
 - **Justificación:** El sistema se descompone en módulos lógicos cohesivos (UI, Módulo de Visualización, Motor de Optimización, Lógica de Modificación de Parámetros, Simulador, Calculador LE). Cada componente encapsula una funcionalidad específica y expone interfaces bien

definidas (e.g., APIs programáticas, llamadas a funciones) para la interacción.

- **Beneficios:** Este enfoque promueve la reutilización de componentes, facilita el desarrollo paralelo y las pruebas unitarias aisladas. Por ejemplo, el Simulador **REBOUND** es un componente externo con su propia API, y el Motor de Optimización **pymoo** también opera como una biblioteca con interfaces claras. La sustitución de **pymoo** por otro motor de optimización sería factible modificando principalmente la Lógica de Modificación de Parámetros y las interacciones con la UI, siempre que el nuevo motor cumpla con un protocolo de comunicación similar para la evaluación de individuos.
- **Arquitectura en Capas (Layered Architecture):**
 - **Justificación:** Las capas (Presentación, Lógica, Simulación/Datos) establecen una jerarquía de abstracción y control. Las interacciones ocurren típicamente entre capas adyacentes. La Capa de Presentación interactúa con la Capa Lógica, la cual a su vez interactúa con la Capa de Simulación/Datos.
 - **Beneficios:** Este estilo reduce el acoplamiento global del sistema. Por ejemplo, la UI (Capa de Presentación) no necesita conocer los detalles internos de la integración numérica en **REBOUND** (Capa de Simulación/Datos); solo se comunica a través de la Capa Lógica. Esto mejora la mantenibilidad, ya que los cambios dentro de una capa (e.g., actualizar la biblioteca de graficación en la Capa de Presentación) tienen un impacto mínimo en otras capas, siempre que las interfaces entre capas se mantengan estables.
- **Arquitectura Dirigida por Eventos (Event-Driven Architecture) - Parcial:**
 - **Justificación:** Principalmente en la interacción entre la UI y el Motor de Optimización, y para la actualización de la visualización. La UI genera eventos (e.g., “botón Iniciar presionado”) que disparan acciones en la Capa Lógica. A su vez, el Motor de Optimización puede emitir eventos (e.g., “nueva generación completada”, “mejor fitness actualizado”) que son capturados por la UI para actualizar la visualización de progreso.
 - **Beneficios:** Permite un comportamiento reactivo y desacoplado, especialmente para la interfaz de usuario. Las actualizaciones de la UI pueden ocurrir asincrónicamente a medida que el proceso de optimización avanza, proporcionando una experiencia de usuario más fluida y retroalimentación en tiempo real sin bloquear el hilo principal de la UI durante cálculos intensivos.

6.1.5. Consideraciones Adicionales de Diseño

- **Mantenibilidad:** La clara separación de responsabilidades y el uso de interfaces definidas entre componentes facilitan la comprensión del código,

la localización de errores y la implementación de modificaciones o nuevas funcionalidades con menor riesgo de introducir efectos secundarios no deseados.

- **Escalabilidad Funcional:** La arquitectura está diseñada para ser extensible. Añadir un nuevo tipo de algoritmo de optimización implicaría desarrollar un nuevo componente o adaptar el existente en la Capa Lógica que interactúe con la API de `pymoo` o una biblioteca alternativa. De manera similar, soportar un nuevo integrador numérico en `REBOUND` sería una configuración dentro de la Capa de Simulación. La escalabilidad en términos de rendimiento para problemas de $N > 2$ cuerpos es inherentemente soportada por `REBOUND` y `pymoo` (que maneja vectores de parámetros), aunque requeriría ajustes en la Lógica de Modificación de Parámetros y en la función de fitness.
- **Capacidad de Pruebas:** La modularidad intrínseca permite la implementación de pruebas unitarias para cada componente de forma aislada (e.g., probar el Calculador LE con trayectorias predefinidas, verificar la lógica de los operadores genéticos en `pymoo` con funciones objetivo sintéticas). Las pruebas de integración se centrarían en verificar las interacciones correctas entre componentes y capas (e.g., asegurar que los parámetros de la UI se transmitan correctamente al Simulador a través del Motor de Optimización).
- **Protocolos de Comunicación:** Las interacciones entre componentes se realizan principalmente mediante llamadas a funciones directas dentro del mismo proceso. El intercambio de datos se realiza mediante estructuras de datos definidas (e.g., listas, diccionarios de Python, arrays de NumPy para trayectorias).

CAPÍTULO 7

Bitácora Implementación

7.1. Módulo de Simulación

7.1.1. Introducción

Este capítulo documenta de forma técnica y reproducible el proceso de implementación del *Módulo de Simulación* empleado para estudiar dinámicas gravitacionales de dos y tres cuerpos, así como la estimación del exponente de Lyapunov (LE) a partir de trayectorias numéricas. Se describe la arquitectura, el *stack* tecnológico, y se ofrece una documentación por script (clases, funciones, parámetros, valores de retorno y ejemplos de uso).

7.1.2. Arquitectura de Alto Nivel

El módulo se divide en cuatro componentes principales:

1. **Adaptador de dinámica gravitacional (`rebound_adapter.py`):** encapsula la creación y propagación de simulaciones usando REBOUND, con saneamiento de entradas y extracción discreta de trayectorias.
2. **Dinámica determinista (`dynamics.py`):** ofrece el campo vectorial (RHS) y el Jacobiano del sistema de dos cuerpos en 2D (CPU), y el esqueleto para aceleración GPU.
3. **Estimación de Lyapunov (`lyapunov.py`):** calcula el exponente máximo (mLCE) a partir de una simulación REBOUND (variacionales) o mediante un esquema discreto tipo Benettin con re-normalización periódica.
4. **Inicialización del paquete (`__init__.py`):** expone una API mínima y controlada para importar las clases principales.

7.1.3. Stack tecnológico

- **Python 3.x** como lenguaje principal.
- **NumPy** para cálculo numérico denso.
- **REBOUND** para integración N-cuerpos (integradores **WHFast**, **IAS15**, etc.).

- (Opcional) CuPy para acelerar kernels en GPU (interfaz en `DynamicsGPU`).

7.1.4. `rebound_adapter.py` — Adaptador REBOUND

Descripción general `ReboundSim` provee una interfaz ligera para: (i) crear una simulación a partir de masas, posiciones y velocidades iniciales; (ii) configurar integrador y sistema de unidades; (iii) integrar la dinámica y extraer una trayectoria discreta en forma de arreglo NumPy [pasos, cuerpos, 6] con (x, y, z, v_x, v_y, v_z) .

`class ReboundSim`

Propósito Gestionar la vida de una simulación REBOUND (creación, *setup*, integración discreta).

`__init__(G: float = 1.0, integrator: str = 'whfast')`

Propósito. Inicializa constantes globales de la simulación (gravitación G) y el integrador numérico por defecto.

Tabla 7.1: Parámetros de `ReboundSim.__init__`

Parámetro	Tipo	Descripción
<code>G</code>	<code>float</code>	Constante gravitacional (unidades consistentes con REBOUND).
<code>integrator</code>	<code>str</code>	Nombre del integrador (<code>whfast</code> , <code>ias15</code> , etc.).

Tabla 7.2: Valor de retorno de `ReboundSim.__init__`

Tipo	Descripción
<code>None</code>	Constructor, no retorna valor.

Listing 2 Bloque de Código de `ReboundSim`

```
class ReboundSim:
    def __init__(self, G: float = 1.0, integrator: str = "whfast") ->
        ↪ None:
        self.G = G
        self.integrator = integrator
```

Listing 3 Ejemplo de inicialización de `ReboundSim`

```
from simulacion.rebound_adapter import ReboundSim
sim_api = ReboundSim(G=1.0, integrator="whfast")
```

`setup_simulation(masses, r0, v0, move_to_com=True) -> Any`

Propósito. Construir e inicializar `rebound.Simulation` con masas y estado inicial (posiciones y velocidades), con opción de mover el sistema al centro de masa.

Tabla 7.3: Parámetros de `setup_simulation`

Parámetro	Tipo	Descripción
<code>masses</code>	<code>Iterable[float]</code>	Masas por cuerpo (≥ 2 y ≤ 3 en la configuración base).
<code>r0</code>	<code>Iterable[Iterable[float]]</code>	Posiciones (x, y, z) por cuerpo.
<code>v0</code>	<code>Iterable[Iterable[float]]</code>	Velocidades (v_x, v_y, v_z) por cuerpo.
<code>move_to_com</code>	<code>bool</code>	Si <code>True</code> , traslada al centro de masa.

Tabla 7.4: Valor de retorno de `setup_simulation`

Tipo	Descripción
<code>rebound.Simulation</code>	Simulación con partículas agregadas y lista para integrar.

Listing 4 Bloque de Código de `setup_simulation`

```
def setup_simulation(
    self,
    masses: Iterable[float],
    r0: Iterable[Iterable[float]],
    v0: Iterable[Iterable[float]],
    move_to_com: bool = True,
) -> Any:
    try:
        import rebound # type: ignore
    except Exception as e:
        raise ImportError(
            "rebound is required for ReboundSim. Use
            ↪ --install-hint for help."
        ) from e

    mass_tuple = tuple(float(m) for m in masses)
    n_bodies = len(mass_tuple)
    if n_bodies < 2 or n_bodies > 3:
        raise ValueError("ReboundSim soporta entre 2 y 3 cuerpos
            ↪ en la configuracion base.")
```

Listing 5 Ejemplo de `setup_simulation`

```

masses = (1.0, 1.0)
r0 = ((-1.0, 0.0, 0.0), (1.0, 0.0, 0.0))
v0 = ((0.0, -0.5, 0.0), (0.0, 0.5, 0.0))
sim = sim_api.setup_simulation(masses, r0, v0, move_to_com=True)

```

Propósito. Avanzar la simulación y extraer una trayectoria discreta de estados `integrate(sim, t_end: float, dt: float) -> Any` (x, y, z, v_x, v_y, v_z) por cuerpo y paso temporal.

Tabla 7.5: Parámetros de `integrate`

Parámetro	Tipo	Descripción
<code>sim</code>	<code>rebound.Simulation</code>	Contexto REBOUND válido.
<code>t_end</code>	<code>float</code>	Tiempo total a integrar.
<code>dt</code>	<code>float</code>	Paso de integración (discretización de salida).

Tabla 7.6: Valor de retorno de `integrate`

Tipo	Descripción
<code>numpy.ndarray</code>	Arreglo [pasos, cuerpos, 6] con posiciones y velocidades.

Listing 6 Bloque de Código de integrate

```

def integrate(self, sim: Any, t_end: float, dt: float) -> Any:
    """Integra el sistema y devuelve la trayectoria como arreglo
    ↪ numpy [pasos, cuerpos, 6]."""
    import numpy as _np

    if not hasattr(sim, "particles"):
        raise ValueError("Se esperaba una instancia de
        ↪ rebound.Simulation.")

    sim.dt = dt
    steps = max(1, int(_np.floor(t_end / dt)))
    n_bodies = len(sim.particles)
    traj = _np.zeros((steps, n_bodies, 6), dtype=float)
    base_t = sim.t if hasattr(sim, "t") else 0.0
    for i in range(steps):
        t_target = base_t + (i + 1) * dt
        sim.integrate(t_target)
        for j, p in enumerate(sim.particles):
            traj[i, j, 0] = float(p.x)
            traj[i, j, 1] = float(p.y)
            traj[i, j, 2] = float(p.z)
            traj[i, j, 3] = float(p.vx)
            traj[i, j, 4] = float(p.vy)
            traj[i, j, 5] = float(p.vz)
    return traj

```

Listing 7 Ejemplo de integración y extracción de trayectoria

```

traj = sim_api.integrate(sim, t_end=10.0, dt=0.5) # shape: [steps,
↪ bodies, 6]
print(traj.shape)

```

7.1.5. dynamics.py — Dinámica (CPU/GPU)

Descripción general Define la dinámica del problema de dos cuerpos en 2D (DynamicsCPU) y un esqueleto para aceleración en GPU (DynamicsGPU). El estado se ordena como

$$x = (x_1, y_1, v_{x1}, v_{y1}, x_2, y_2, v_{x2}, v_{y2}).$$

class DynamicsCPU

Propósito Proveer el RHS $\dot{x} = f(x, t)$ y el Jacobiano $J = \partial f / \partial x$ en 2D.

f (x, t, m1, m2, G) -> Any

Propósito. Evaluar el campo vectorial (posiciones y aceleraciones Newtonianas suavizadas por ε numérico) para dos cuerpos.

Tabla 7.7: Parámetros de DynamicsCPU.f

Parámetro	Tipo	Descripción
x	array-like (8,)	Estado $(x_1, y_1, v_{x1}, v_{y1}, x_2, y_2, v_{x2}, v_{y2})$.
t	float	Tiempo de evaluación (no autónomo opcional).
m1, m2	float	Masas de los cuerpos.
G	float	Constante gravitacional.

Tabla 7.8: Valor de retorno de DynamicsCPU.f

Tipo	Descripción
numpy.ndarray	Vector de dimensión 8 con $(\dot{x}_1, \dot{y}_1, \dot{v}_{x1}, \dot{v}_{y1}, \dot{x}_2, \dot{y}_2, \dot{v}_{x2}, \dot{v}_{y2})$.

Listing 8 Bloque de Código de DynamicsCPU.f

```

class DynamicsCPU:
    """Calcula el RHS y jacobiano del sistema de dos cuerpos en 2D."""

    def f(self, x: Any, t: float, m1: float, m2: float, G: float) ->
        ↪ Any:
            import numpy as _np

            eps = 1e-12
            x1, y1, vx1, vy1, x2, y2, vx2, vy2 = x
            dx = x2 - x1
            dy = y2 - y1
            r2 = dx * dx + dy * dy + eps
            r = _np.sqrt(r2)
            r3 = r2 * r

            ax1 = G * m2 * dx / r3
            ay1 = G * m2 * dy / r3
            ax2 = -G * m1 * dx / r3
            ay2 = -G * m1 * dy / r3

            return _np.array([vx1, vy1, ax1, ay1, vx2, vy2, ax2, ay2],
                ↪ dtype=float)

```

Listing 9 Ejemplo de uso de DynamicsCPU.f

```
import numpy as np
from simulacion.dynamics import DynamicsCPU

dyn = DynamicsCPU()
state = np.array([-1.0, 0.0, 0.0, -0.5, 1.0, 0.0, 0.0, 0.5],
    ↪ dtype=float)
rhs = dyn.f(state, t=0.0, m1=1.0, m2=1.0, G=1.0)
```

Propósito: Calcular la matriz Jacobiana 8×8 del sistema 2D de dos cuerpos.

Parámetros: $x, t, m1, m2, G \rightarrow Any$

Tabla 7.9: Parámetros de DynamicsCPU.jac

Parámetro	Tipo	Descripción
x	array-like (8,)	Estado en el que se evalúa el Jacobiano.
t	float	Tiempo de evaluación.
m1, m2	float	Masas de los cuerpos.
G	float	Constante gravitacional.

Tabla 7.10: Valor de retorno de DynamicsCPU.jac

Tipo	Descripción
numpy.ndarray	Matriz 8×8 con derivadas parciales $\partial f / \partial x$.

Listing 10 Bloque de Código de DynamicsCPU.jac

```

class DynamicsCPU:
    """Calcula el RHS y jacobiano del sistema de dos cuerpos en 2D."""
    def jac(self, x: Any, t: float, m1: float, m2: float, G:
        ↪ float) -> Any:
        import numpy as _np

        eps = 1e-12
        x1, y1, vx1, vy1, x2, y2, vx2, vy2 = x
        dx = x2 - x1
        dy = y2 - y1
        r2 = dx * dx + dy * dy + eps
        r = _np.sqrt(r2)
        r3 = r2 * r
        r5 = r3 * r2

        dfx_dx1 = G * (m2 * (3.0 * dx * dx / r5 - 1.0 / r3))
        dfx_dy1 = G * (m2 * (3.0 * dx * dy / r5))
        dfx_dx2 = -dfx_dx1
        dfx_dy2 = -dfx_dy1

        dfy_dx1 = G * (m2 * (3.0 * dx * dy / r5))
        dfy_dy1 = G * (m2 * (3.0 * dy * dy / r5 - 1.0 / r3))
        dfy_dx2 = -dfy_dx1
        dfy_dy2 = -dfy_dy1

        g2x_dx1 = G * (m1 * (3.0 * dx * dx / r5 - 1.0 / r3))
        g2x_dy1 = G * (m1 * (3.0 * dx * dy / r5))
        g2x_dx2 = -g2x_dx1
        g2x_dy2 = -g2x_dy1

        g2y_dx1 = G * (m1 * (3.0 * dx * dy / r5))
        g2y_dy1 = G * (m1 * (3.0 * dy * dy / r5 - 1.0 / r3))
        g2y_dx2 = -g2y_dx1
        g2y_dy2 = -g2y_dy1

        J = _np.zeros((8, 8), dtype=float)
        J[0, 2] = 1.0
        J[1, 3] = 1.0
        J[4, 6] = 1.0
        J[5, 7] = 1.0

        J[2, 0] = -dfx_dx1
        J[2, 1] = -dfx_dy1
        J[2, 4] = dfx_dx2
        J[2, 5] = dfx_dy2

        J[3, 0] = -dfy_dx1
        J[3, 1] = -dfy_dy1
        J[3, 4] = dfy_dx2
        J[3, 5] = dfy_dy2

        J[6, 0] = g2x_dx1
        J[6, 1] = g2x_dy1
        J[6, 4] = g2x_dx2
        J[6, 5] = g2x_dy2

```

Listing 11 Ejemplo de uso de DynamicsCPU.jac

```
J = dyn.jac(state, t=0.0, m1=1.0, m2=1.0, G=1.0) # shape: (8, 8)
```

class DynamicsGPU

Propósito Implementación acelerada en GPU del RHS $\dot{x} = f(x, t)$ y del Jacobiano $J = \partial f / \partial x$ para el problema de dos cuerpos en 2D usando **CuPy**. Detecta disponibilidad en `__init__`, y exige GPU vía `_require_gpu ()`. El estado sigue el mismo orden que en CPU:

$$x = (x_1, y_1, v_{x1}, v_{y1}, x_2, y_2, v_{x2}, v_{y2}).$$

Se usa un suavizado numérico $\varepsilon = 10^{-12}$ para evitar divisiones por cero en r^{-3} y r^{-5} .

Tabla 7.11: DynamicsGPU: interfaz

Miembro	Tipo/Firma	Descripción
<code>available</code>	<code>bool</code>	True si <code>cupy</code> está disponible en el entorno.
<code>_require_gpu</code>	<code>_require_gpu () -> None</code>	Lanza <code>RuntimeError</code> si no hay GPU/CuPy.
<code>f</code>	<code>f (x, t, m1, m2, G) -> cp.ndarray</code>	Campo vectorial (dim. 8).
<code>jac</code>	<code>jac (x, t, m1, m2, G) -> cp.ndarray</code>	Jacobiano (matriz 8×8).

f (x, t, m1, m2, G) -> cp.ndarray

Propósito. Evaluar el RHS en GPU para dos cuerpos en 2D, devolviendo $(\dot{x}_1, \dot{y}_1, \dot{v}_{x1}, \dot{v}_{y1}, \dot{x}_2, \dot{y}_2, \dot{v}_{x2}, \dot{v}_{y2})$.

Tabla 7.12: Parámetros de DynamicsGPU.f

Parámetro	Tipo	Descripción
<code>x</code>	<code>array-like (8,)</code>	Estado: $(x_1, y_1, v_{x1}, v_{y1}, x_2, y_2, v_{x2}, v_{y2})$.
<code>t</code>	<code>float</code>	Tiempo de evaluación (no autónomo opcional).
<code>m1, m2</code>	<code>float</code>	Masas de los cuerpos.
<code>G</code>	<code>float</code>	Constante gravitacional.

Tabla 7.13: Valor de retorno de DynamicsGPU.f

Tipo	Descripción
<code>cupy.ndarray (8,)</code>	Derivadas del estado, <code>float64</code> .

Listing 12 Bloque de código de DynamicsGPU.f

```

class DynamicsGPU:
    """Implementación del RHS y jacobiano en GPU usando CuPy."""

    def __init__(self) -> None:
        self.available = False
        try:
            import cupy as _cp # noqa: F401
            self.available = True
        except Exception:
            self.available = False

    def _require_gpu(self) -> None:
        if not self.available:
            raise RuntimeError(
                "CuPy no está disponible. Instálalo o usa DynamicsCPU."
            )

    def f(self, x: Any, t: float, m1: float, m2: float, G: float) ->
        ↪ Any:
        self._require_gpu()
        import cupy as cp

        eps = cp.float64(1e-12)
        state = cp.asarray(x, dtype=cp.float64)

        x1, y1, vx1, vy1, x2, y2, vx2, vy2 = state
        dx = x2 - x1
        dy = y2 - y1
        r2 = dx * dx + dy * dy + eps
        r = cp.sqrt(r2)
        r3 = r2 * r

        ax1 = G * m2 * dx / r3
        ay1 = G * m2 * dy / r3
        ax2 = -G * m1 * dx / r3
        ay2 = -G * m1 * dy / r3

        return cp.array([vx1, vy1, ax1, ay1, vx2, vy2, ax2, ay2],
            ↪ dtype=cp.float64)

```

Listing 13 Ejemplo de uso de DynamicsGPU.f con retroceso a CPU

```

import numpy as np
from simulacion.dynamics import DynamicsGPU, DynamicsCPU

state = np.array([-1.0, 0.0, 0.0, -0.5, 1.0, 0.0, 0.0, 0.5],
    ↪ dtype=float)
m1 = m2 = G = 1.0
t = 0.0

dyn_gpu = DynamicsGPU()
try:
    rhs_gpu = dyn_gpu.f(state, t=t, m1=m1, m2=m2, G=G) # cupy.ndarray
    print(rhs_gpu.shape)
except RuntimeError:
    # Fallback si no hay CuPy
    dyn_cpu = DynamicsCPU()
    rhs_cpu = dyn_cpu.f(state, t=t, m1=m1, m2=m2, G=G)
    print(rhs_cpu.shape)

```

jac(x, t, m1, m2, G) $\rightarrow$ cp.ndarray

Propósito: Calcular en GPU la matriz Jacobiana 8×8 , incluyendo derivadas cruzadas $(x_1, y_1)-(x_2, y_2)$ según las expresiones analíticas de $\partial(a_x, a_y)/\partial(x, y)$ para potencial newtoniano en 2D.

Tabla 7.14: Parámetros de DynamicsGPU.jac

Parámetro	Tipo	Descripción
x	array-like (8,)	Estado donde se evalúa el Jacobiano.
t	float	Tiempo de evaluación.
m1, m2	float	Masas de los cuerpos.
G	float	Constante gravitacional.

Tabla 7.15: Valor de retorno de DynamicsGPU.jac

Tipo	Descripción
cupy.ndarray (8, 8)	Jacobiano del sistema, float64.

Listing 14 Bloque de Código de DynamicsGPU.jac

```

class DynamicsGPU:
    """Implementación del RHS y jacobiano en GPU usando CuPy."""
    def jac(self, x: Any, t: float, m1: float, m2: float, G: float) ->
        Any:
            self._require_gpu()
            import cupy as cp

            eps = cp.float64(1e-12)
            state = cp.asarray(x, dtype=cp.float64)

            x1, y1, vx1, vy1, x2, y2, vx2, vy2 = state
            dx = x2 - x1
            dy = y2 - y1
            r2 = dx * dx + dy * dy + eps
            r = cp.sqrt(r2)
            r3 = r2 * r
            r5 = r3 * r2

            dfx_dx1 = G * (m2 * (3.0 * dx * dx / r5 - 1.0 / r3))
            dfx_dy1 = G * (m2 * (3.0 * dx * dy / r5))
            dfx_dx2 = -dfx_dx1
            dfx_dy2 = -dfx_dy1

            dfy_dx1 = G * (m2 * (3.0 * dx * dy / r5))
            dfy_dy1 = G * (m2 * (3.0 * dy * dy / r5 - 1.0 / r3))
            dfy_dx2 = -dfy_dx1
            dfy_dy2 = -dfy_dy1

            g2x_dx1 = G * (m1 * (3.0 * dx * dx / r5 - 1.0 / r3))
            g2x_dy1 = G * (m1 * (3.0 * dx * dy / r5))
            g2x_dx2 = -g2x_dx1
            g2x_dy2 = -g2x_dy1

            g2y_dx1 = G * (m1 * (3.0 * dx * dy / r5))
            g2y_dy1 = G * (m1 * (3.0 * dy * dy / r5 - 1.0 / r3))
            g2y_dx2 = -g2y_dx1
            g2y_dy2 = -g2y_dy1

            J = cp.zeros((8, 8), dtype=cp.float64)
            J[0, 2] = 1.0
            J[1, 3] = 1.0
            J[4, 6] = 1.0
            J[5, 7] = 1.0

            J[2, 0] = -dfx_dx1
            J[2, 1] = -dfx_dy1
            J[2, 4] = dfx_dx2
            J[2, 5] = dfx_dy2

            J[3, 0] = -dfy_dx1
            J[3, 1] = -dfy_dy1
            J[3, 4] = dfy_dx2
            J[3, 5] = dfy_dy2

            J[6, 0] = g2x_dx1
            J[6, 1] = g2x_dy1

```

Listing 15 Ejemplo de uso de DynamicsGPU.jac

```
import numpy as np
from simulacion.dynamics import DynamicsGPU

x = np.array([-1.0, 0.0, 0.0, -0.5, 1.0, 0.0, 0.0, 0.5], dtype=float)
J = DynamicsGPU().jac(x, t=0.0, m1=1.0, m2=1.0, G=1.0) # cupy.ndarray
↪ (8, 8)
```

7.1.6. lyapunov.py — Estimación de Exponente de Lyapunov

Descripción general LyapunovEstimator expone dos rutas de cálculo del mLCE: una corta (mLCE.short) y una larga (mLCE.long), ambas soportadas por un método interno que intenta, primero, la API variacional de REBOUND y, en su defecto, un esquema discreto de Benettin con re-normalización.

```
class LyapunovEstimator
```

Propósito Estimar el exponente de Lyapunov máximo (mLCE) a partir de un contexto de simulación REBOUND o de una trayectoria discreta.

```
mLCE_short (trajectory=None, x0=None, window=100.0) -> dict
```

Propósito. Estimar mLCE sobre una ventana temporal corta.

Tabla 7.16: Parámetros de mLCE_short

Parámetro	Tipo	Descripción
trajectory	Any	Simulación REBOUND o contexto discreto.
x0	Iterable[float] None	(Reservado) Semilla/estado inicial, si aplica.
window	float	Horizonte temporal (unidades de la simulación).

Tabla 7.17: Valor de retorno de mLCE_short

Tipo	Descripción
dict	{‘‘lambda’’, ‘‘series’’, ‘‘meta’’} con detalles de cómputo.

Listing 16 Bloque de Código de mLCE_short

```

class LyapunovEstimator:
    def __init__(self) -> None:
        self._cpu_dyn = DynamicsCPU()

    def mLCE_short(
        self,
        trajectory: Any = None,
        x0: Optional[Iterable[float]] = None,
        window: float = 100.0,
    ) -> Dict[str, Any]:
        try:
            lam, series, meta = self._mlce_with_context(trajectory,
                ↪ window)
            return {"lambda": lam, "series": series, "meta": meta}
        except Exception as e:
            return {"lambda": float("inf"), "series": None, "meta":
                ↪ {"impl": "error", "err": str(e)}}

```

Listing 17 Ejemplo de uso de mLCE_short

```

from simulacion.rebound_adapter import ReboundSim
from simulacion.lyapunov import LyapunovEstimator

sim_api = ReboundSim(G=1.0, integrator="whfast")
sim = sim_api.setup_simulation(
    masses=(1.0, 1.0),
    r0=(( -1.0, 0.0, 0.0), (1.0, 0.0, 0.0)),
    v0=(( 0.0, -0.5, 0.0), (0.0, 0.5, 0.0)),
)
est = LyapunovEstimator()
res = est.mLCE_short(trajectory=sim, window=100.0)
print(res["lambda"], res["meta"])

```

Proposición 18. (Estimar mLCE sobre una ventana temporal mayor (muestras más estables para diagnóstico).

Tabla 7.18: Parámetros y retorno de `mLCE_long`

Parámetro	Tipo	Descripción
<code>trajectory</code>	<code>Any</code>	Simulación REBOUND o contexto discreto.
<code>x0</code>	<code>Iterable[float] None</code>	(Reservado) Semilla/estado inicial, si aplica.
<code>window</code>	<code>float</code>	Horizonte temporal (unidades de la simulación).
Tipo de retorno		Descripción
<code>dict</code>		<code>{'lambda', 'series', 'meta'}</code> .

Listing 18 Bloque de Código de `mLCE_long`

```

class LyapunovEstimator:

    def mLCE_long(
        self,
        trajectory: Any = None,
        x0: Optional[Iterable[float]] = None,
        window: float = 1000.0,
    ) -> Dict[str, Any]:
        try:
            lam, series, meta = self._mlce_with_context(trajectory,
                ↪ window)
            return {"lambda": lam, "series": series, "meta": meta}
        except Exception as e:
            return {"lambda": float("inf"), "series": None, "meta":
                ↪ {"impl": "error", "err": str(e)}}

```

Listing 19 Ejemplo de uso de `mLCE_long`

```

res_long = est.mLCE_long(trajectory=sim, window=1000.0)
print(res_long["lambda"])

```

Notas sobre la implementación interna

Cuando la API variacional de REBOUND está disponible, se añade al simulador un sistema variacional y se integra hasta completar el horizonte `window`, extrayendo el LE. Si la API no existe o falla, se replica el sistema para un par *referencia/perturbado*, se aplica una perturbación pequeña ε al estado y, tras cada paso, se re-normaliza la separación para acumular el promedio logarítmico λ .

Listing 20 Bloque de Código de métodos auxiliares I

```
class LyapunovEstimator:
```

```

def _mlce_with_context(self, ctx: Any, window: float) ->
    ↪ Tuple[float, Optional[Any], Dict[str, Any]]:
    import numpy as _np

    try:
        import rebound # noqa: F401
    except Exception:
        raise

    if ctx is None:
        raise ValueError("Se requiere una simulacion o contexto
            ↪ discreto para estimar Lyapunov")

    if isinstance(ctx, dict) and "sim" in ctx:
        sim = ctx["sim"]
        dt = float(ctx.get("dt", 0.5))
        t_end = float(ctx.get("t_end", window))
        masses_ctx = ctx.get("masses")
    else:
        sim = ctx
        dt = 0.5
        t_end = window
        masses_ctx = None

    if masses_ctx is not None:
        masses = tuple(float(m) for m in masses_ctx)
    else:
        masses, _, _ = self._extract_rebound_state(sim)

    if dt <= 0.0:
        raise ValueError("dt debe ser positivo para el Lyapunov
            ↪ discreto")

    steps = max(1, math.ceil(t_end / dt)) if t_end is not None
    ↪ else 1

    try:
        lam = self._mlce_rebound_variational(sim, dt=dt,
            ↪ steps=steps)
        meta = {"impl": "rebound_variational_discrete", "steps":
            ↪ steps, "dt": dt, "n_bodies": len(masses), "masses":
            ↪ masses}
        return lam, None, meta
    except Exception:
        lam, series = self._mlce_benettin(sim, masses, dt=dt,
            ↪ steps=steps)
        meta = {"impl": "benettin_discrete", "steps": steps, "dt":
            ↪ dt, "n_bodies": len(masses), "masses": masses}
        return lam, series, meta

```

Listing 21 Bloque de Código de métodos auxiliares II

```
class LyapunovEstimator:

    @staticmethod
    def _extract_rebound_state(
        sim: Any,
    ) -> Tuple[Tuple[float, ...], Tuple[Tuple[float, float, float],
    ↪ ..., Tuple[Tuple[float, float, float], ...]]:
        if not hasattr(sim, "particles"):
            raise ValueError("La simulacion proporcionada no contiene
            ↪ particulas.")
        masses: list[float] = []
        positions: list[Tuple[float, float, float]] = []
        velocities: list[Tuple[float, float, float]] = []
        for p in sim.particles:
            masses.append(float(getattr(p, "m", 0.0)))
            positions.append((float(p.x), float(p.y), float(p.z)))
            velocities.append((float(p.vx), float(p.vy), float(p.vz)))
        return tuple(masses), tuple(positions), tuple(velocities)
```

Listing 22 Bloque de Código de métodos auxiliares III

```

class LyapunovEstimator:

    def _mlce_rebound_variational(self, sim: Any, dt: float, steps:
        ↪ int) -> float:
        if not hasattr(sim, "particles"):
            raise RuntimeError("Invalid REBOUND Simulation provided")
        try:
            if hasattr(sim, "add_variational"):
                sim.add_variational(1)
            elif hasattr(sim, "add_variational"):
                sim.add_variational()
            else:
                raise AttributeError("REBOUND variational API not
                    ↪ found")

            sim.dt = dt
            steps = max(1, int(steps))
            if hasattr(sim, "calculate_lyapunov"):
                t = sim.t if hasattr(sim, "t") else 0.0
                sim.integrate(t + steps * dt)
                val = sim.calculate_lyapunov()
                return float(val) * dt
            lam_sum = 0.0
            d0 = None
            base_t = sim.t if hasattr(sim, "t") else 0.0
            for i in range(steps):
                sim.integrate(base_t + (i + 1) * dt)
                if hasattr(sim, "variational") and
                    ↪ len(sim.variational) > 0:
                    try:
                        import numpy as _np

                        vv = _np.asarray(list(sim.variational[0]))
                        d = float(_np.linalg.norm(vv))
                    except Exception:
                        raise AttributeError("Unsupported variational
                            ↪ vector structure")
                    else:
                        raise AttributeError("variational not accessible")
                if d0 is None:
                    d0 = max(d, 1e-30)
                lam_sum += _np.log(max(d, 1e-30) / d0)
            return lam_sum / float(steps)
        except Exception as e:
            raise RuntimeError(f"REBOUND variational path failed: {e}")

```

Listing 23 Bloque de Código de métodos auxiliares IV

```

class LyapunovEstimator:

    def _mlce_benettin(self, sim: Any, masses: Tuple[float, ...], dt:
        ↪ float, steps: int) -> Tuple[float, Any]:
        import numpy as _np

        masses = tuple(float(m) for m in masses)
        if len(masses) < 2:
            raise ValueError("Se requieren al menos dos masas para
                ↪ estimar el Lyapunov discreto.")

        try:
            sim_ref = sim.copy()
            sim_pert = sim.copy()
        except Exception:
            extracted = self._extract_rebound_state(sim)
            masses_extracted, r0_init, v0_init = extracted
            rbs = ReboundSim(G=getattr(sim, "G", 1.0),
                ↪ integrator=getattr(sim, "integrator", "whfast"))
            sim_ref = rbs.setup_simulation(masses_extracted, r0_init,
                ↪ v0_init, move_to_com=False)
            sim_pert = rbs.setup_simulation(masses_extracted, r0_init,
                ↪ v0_init, move_to_com=False)

        eps = 1e-9
        sim_ref.dt = dt
        sim_pert.dt = dt
        sim_pert.particles[0].x += eps

    def state_vector(S: Any) -> _np.ndarray:
        data = []
        for p in S.particles:
            data.extend([p.x, p.y, p.z, p.vx, p.vy, p.vz])
        return _np.array(data, dtype=float)

    def set_state(S: Any, vec: _np.ndarray) -> None:
        for idx, p in enumerate(S.particles):
            base = idx * 6
            p.x = float(vec[base])
            p.y = float(vec[base + 1])
            p.z = float(vec[base + 2])
            p.vx = float(vec[base + 3])
            p.vy = float(vec[base + 4])
            p.vz = float(vec[base + 5])

        x_ref0 = state_vector(sim_ref)
        x_pert0 = state_vector(sim_pert)
        d0 = float(_np.linalg.norm(x_pert0 - x_ref0))
        d0 = max(d0, 1e-30)

        steps = max(1, int(steps))
        lam_sum = 0.0
        series = _np.zeros(steps, dtype=float)

        base_t = getattr(sim_ref, "t", 0.0)

```

7.1.7. `__init__.py` — API del Paquete

Descripción general Expone una interfaz pública compacta para importar las clases principales del módulo:

- `DynamicsCPU`, `DynamicsGPU`
- `ReboundSim`
- `LyapunovEstimator`

Listing 24 Bloque de Código de `mLCE_long`

```
"""
Capa de simulacion que encapsula REBOUND y el calculo del exponente de
↳ Lyapunov.
"""

from .dynamics import DynamicsCPU, DynamicsGPU # noqa: F401
from .rebound_adapter import ReboundSim # noqa: F401
from .lyapunov import LyapunovEstimator # noqa: F401

__all__ = ["DynamicsCPU", "DynamicsGPU", "ReboundSim",
↳ "LyapunovEstimator"]

if __name__ == "__main__":
    dyn = DynamicsCPU()
    print("Norma del Jacobiano con masas=1:", dyn.jac([1, 0, 0, 0, -1,
↳ 0, 0, 0], 0, 1.0, 1.0, 1.0))
```

Listing 25 Ejemplo de importación desde el paquete

```
from simulacion import DynamicsCPU, ReboundSim, LyapunovEstimator
```

En esta sección se detallan las diversas pruebas llevadas a cabo durante el desarrollo de este Proyecto. Para cada prueba se describe el objetivo que se persigue, la implementación realizada, los resultados esperados y los resultados obtenidos.

1.1. Pruebas realizadas

La primera prueba consistió en una ejecución básica de la biblioteca REBOUND. El objetivo principal fue familiarizarnos con su funcionamiento y explorar los diferentes integradores disponibles. Esto nos permitiría evaluar cuál se adapta mejor a las necesidades específicas de nuestro proyecto.

1.1.1. Primeros pasos con REBOUND

Para esto, se buscó simular la órbita de dos cuerpos con masas iguales, definiendo como condiciones iniciales a la velocidad y la posición y finalmente se utiliza matplotlib para tener una representación gráfica de las trayectorias.

Tras realizar este primer acercamiento a REBOUND, observamos que el integrador IAS15 presentaba la mejor relación entre precisión y velocidad en nuestras pruebas iniciales. Esta observación nos convenció de que sería el integrador más adecuado para utilizar en el desarrollo de nuestro proyecto. Adicionalmente, este primer contacto nos permitió familiarizarnos con la herramienta, facilitando su posterior uso a lo largo del proyecto.

Resultados

Listing 26 Ejemplo de código Rebound para simulación de dos cuerpos

```

1  import rebound
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # Crear la simulación y establecer la constante gravitacional ( $G$ 
   ↪  = 1 en unidades arbitrarias)
6  sim = rebound.Simulation()
7  sim.G = 1.0
8
9  # Definir las masas (masas similares)
10 m1 = 1.0
11 m2 = 1.0
12
13 # Para que el período del movimiento relativo sea de 1 año:
14 #  $T = 2\pi\sqrt{a_{\text{rel}}^3/(m_1+m_2)} = 1 \Rightarrow a_{\text{rel}}^3 = 1/(2\pi^2)$ 
15 a_rel = (1.0 / (2 * np.pi**2))**(1/3)
16
17 # Cada cuerpo orbita a una distancia  $r$  del centro de masas:
18 r = a_rel / 2.0
19
20 # Velocidad orbital para órbita circular:  $v = \sqrt{G/(4r)}$  (para
   ↪  cuerpos de igual masa)
21 v = np.sqrt(1.0 / (4 * r))
22
23 # Agregar las partículas con condiciones iniciales:
24 # Se colocan simétricamente respecto al origen.
25 # Cuerpo 1: posición  $(-r, 0)$  y velocidad perpendicular  $(0, -v)$ 
26 # Cuerpo 2: posición  $(r, 0)$  y velocidad  $(0, v)$ 
27 sim.add(m=m1, x=-r, y=0, z=0, vx=0, vy=-v, vz=0)
28 sim.add(m=m2, x=r, y=0, z=0, vx=0, vy=v, vz=0)
29
30 # Tras agregar las partículas, mover el sistema al centro de
   ↪  masas (opcional pero recomendable)
31 sim.move_to_com()
32
33 # Configurar el integrador (se usa "ias15" por su alta
   ↪  precisión)
34 sim.integrator = "ias15"
35
36 # Podemos definir un dt (aunque con "ias15" no es estrictamente
   ↪  necesario)
37 sim.dt = 0.001
38
39 # Definir el tiempo final de la simulación: 1 año (equivalente a
   ↪  365 días en este sistema de unidades)
40 t_final = 1.0
41
42 # Crear un arreglo de tiempos para almacenar las posiciones (por
   ↪  ejemplo, 1000 puntos a lo largo del año)
43 n_outputs = 1000
44 times = np.linspace(0, t_final, n_outputs)
45

```

.1.2. Cálculo de métricas

Como segunda prueba, nos enfocamos en el cálculo de distintas métricas que nos podrían ayudar a decidir cuando un sistema ya es estable, además de esto, para esta segunda prueba en lugar de dar todas las condiciones iniciales, solo se utiliza la masa inicial y las trayectorias que sigue cada cuerpo.

Resultados

Calculamos el semieje mayor, la excentricidad, la energía relativa y el exponente de Lyapunov y tras ver como cambiaban estos datos con respecto al tiempo, decidimos que la métrica con la que vamos a trabajar será el exponente de Lyapunov, siendo que cuando este exponente sea menor a 0.1 se considerara que el sistema se encuentra en equilibrio.

Listing 27 Ejemplo de código Rebound con cálculo de métricas

```

1  if __name__ == "__main__":
2      # Parámetros de simulación
3      t_max = 1e3 # Años
4      N_steps = 1000
5      inc_deg = 30 # Inclinação en grados para la órbita del
        ↪ planeta
6
7      # Ejecutar simulación
8      times, a_arr, e_arr, energy, a_pert, e_pert, x, y, z =
        ↪ run_simulation(t_max, N_steps, inc_deg)
9
10     # Calcular métricas
11     metrics = calculate_stability(times, a_arr, e_arr, energy,
        ↪ a_pert, e_pert)
12
13     # Generar gráficos de análisis
14     plot_analysis(times, a_arr, e_arr, energy, a_pert, e_pert,
        ↪ metrics)
15
16     # Generar animación 3D (en ventana separada)
17     animate_trajectory(x, y, z, times,
        ↪ save_path="trayectoria.gif", save_format="gif")

```

.1.3. Primeras pruebas con algoritmos bioinspirados

Ya que nos familiarizamos con REBOUND y analizamos las métricas calculadas, empezamos a bocetar como sería el uso de los algoritmos bioinspirados dentro de nuestro proyecto, por lo que realizamos una prueba rápida con el algoritmo NSGA2 como primera opción, ya que consideramos el problema como una optimización multiobjetivo.

resultados

Nos dimos cuenta de que con el uso de tecnologías como pymoo, la tarea de realizar el algoritmo de optimización se facilitaba bastante, ya que contábamos

con una estructura bastante solida y solo nos tendríamos que preocupar por las adecuaciones específicas para nuestro proyecto, además de que se obtuvo una ejecución bastante rápida, siendo de 0.3 segundos para optimizar con el cálculo de 100 generaciones de 50 individuos.

Listing 28 Sección principal del script de optimización multi-objetivo

```

1  if __name__ == "__main__":
2      # Crear la configuración de la simulación usando variables
      ↪ globales.
3      config = SimulationConfig()
4      sim_runner = SimulationRunner(config)
5
6      # Lista de funciones objetivo (buscando estabilidad, sin
      ↪ target específico)
7      objectives = [
8          objective_variation_a,
9          objective_lyapunov # Esta función requiere el runner y
      ↪ opt_vars
10     ]
11
12     # Usar la lista global de restricciones
13     constraints = GLOBAL_CONSTRAINTS
14
15     # Definir el problema de optimización.
16     problem = ModularOptimizationProblem(
17         sim_runner,
18         n_free_bodies=3, # 3 cuerpos no centrales en
      ↪ GLOBAL_BODIES_CONFIG
19         objective_funcs=objectives,
20         constraint_funcs=constraints
21     )
22
23     # Seleccionar NSGA-II como algoritmo multiobjetivo
24     algorithm = NSGA2(pop_size=50)
25
26     # Ejecutar la optimización (ajustar el criterio de
      ↪ terminación según sea necesario)
27     res = minimize(problem, algorithm, termination=('n_gen',
      ↪ 100), seed=1, verbose=True)
28
29     # Mostrar resultados
30     print("Soluciones no dominadas (condiciones iniciales):")
31     print(res.X)
32     print("\nValores de los objetivos (variación en a y lambda):")
33     print(res.F)
34     print("\nRestricciones (deben ser <= 0):")
35     print(res.G)

```

Bibliografía

- [1] S. Orlov. «C++ Playground for Numerical Integration Method Developers». En: *Supercomputing*. Ed. por V. Voevodin y S. Sobolev. Cham: Springer International Publishing, 2017, págs. 418-429. DOI: 10.1007/978-3-319-71255-0_34.
- [2] R. Juan D. Ayala P. Brunet e I. Navazo. «Object representation by means of nonminimal division quadrees and octrees». En: *ACM Trans. Graph.* 4.1 (1985), págs. 41-59. DOI: 10.1145/3973.3975.
- [3] J. Maguire y L. Woodcock. «Artificial Intelligent Molecular Dynamics and Hamiltonian Surgery». Tesis doct. Research Gate, 2005. URL: https://www.researchgate.net/profile/Leslie-Woodcock/publication/355486308_Artificial_Intelligent_Molecular_Dynamics_and_Hamiltonian_Surgery/links/617535ba0be8ec17a921a0c1/Artificial-Intelligent-Molecular-Dynamics-and-Hamiltonian-Surgery.pdf.
- [4] Michael P. Allen y Dominic J. Tildesley. *Computer Simulation of Liquids*. Oxford University Press, jun. de 2017. ISBN: 9780198803195. DOI: 10.1093/oso/9780198803195.001.0001. URL: <https://doi.org/10.1093/oso/9780198803195.001.0001>.
- [5] Peter Atkins y Julio de Paula. *Physical Chemistry*. 9th. Oxford: Oxford University Press, 2008. ISBN: 9780195685220. URL: <http://www.worldcat.org/isbn/9780195685220>.
- [6] D. Potter I. Alonso Asensio C. Dalla Vecchia y J. Stadel. «Mesh-free hydrodynamics in pkdgrav3 for galaxy formation simulations». En: *Monthly Notices of the Royal Astronomical Society* 519.1 (2022), págs. 300-317. DOI: 10.1093/mnras/stac3447.
- [7] H. Rein y S.-F. Liu. «REBOUND: an open-source multi-purpose N-body code for collisional dynamics». En: *Astronomy & Astrophysics* 537 (2012). Published online 20 January 2012, pág. 10. DOI: 10.1051/0004-6361/201118085. URL: <https://doi.org/10.1051/0004-6361/201118085>.
- [8] Dong-Hong Wu, Sheng Jin y Jason H. Steffen. «Enhanced Stability in Planetary Systems with Similar Masses». En: *The Astronomical Journal* 169.1 (dic. de 2024), pág. 28. DOI: 10.3847/1538-3881/ad9388. URL: <https://dx.doi.org/10.3847/1538-3881/ad9388>.

- [9] Hanno Rein y col. «Hybrid symplectic integrators for planetary dynamics». En: *Monthly Notices of the Royal Astronomical Society* 485.4 (jun. de 2019), págs. 5490-5497. DOI: 10.1093/mnras/stz769. arXiv: 1903.04972 [astro-ph.EP].
- [10] Jack Wisdom y Matthew Holman. «Symplectic maps for the N-body problem.» En: *The Astronomical Journal* 102 (oct. de 1991), págs. 1528-1538. DOI: 10.1086/115978.
- [11] T.J. Stuchi. «Symplectic integrators revisited». En: *Brazilian Journal of Physics* (2002). ISSN: 0103-9733. DOI: <https://doi.org/10.1590/S0103-97332002000500022>. URL: <https://www.scielo.br/j/bjp/a/NgqxKCKkBmrC93BgP4FJ4fk/?lang=en#>.
- [12] Siu A. Chin y C. R. Chen. «Forward Symplectic Integrators for Solving Gravitational Few-Body Problems». En: *Celestial Mechanics and Dynamical Astronomy* 91.3 (2005), págs. 301-322. ISSN: 1572-9478. DOI: 10.1007/s10569-004-4622-z. URL: <https://doi.org/10.1007/s10569-004-4622-z>.
- [13] David M Hernandez y Matthew J Holman. «`jsclenckehhj/scpj`: an integrator for gravitational dynamics with a dominant mass that achieves optimal error behaviour». En: *Monthly Notices of the Royal Astronomical Society* 502.1 (dic. de 2020), págs. 556-563. ISSN: 1365-2966. DOI: 10.1093/mnras/staa3945. URL: <http://dx.doi.org/10.1093/mnras/staa3945>.
- [14] Will M. Farr y Edmund Bertschinger. «Variational Integrators for the Gravitational N-Body Problem». En: *The Astrophysical Journal* 663.2 (jul. de 2007), págs. 1420-1433. ISSN: 1538-4357. DOI: 10.1086/518641. URL: <http://dx.doi.org/10.1086/518641>.
- [15] Haruo Yoshida. «Recent progress in the theory and application of symplectic integrators». En: *Celestial Mechanics and Dynamical Astronomy* 56.1 (1993), págs. 27-43. ISSN: 1572-9478. DOI: 10.1007/BF00699717. URL: <https://doi.org/10.1007/BF00699717>.
- [16] D. M. Hernandez y E. Bertschinger. «Symplectic integration for the collisional gravitational N-body problem». En: *Monthly Notices of the Royal Astronomical Society* 452.2 (2015), págs. 1934-1944. DOI: 10.1093/mnras/stv1439.
- [17] Haruo Yoshida. «Construction of higher order symplectic integrators». En: *Physics Letters A* 150.5 (1990), págs. 262-268. ISSN: 0375-9601. DOI: [https://doi.org/10.1016/0375-9601\(90\)90092-3](https://doi.org/10.1016/0375-9601(90)90092-3). URL: <https://www.sciencedirect.com/science/article/pii/0375960190900923>.
- [18] Hanno Rein, David S. Spiegel y David Tamayo. *REBOUND Documentation and Integrators*. Conceptual reference to REBOUND documentation and related papers. See H. Rein and D. S. Spiegel, "IAS15: A fast, adaptive, high-order integrator for gravitational dynamics, accurate to machine precision," MNRAS, vol. 446, pp. 1424–1437, Jan. 2015; H. Rein and D. Tamayo, "WHFAST: A fast and unbiased implementation of the Wisdom-Holman algorithm for long-term gravitational simulations," MNRAS, vol. 452, pp. 376–388, Sep. 2015. 2025. URL: <https://rebound.readthedocs.io/en/latest/integrators/>.

- [19] Alexander S. Glasser y Hong Qin. «A gauge-compatible Hamiltonian splitting algorithm for particle-in-cell simulations using finite element exterior calculus». En: *Journal of Plasma Physics* 88.2 (2022), pág. 835880202. DOI: 10.1017/S0022377822000290.
- [20] Xiongbiao Tu, Qiao Wang y Yifa Tang. *Symplectic Integrators in Corotating Coordinates*. 2022. arXiv: 2202.03779 [astro-ph.IM]. URL: <https://arxiv.org/abs/2202.03779>.
- [21] David M Hernandez. «Should N-body integrators be symplectic everywhere in phase space?». En: *Monthly Notices of the Royal Astronomical Society* 486.4 (abr. de 2019), págs. 5231-5238. ISSN: 1365-2966. DOI: 10.1093/mnras/stz884. URL: <http://dx.doi.org/10.1093/mnras/stz884>.
- [22] Hanno Rein y Daniel Tamayo. «whfast: a fast and unbiased implementation of a symplectic Wisdom–Holman integrator for long-term gravitational simulations». En: *Monthly Notices of the Royal Astronomical Society* 452.1 (jul. de 2015), págs. 376-388. ISSN: 0035-8711. DOI: 10.1093/mnras/stv1257. eprint: <https://academic.oup.com/mnras/article-pdf/452/1/376/4912268/stv1257.pdf>. URL: <https://doi.org/10.1093/mnras/stv1257>.
- [23] Ernst Hairer, Christian Lubich y Gerhard Wanner. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. 2.^a ed. Springer Series in Computational Mathematics. Springer Berlin, Heidelberg, 2006. ISBN: 978-3-540-30666-5. DOI: 10.1007/3-540-30666-8. URL: <https://link.springer.com/book/10.1007/3-540-30666-8>.
- [24] REBOUND documentation. *Integrators - WHFast*. <https://rebound.readthedocs.io/en/latest/integrators/>. Accessed: Abril 04, 2025. 2025.
- [25] REBOUND documentation. *Advanced settings for WHFast: Extra speed, accuracy, and additional forces*. https://rebound.readthedocs.io/en/latest/ipython_examples/AdvWHFast/. Accessed: Abril 04, 2025. 2025.
- [26] Stephen Boyd y Lieven Vandenbergh. *Convex Optimization*. Cambridge, UK: Cambridge University Press, 2004. ISBN: 978-0521833783.
- [27] Aaron Sidford. *MS&E 213 / CS 2690 : Bonus Chapter 1 Feasibility Problem and Cutting Plane Methods*. Course Notes, Stanford University. Document dated November 17, 2020. Nov. de 2020.
- [28] Qt Group. *Qt Group Frequently Asked Questions*. Qt Group page, last edited May 1, 2025. 2025. URL: <https://www.qt.io/faq>.
- [29] The Qt Company. *Qt Framework: Development Framework for Cross-platform Applications*. Official Qt product page. 2025. URL: <https://www.qt.io/product/framework>.
- [30] Ivan Kubara. *What is Qt framework, Why to Use It, and How?* Lemberg Solutions blog post (Embedded Engineering). 2023. URL: <https://lembergsolutions.com/blog/why-use-qt-framework>.
- [31] BusinessCloud staff. *Introduction to Qt development: A beginner's guide*. BusinessCloud news article, posted April 24, 2025. 2025. URL: <https://businesscloud.co.uk/news/introduction-to-qt-development-a-beginners-guide/>.

- [32] Qt Wiki contributors. *Qt History*. Qt Wiki page (last edited August 14, 2024). 2024. URL: https://wiki.qt.io/Qt_History.
- [33] Trolltech (archived). *Signals and Slots*. Archived Qt 3.1.2 documentation (Trolltech, 2003). 2003. URL: https://web.mit.edu/~firebird/arch/sun4x_58/doc/html/signalsandslots.html.
- [34] TutorialsPoint. *PyQt Tutorial*. Online tutorial (TutorialsPoint). 2025. URL: <https://www.tutorialspoint.com/pyqt/index.htm>.
- [35] The Qt Company. *Qt for Python: Design GUI with Python (Python Bindings for Qt)*. Official Qt page on Python bindings (PySide6). 2025. URL: <https://www.qt.io/qt-for-python>.
- [36] The Qt Company. *Qt Use Cases*. Official Qt page on industry use cases. 2025. URL: <https://www.qt.io/use-cases>.
- [37] The Qt Company. *Qt Quick Module*. Qt 6.9 Documentation (Qt Quick overview). 2025. URL: <https://doc.qt.io/qt-6/qtquick-index.html>.
- [38] Ontosight AI. *Introduction to PyQt Basics*. OntoSight AI glossary entry. 2024. URL: <https://ontosight.ai/glossary/term/introduction-to-pyqt-basics>.
- [39] Ellie Yantsan. «What is Qt – Key Features of Qt Development». En: *IT Supply Chain (Industry Talk)* (2024). Industry blog post. URL: <https://itsupplychain.com/what-is-qt-key-features-of-qt-development/>.
- [40] GeeksforGeeks. *Python GUI – PyQt vs. Tkinter*. GeeksforGeeks tutorial (last updated Dec 11, 2020). 2020. URL: <https://www.geeksforgeeks.org/python-gui-pyqt-vs-tkinter/>.
- [41] John D. Hunter. «Matplotlib: A 2D Graphics Environment». En: *Computing in Science 'I&' Engineering* 9.3 (2007), págs. 90-95. DOI: 10.1109/MCSE.2007.55.
- [42] Jake VanderPlas. *Python Data Science Handbook: Essential Tools for Working with Data*. Accessed via the O'Reilly learning platform. Released November 2016. Sebastopol, CA: O'Reilly Media, Inc., 2016. ISBN: 9781491912058. URL: <https://www.oreilly.com/library/view/python-data-science/9781491912126/ch04.html> (visitado 12-05-2025).
- [43] Matplotlib Development Team. *Matplotlib – Visualization with Python*. 2025. URL: <https://matplotlib.org/> (visitado 12-05-2025).
- [44] ActiveState. *What Is Matplotlib In Python? How to use it for plotting?* 2025. URL: <https://www.activestate.com/resources/quick-reads/what-is-matplotlib-in-python-how-to-use-it-for-plotting/> (visitado 12-05-2025).
- [45] Eric Jones, Travis Oliphant, Pearu Peterson y col. «matplotlib». En: *The Architecture of Open Source Applications, Volume II*. Ed. por Amy Brown y Greg Wilson. Accessed via AOSA website. Published March 30, 2012. Raleigh, NC: Lulu.com, 2012, pág. 390. ISBN: 9781105571817. URL: <https://aosabook.org/en/v2/matplotlib.html> (visitado 12-05-2025).

- [46] delftswa. *Matplotlib - The Python 2D Plotting Library*. 2017. URL: <https://delftswa.gitbooks.io/desosa-2017/content/matplotlib/chapter.html> (visitado 12-05-2025).
- [47] Matplotlib Development Team. *Pyplot tutorial – Matplotlib 3.10.3 documentation*. 2025. URL: <https://matplotlib.org/stable/tutorials/pyplot.html> (visitado 12-05-2025).
- [48] GeeksforGeeks. *Introduction to Matplotlib*. 2025. URL: <https://www.geeksforgeeks.org/python-introduction-matplotlib/> (visitado 12-05-2025).
- [49] Yury Zhauniarovich. *Matplotlib Graphs in Research Papers*. 2022. URL: <https://zhauniarovich.com/post/2022/2022-09-matplotlib-graphs-in-research-papers/> (visitado 12-05-2025).
- [50] llego.dev. *Interactive Plots with Matplotlib Animations in Python*. 2023. URL: <https://llego.dev/posts/interactive-plots-matplotlib-animations-python/> (visitado 12-05-2025).
- [51] Rubin H. Landau, Manuel J. Páez y Cristian C. Bordeianu. *Computational Physics: Problem Solving with Python*. 3rd. Representative textbook using Python/Matplotlib for physics examples. Wiley-VCH, 2015.
- [52] Real Python. *Exploring Astrophysics in Python With pandas and Matplotlib*. 2025. URL: <https://realpython.com/courses/astrophysics-pandas-matplotlib/> (visitado 12-05-2025).
- [53] Michael Waskom. «Seaborn: statistical data visualization». En: *Journal of Open Source Software* 6.60 (abr. de 2021), pág. 3021. DOI: 10.21105/joss.03021.
- [54] SciPy Lecture Notes Team. *1.1. Python scientific computing ecosystem*. 2025. URL: <https://scipy-lectures.org/intro/intro.html> (visitado 12-05-2025).
- [55] The Event Horizon Telescope Collaboration y col. «First M87 Event Horizon Telescope Results. I. The Shadow of the Supermassive Black Hole». En: *The Astrophysical Journal Letters* 875.1 (abr. de 2019). Matplotlib was used in the analysis pipeline, though not explicitly cited in this specific paper usually, it's a known tool used by the collaboration, pág. L1. DOI: 10.3847/2041-8213/ab0ec7.
- [56] Trung Vu y col. «Mastering data visualization with Python: practical tips for researchers». En: *PeerJ Comput Sci* 9 (ene. de 2023), e1203. DOI: 10.7717/peerj-cs.1203. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10728683/> (visitado 12-05-2025).
- [57] Matplotlib Development Team. *Customizing Matplotlib with style sheets and rcParams*. 2025. URL: <https://matplotlib.org/stable/users/explain/customizing.html> (visitado 12-05-2025).
- [58] John Garrett y col. *garrettj403/SciencePlots: Matplotlib styles for scientific plotting*. 2025. URL: <https://github.com/garrettj403/SciencePlots> (visitado 12-05-2025).

- [59] Matplotlib Development Team. *Quick start guide – Matplotlib 3.10.3 documentation*. 2025. URL: https://matplotlib.org/stable/users/explain/quick_start.html (visitado 12-05-2025).
- [60] Reddit user discussion. *Matplotlib sucks : r/bioinformatics*. Reflects user perception, not necessarily an academic source, but indicates common beginner issues. 2022. URL: https://www.reddit.com/r/bioinformatics/comments/t13510/matplotlib_sucks/ (visitado 12-05-2025).
- [61] Stack Overflow discussion. *gnuplot vs Matplotlib*. Community discussion on performance. 2009. URL: <https://stackoverflow.com/questions/911655/gnuplot-vs-matplotlib> (visitado 12-05-2025).
- [62] N. V. Kuznetsov, T. A. Alexeeva y G. A. Leonov. «Invariance of Lyapunov exponents and Lyapunov dimension for regular and irregular linearizations». En: *Nonlinear Dynamics* 85.1 (feb. de 2016), págs. 195-201. ISSN: 1573-269X. DOI: 10.1007/s11071-016-2678-4. URL: <http://dx.doi.org/10.1007/s11071-016-2678-4>.
- [63] B. Quarles y col. «The instability transition for the restricted 3-body problem: III. The Lyapunov exponent criterion». En: *Astronomy & Astrophysics* 533 (ago. de 2011), A2. ISSN: 1432-0746. DOI: 10.1051/0004-6361/201016199. URL: <http://dx.doi.org/10.1051/0004-6361/201016199>.
- [64] James L. Kaplan y James A. Yorke. «Chaotic behavior of multidimensional difference equations». En: *Functional Differential Equations and Approximation of Fixed Points*. Ed. por Heinz-Otto Peitgen y Hans-Otto Walther. Berlin, Heidelberg: Springer Berlin Heidelberg, 1979, págs. 204-227. ISBN: 978-3-540-35129-0.
- [65] N.V. Kuznetsov. «The Lyapunov dimension and its estimation via the Leonov method». En: *Physics Letters A* 380.25–26 (jun. de 2016), págs. 2142-2149. ISSN: 0375-9601. DOI: 10.1016/j.physleta.2016.04.036. URL: <http://dx.doi.org/10.1016/j.physleta.2016.04.036>.
- [66] Stephen Boyd y Lieven Vandenberghe. *Convex Optimization*. Accessed: 2025-05-12. Cambridge: Cambridge University Press, 2004. ISBN: 9780521833783. URL: <https://www.cambridge.org/gb/universitypress/subjects/statistics-probability/optimization-or-and-risk/convex-optimization?format=HB>.
- [67] Arne E. Eiben y James E. Smith. *Introduction to Evolutionary Computing*. English. 2.^a ed. Natural Computing Series. Second edition. Includes 55 b/w and 12 colour illustrations. Part of the Springer Computer Science package. © Springer-Verlag GmbH Germany, part of Springer Nature 2015. Berlin, Heidelberg: Springer, 2015, págs. XII + 287. ISBN: 978-3-662-44873-1. DOI: 10.1007/978-3-662-44874-8. URL: <https://doi.org/10.1007/978-3-662-44874-8>.
- [68] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. 1st. USA: Addison-Wesley Longman Publishing Co., Inc., 1989. ISBN: 0201157675.